ПОЛУГОДОВОЙ КУРС · МЕТОДИЧКА

Сетевые технологии

16 семинаров: от модели OSI и утилит Linux до HAProxy и программ eBPF. Теория на примерах и демонстрациях, практики и лабораторные, домашние задания с автотестами в GitHub Actions, каталог мини-исследований.

Реализации — на C и C++

Linux · iproute2 · tcpdump · Cisco Packet Tracer · HAProxy · XDP/eBPF

2026/2027 УЧЕБНЫЙ ГОД

Оглавление

1. Пояснительная записка 5

1.1. Цели и ожидаемые результаты обучения 5

1.2. Предварительные требования 6

1.3. Язык и инструменты курса 7

1.4. Как устроена эта методичка 7

1.5. Контрольные точки по блокам 8

1.6. Литература и источники 8

2. Организация курса и система оценивания 8

2.1. Формат семинара (90 минут) 8

2.2. Инфраструктура курса 9

2.3. Как сдаются домашние задания 9

2.4. Балльная система 10

2.5. Календарная карта курса 11

2.6. Академическая этика 12

2.7. FAQ студентов по инфраструктуре (первая неделя) 12

2.8. Формат ревью кода 13

3. Блок I. Модель OSI, стек TCP/IP и инструментарий 13

Семинар 1. Зачем разработчику сети: модели OSI и TCP/IP, инкапсуляция 13

Семинар 2. Инструментарий: ping, ip, ss, nc, traceroute, dig, tcpdump, Wireshark 17

4. Блок II. Физический и канальный уровни 20

Семинар 3. Физический уровень и Ethernet: кадр, MAC, коммутация, VLAN 20

Семинар 4. ARP, ICMP и архитектура доменов; STP (обзор) 23

5. Блок III. Сетевой и транспортный уровни 27

Семинар 5. IP: адресация, CIDR, подсети, фрагментация 27

Семинар 6. Маршрутизация: таблицы, longest prefix match, мини-роутер на netns 30

Семинар 7. UDP: дейтаграммы, порты, и первый протокол поверх UDP 33

Семинар 8. TCP, часть 1: заголовок, handshake, машина состояний 37

Семинар 9. TCP, часть 2: окно, контроль перегрузки, Nagle; старт hw-9 40

6. Блок IV. Корпоративные сети, изоляция и QoS 43

Семинар 10. Корпоративные сети: иерархия, VLAN, router-on-a-stick 43

Семинар 11. DHCP, NAT и ACL: адресация, выход наружу, разграничение 45

Семинар 12. QoS: приоритизация трафика, очереди, netem 46

7. Блок V. Прикладная инфраструктура: DNS, HTTP, прокси, балансировка 48

Семинар 13. DNS и HTTP: резолвинг, кэши, версии протокола, TLS 49

Семинар 14. Reverse-proxy, кэширование, HAProxy, балансировка, sticky-session 52

8. Блок VI. eBPF/XDP и сетевое ядро Linux 54

Семинар 15. eBPF и XDP: программируемое ядро 55

Семинар 16. Путь пакета в ядре Linux; итоги курса; защиты исследований 57

9. Мини-исследования: каталог тем 59

9.1. Прикладные темы: конфигурации, замеры, отчёт о сравнении 59

9.2. Смешанные темы: инструмент + небольшая разработка 61

9.3. Теоретические темы: обзор протокола «простыми словами» 62

9.4. Общие требования к отчёту и выступлению 63

10. Методические заметки преподавателю 63

10.1. Приёмы активного ведения семинара 63

10.2. Типовые заблуждения по блокам 64

10.3. Типовые ошибки в домашних заданиях 64

10.4. Инфраструктурные грабли 65

10.5. Регулировка темпа курса 65

Приложение А. Эталонный boilerplate с автотестами 66

А.1. Структура репозитория задания 66

А.2. CMakeLists.txt (полный листинг) 67

А.3. Публичные тесты tests/test_ethernet.cpp (репрезентативный фрагмент) 67

А.4. GitHub Actions: .github/workflows/ci.yml (полный листинг) 69

А.5. Шаблон README.md 69

А.6. Скрытые тесты: механика для преподавателя 70

А.7. Чек-лист преподавателя: добавить новую задачу за 30 минут 71

Приложение Б. Спецификации домашних заданий 71

Б.1. Полная спецификация hw-2: формат входа и вывода 72

Б.2. Общие требования к коду всех заданий . . . 72

Приложение В. Шпаргалка по утилитам . . . 73

Приложение Г. Литература и ресурсы . . . 74

Г.1. Книги . . . 74

Г.2. Ключевые RFC (по темам курса) . . . 75

Г.3. Онлайн-ресурсы . . . 75

Приложение Д. Демонстрационные программы . . . 75

Д.1. UDP-эхо-клиент (семинар 7, дополнение к демо 14) . . . 76

Д.2. bpftrace: сетевые однострочники (семинар 16, наблюдаемость) . . . 76

Д.3. Полный скрипт стенда netns (лабы недель 6, 9, 12, 14) . . . 77

Д.4. Userspace-загрузчик XDP со счётчиками (к hw-15) . . . 78

Приложение Е. Контрольные вопросы с эталонными ответами . . . 78

Е.1. Блоки I–II: модели, утилиты, канальный уровень . . . 79

Е.2. Блок III: IP, маршрутизация, UDP, TCP . . . 80

Е.3. Блоки IV–VI: корпоративные сети, прокси, ядро . . . 81

Е.4. Сквозные вопросы: связать блоки курса . . . 82

Приложение Ж. Глоссарий курса . . . 83

Приложение З. Чек-листы сдачи . . . 85

З.1. Чек-лист программного задания (hw-N) . . . 85

З.2. Чек-лист лабораторной работы . . . 85

З.3. Чек-лист доклада по мини-исследованию . . . 86

Приложение И. Лабораторные работы: полные постановки . . . 86

И.1. Лаба PT-1: VLAN и router-on-a-stick (неделя 10) . . . 86

И.2. Лаба PT-2: DHCP, NAT и ACL (неделя 11) . . . 87

И.3. Лаба QoS: tc/netem в netns (неделя 12) . . . 88

И.4. Лаба HAProxy: балансировка и отказоустойчивость (неделя 14) . . . 88

Приложение К. Примеры итоговых билетов . . . 89

Приложение Л. Маршрут чтения по неделям . . . 90

Приложение М. Эталонное решение hw-3 91

Приложение Н. Полные конфигурации лаб PT-1/PT-2 93

Приложение О. Мини-справочник сокетного API 95

Приложение П. Шаблон отчёта по лабораторной и исследованию 96

Приложение Р. Микродоклады: темы по неделям 96

Приложение С. Графики для отчётов: правила и шаблон 97

Приложение Т. Формы документов курса 98

Т.1. Форма заявки на мини-исследование (сдаётся до недели 12) 98

Т.2. Ведомость курса (шаблон для ассистента) 98

Т.3. Журнал сдач: что фиксируется автоматически 99

Приложение У. Учебный репозиторий курса 99

У.2. Сопровождение методички (для кафедры) 100

КУРС «СЕТЕВЫЕ ТЕХНОЛОГИИ» · 2026/2027

1. Пояснительная записка

Полугодовой курс «Сетевые технологии» посвящён тому, как устроены компьютерные сети — от медного кабеля до обратного прокси-сервера и программ eBPF в ядре Linux. Курс построен для студентов, которые уже умеют программировать на C и C++ и хотят перестать воспринимать сеть как «чёрный ящик, в котором внезапно падает соединение». Мы разбираем модель OSI и стек TCP/IP не как абстрактную классификацию, а как набор инженерных решений: каждый уровень решает свою задачу, имеет свои протоколы, алгоритмы и ограничения, и каждое из этих решений видно в реальных системах — в выводе утилит, в дампе tcpdump, в конфигурации HAProxy, в трассировке ядра.

В отличие от классического пересказа учебника Таненбаума «от первого уровня к седьмому», курс разбит на шесть тематических блоков, а каждый блок — на недельные задачи. Каждая неделя заканчивается конкретным артефактом: решённой задачей, прошедшей автотесты в GitHub Actions, выполненной лабораторной работой или отчётом по мини-исследованию. Теория на семинаре всегда подаётся через примеры, исходный код и демонстрационные программы, а не через слайды с определениями.

Методичка адресована двум читателям одновременно. Студент использует её как программу курса: в ней описано, что происходит на каждом семинаре, что читать, что повторить и что сдать. Преподаватель или ассистент использует её как сценарий ведения курса: расписан хронометраж семинара, демонстрации, вопросы для обсуждения, критерии оценивания и полные инструкции по инфраструктуре автотестов.

1.1. Цели и ожидаемые результаты обучения

По завершении курса студент должен уметь следующее. Этот список — рабочая декларация курса: каждому пункту соответствуют конкретные семинары, практики и домашние задания, а итоговое зачитывание проверяет именно эти умения, а не пересказ определений.

1. **Использовать утилиты Linux для мониторинга, анализа и конфигурирования сети**: ping, ip (link/addr/route/neigh), ss, nc, traceroute/mtr, dig, curl, tcpdump, wireshark, ethtool, tc/netem, а также элементы nftables и bpftool. Это целая глава-блок I плюс сквозная практика: в каждой лабораторной работы входит анализ трафика сниффером (семинары 1–2, приложение B).
2. **Описывать основные протоколы и алгоритмы уровней 1–4 модели OSI**: Ethernet и ARP, IPv4/IPv6 и маршрутизацию, UDP, TCP и алгоритмы управления потоком и перегрузкой. Знание проверяется не пересказом, а мини-реализациями: парсеры кадров и пакетов, ARP-кэш, таблица маршрутизации с longest prefix match, протокол поверх UDP, симулятор TCP (семинары 3–9, домашние задания hw-3…hw-9).
3. **Понимать принципы построения и изоляции корпоративных сетей**: иерархию access/distribution/core, VLAN и транки, маршрутизацию между VLAN, DHCP, NAT, ACL; уметь приоритизировать трафик (DSCP, очереди, shaping), настраивать reverse-proxy, кэширование, балансировку нагрузки и sticky-session на примере HAProxy и nginx (семинары 10–14, лабораторные

работы в Cisco Packet Tracer и Linux netns).

4. **Иметь практическое знакомство с XDP и eBPF**: понимать архитектуру (программы, верификатор, карты, JIT), собирать и запускать простейшую XDP-программу, читать вывод bpftool, знать типовые сценарии применения — фильтрация, балансировка, телеметрия (семинар 15).
5. **Объяснять устройство, задачи, протоколы, алгоритмы и ограничения каждого уровня модели OSI** и соотносить семь уровней OSI с четырьмя уровнями стека TCP/IP — включая то, почему модель TCP/IP «прижилась» на практике, а OSI осталась эталонным языком описания (семинары 1, 5–9, 16).
6. **Связывать материал курса с прикладной разработкой**: понимать путь пакета через сетевое ядро Linux, влияние настроек ядра и сети на производительность приложений (отложенная доставка, Nagle, SO_REUSEPORT, буферы, C10K), причины типовых ошибок прикладного программиста — от «залипших» соединений до неправильной работы за NAT (сквозная тема; на каждом семинаре есть врезка «Связь с разработкой»).

Список намеренно сформулирован через глаголы «использовать», «описывать», «настраивать», «реализовывать»: курс оценивает умение делать, а не знание терминов. Тем не менее теоретическая подготовка обязательна — без понимания ограничений протоколов невозможно ни выполнить лабораторные, ни защитить мини-исследование.

1.2. Предварительные требования

От студента ожидается уверенное владение C на уровне написания и отладки программ со структурами, указателями, ручным управлением памятью и побитовой арифметикой, а также базовое знание C++ (классы, std::vector, std::string) — часть тест-харнессов и примеров написана на C++. Необходимы базовые навыки работы в Linux-терминале: навигация, запуск программ, редактирование файлов. Опыт сетевого программирования и знание протоколов не требуются — курс начинает с нуля. Желательно (но не обязательно) иметь аккаунт GitHub к первому семинару: вся сдача домашних заданий построена вокруг Git и GitHub Actions.

Аппаратно достаточно ноутбука с Linux (нативно, WSL2 или виртуальная машина) с 4 ГБ ОЗУ. Для лабораторных с Packet Tracer подойдёт любой ПК; для лабораторных с netns, HAProxy и eBPF используется Linux-окружение кафедры или собственная виртуальная машина по инструкции из раздела 2.2. Программы XDP требуют ядра не старше 5.x и прав root — поэтому они выполняются в предоставляемой среде.

1.3. Язык и инструменты курса

Основной язык реализаций — C (стандарт C11) с отдельными задачами и тест-харнессами на C++17. Сборка — CMake, тесты — GoogleTest (подключается через FetchContent, вручную ставить ничего не нужно), непрерывная интеграция — GitHub Actions. Такой набор выбран сознательно: он совпадает с типовым стеком системных команд и позволяет преподавателю раздавать задачи с готовым boilerplate, а студентам — получать автоматическую проверку каждой сдачи. Сетевые лаборатории используют стандартные средства Linux (ip, tc, namespaces), эмулятор Cisco Packet Tracer, HAProxy и nginx; модуль eBPF — libbpf, bpftool и clang.

Примеры на семинарах демонстрируются вживую в терминале; все листинги из методички воспроизводимы копированием. Если демонстрация зависит от конкретной версии инструмента (например, вывод ip в разных дистрибутивах немного отличается), это отмечено в тексте. Эталонное окружение курса — Ubuntu 22.04/24.04 LTS.

1.4. Как устроена эта методичка

Документ состоит из десяти глав и восемнадцати приложений (А–Т). Главы 1–2 — организационные: цели, правила игры, оценивание, календарь. Главы 3–8 — шесть тематических блоков, по одному разделу на блок; внутри блока каждый семинар оформлен единой структурой: конспект теории с примерами и таблицами, демонстрации с готовыми листингами, практика на семинаре, самостоятельная работа, домашнее задание с интерфейсом и описанием автотестов, врезка «Связь с прикладной разработкой» и вопросы для обсуждения. Глава 9 — каталог тем мини-исследований; глава 10 — методические заметки для ведущего.

Приложения — рабочий инструментарий: А–В описывают эталонный boilerplate, спецификации заданий и шпаргалку команд; Г — литературу; Д–З — демонстрационные программы, контрольные вопросы с ответами, глоссарий и чек-листы; И–Л — полные постановки лабораторных, итоговые билеты и карту чтения; М–Т — эталонное решение, конфигурации лаб, справочник сокетного API, шаблоны отчётов и форм; У — устройство учебного репозитория. Студенту в первую неделю стоит прочитать главы 1–2 и просмотреть приложение А — остальное осваивается по ходу недель.

Три сквозных условности. Во-первых, нумерация: «неделя» = номер учебной недели семестра, «семинар» = занятие этой недели, «hw-N» — домашнее задание недели N. Во-вторых, все листинги проверены на Ubuntu 22.04/24.04; строки вывода могут отличаться на других системах. В-третьих, врезки «Связь с прикладной разработкой» — не отступления, а часть программы: курс отвечает на вопрос «зачем это программисту» каждой темой, а не только в конце.

1.5. Контрольные точки по блокам

Промежуточный самоконтроль: после каждого блока студент должен уверенно выполнять перечисленное ниже. Таблица используется и для самооценки, и как минимальный список вопросов устной защиты заданий блока.

Блок	Студент умеет
I	Читать сводку сети командами ip/ss/ping; объяснить инкапсуляцию на живом дампе; пройти полный цикл сдачи задания через GitHub Actions.
II	Разобрать Ethernet-кадр и ARP-сообщение по байтам; объяснить работу коммутатора и STP; реализовать парсер кадра и ARP-кэш с таймерами.
III	Считать подсети и LPM на бумаге; собрать маршрутизацию в netns; объяснить handshake и FSM TCP по дампу; реализовать протокол с окном и ретрансмиссиями поверх UDP.
IV	Собрать в PT сеть с VLAN, DHCP, NAT и ACL; приоритизировать трафик классами HTB и измерить эффект; прочитать и объяснить чужой конфиг сети.
V	Объяснить путь DNS-запроса и TLS-handshake; собрать HAProxy с health-check и sticky; интерпретировать 502/504 и счётчики /stats.
VI	Собрать, загрузить и снять XDP-программу; объяснить путь пакета в ядре и влияние буферов/очередей на приложение; презентовать результаты измерений.

Умения по завершении блока

1.6. Литература и источники

Базовый учебник курса — А. Таненбаум, Д. Уэзеролл, «Компьютерные сети» (6-е издание). Курс следует его логике уровней, но перераспределяет акценты: больше практики, больше Linux-специфики, меньше истории протоколов, утративших значение. Дополнительная литература по темам приведена в приложении Г: классические тома Стивенса по программированию сокетов и стеку TCP/IP, книга Куроуза и Росса как альтернативное изложение теории, документация ядра Linux по eBPF/XDP. Первоисточником по каждому протоколу являются RFC — в методичке они указаны по ходу тем, чтение выборочных разделов RFC входит в самостоятельную работу.

ФОРМАТ, ИНФРАСТРУКТУРА, БАЛЛЫ, КАЛЕНДАРЬ

2. Организация курса и система оценивания

2.1. Формат семинара (90 минут)

Каждый семинар — это не лекция с молчаливой аудиторией, а совместный разбор темы. Рекомендуемый хронометраж, выверенный на прошлых годах ведения курса, приведён ниже; отклонения возможны, но соотношение «объяснение — обсуждение — практика» сохраняется.

Минуты	Часть	Что происходит
0–10	Разбор домашки	Разбираем результаты прошлого задания: типовые ошибки, интересные решения, живой прогон одного из PR-ов с CI-логом на экране.
10–15	Микроквиз	3–4 коротких вопроса по теории прошлой недели письменно или голосованием; обсуждаются спорные ответы.
15–50	Теория с примерами	Новая тема: объяснение с примерами, исходниками, демонстрационными программами и живым дампом трафика. Конспекты тем — в главах 3–8.
50–60	Свободные вопросы	Активное обсуждение: студенты задают вопросы, преподаватель и группа разбирают кейсы «а что если…». Раздел «Вопросы для обсуждения» каждой недели.
60–85	Практика на семинаре	Упражнения из раздела «Практика на семинаре»: в парах или тройках, преподаватель помогает. Часть лабораторных (Packet Tracer) начинается здесь.
85–90	Выдача домашки	Разбор постановки нового задания, интерфейсных файлов, дедлайнов и критериев.

Хронометраж семинара

2.2. Инфраструктура курса

Всё, что нужно студенту, укладывается в четыре компонента. Во-первых, Linux-окружение: Ubuntu 22.04/24.04 (нативно, WSL2 или VM), пакеты build-essential, cmake, git, tcpdump, wireshark, netcat-openbsd, iproute2, mtr, dnsutils, haproxy, nginx, clang, libbpf-dev. Во-вторых, учебный Git-репозиторий организации, где преподаватель публикует задачи: каждая домашняя работа — отдельный репозиторий-шаблон с подключённым boilerplate (приложение А). В-третьих, GitHub Actions как среда автопроверки: тесты запускаются на каждом push и PR. В-четвёртых, Cisco Packet Tracer для лабораторных по корпоративным сетям и локальная Linux-среда с namespaces для лабораторных по QoS и прокси.

Для работы с eBPF/XDP (семинары 15) используется сервер кафедры или преднастроенная виртуальная машина: программы eBPF требуют прав root и относительно свежего ядра. GitHub Actions проверяет сборку и компилируемость таких заданий, а демонстрация работы происходит на виртуальной машине преподавателя во время консультации.

2.3. Как сдаются домашние задания

Стандартный сценарий сдачи одинаков для всех «программистских» заданий и осваивается на первой неделе (задание hw-1 — сознательно простое, его роль — отработать сам процесс):

1. Студент принимает приглашение к заданию (GitHub Classroom) или делает fork репозитория задачи себе; получает копию с готовой структурой проекта и публичными тестами.
2. Реализует функции по спецификации (файл заголовка с интерфейсом уже в репозитории) — в src/, не трогая папку tests/.
3. Локально проверяет себя: cmake -B build && cmake --build build && ctest --test-dir build — публичные тесты совпадают с локальными, «сюрпризов» в стиле «тесты не собираются» нет.

4. Делает push — GitHub Actions собирает проект матрицей gcc + clang и прогоняет тесты. Зелёная галочка — публичная часть сдана.
5. Скрытые тесты преподавателя прогоняются на проверке (механизм — в приложении А): они дополняют публичные граничными случаями и анти-чит-проверками.
6. При спорных случаях или для сложных задач (hw-9, hw-15) — короткая устная защита: студент объясняет своё решение и отвечает, «что будет, если…».

Дедлайн по умолчанию — начало следующего семинара (7 дней). Допускается одно автоматическое продление на 72 часа («поздний дедлайн») со штрафом 25% баллов. Повторная сдача после ревью возможна до конца блока без штрафа, если исправление вызвано замечаниями преподавателя. Копирование решений преследуется по разделу 2.6.

2.4. Балльная система

Итоговая оценка складывается из баллов за домашние задания, лабораторные работы, активность и мини-исследование. Максимум за семестр — 148 баллов: база 133 и до 15 бонусных. Зачёт выставляется при наборе не менее 90 баллов; при 120 и выше — зачёт «автоматом» при условии защищённого мини-исследования.

Категория	Состав	Максимум
Домашние задания с автотестами	hw-1 (4 б.) + hw-2…hw-8 (6 б. каждый) + hw-9 (10 б.) + hw-13 (8 б.) + hw-15 (8 б.)	72
Лабораторные работы	Лаба PT-1 «VLAN и router-on-a-stick» (6 б.), лаба PT-2 «DHCP, NAT, ACL» (6 б.), лаба «QoS и netem» (6 б.), лаба «HAProxy: балансировка и отказоустойчивость» (6 б.)	24
Активность	Микроквизы (до 6 б.), микродоклады и разборы у доски (до 6 б.)	12
Мини-исследование	Отчёт (до 15 б.) + выступление и ответы на вопросы (до 10 б.)	25
Бонусы	Бонусные части заданий (каждая до 3–5 б.), дополнительные вопросы на защите	15

Структура баллов

Балл за программное задание складывается из трёх частей: пройденные публичные тесты (40% стоимости задания), пройденные скрытые тесты (40%) и качество кода по ревью — читаемость, отсутствие утечек, адекватная обработка ошибок (20%). Бонусные части помечены в постановках заданий и необязательны.

Пример расчёта. Студент А. сдал всё, но с деталями: hw-1 — 4 из 4; hw-2…hw-8 — по 6 (итого 42); hw-9 — 9 из 10 (частично упали скрытые тесты на «бурю потерь»); hw-13 — 8 из 8; hw-15 — 8 из 8. Программные: 71 из 72. Лабы — 24 из 24; активность — 8 из 12; исследование — 22 из 25; бонус — 3. Итог: 71 + 24 + 8 + 22 + 3 = 128 из 148 — зачёт «автоматом» (порог 120) при защищённом исследовании. Такой же прозрачный расчёт ведётся в ведомости (приложение Т) и доступен студенту в любой момент семестра.

2.5. Календарная карта курса

Ниже — полная карта полугодия: шесть тематических блоков, шестнадцать семинаров, задание каждой недели и его вес. Дедлайн каждого задания — начало следующего семинара, если не указано иное.

Нед.	Блок	Тема семинара	Задание недели	Баллы
1	I	Зачем разработчику сети: модели OSI и TCP/IP, инкапсуляция	`hw-1: разминка CI — парсер MAC-адресов и IPv4-чексуммы`	4
2	I	Инструментарий: ping, ip, ss, nc, traceroute, dig, tcpdump, wireshark	`hw-2: «сетевой снапшот» — парсер вывода ip/ss по фикстурам`	6
3	II	Физический уровень и Ethernet: кадр, MAC, коммутация, VLAN-тег	`hw-3: парсер/сериализатор Ethernet-кадра (эталон boilerplate)`	6
4	II	ARP, ICMP, домены коллизий и широковещания, STP (обзор)	`hw-4: ARP-кэш с таймерами; лаба PT-1 — старт`	6
5	III	IP: адресация, CIDR, подсети, фрагментация	`hw-5: парсер IPv4 + подсчёт чексуммы заголовка`	6
6	III	Маршрутизация: таблицы, longest prefix match, netns-мини-роутер	`hw-6: LPM-таблица на префиксном дереве`	6
7	III	UDP: дейтаграммы, порты, чексуммы, broadcast/multicast	`hw-7: мини-TFTP с инъекцией транспорта`	6
8	III	TCP, часть 1: заголовок, handshake, разрыв, машина состояний	`hw-8: парсер TCP-заголовка + FSM-симулятор`	6
9	III	TCP, часть 2: окно, перегрузка (slow start, AIMD, CUBIC, BBR), Nagle	`hw-9: мини-TCP поверх UDP — ARQ со скользящим окном (сдача к неделе 11)`	10
10	IV	Корпоративные сети: иерархия, VLAN, транки, router-on-a-stick	`Лаба PT-1: VLAN + маршрутизация между VLAN (отчёт)`	6
11	IV	DHCP, NAT, ACL и базовый firewall	`Лаба PT-2: DHCP + NAT + ACL (отчёт); дедлайн hw-9`	6
12	IV	QoS: DSCP, очереди, shaping/policing, netem	`Лаба «QoS»: tc/netem в netns, замеры и отчёт`	6
13	V	DNS и HTTP: резолвинг, кэши, HTTP/1.1 vs HTTP/2, TLS (обзор)	`hw-13: мини-HTTP-сервер на сокетах`	8
14	V	Reverse-proxy, кэширование, HAProxy, балансировка, sticky-session	`Лаба «HAProxy»: конфигурация, отказы, замеры; ДЗ-14: конфиг под сценарий`	6
15	VI	eBPF и XDP: архитектура, верификатор, карты, кейсы	`hw-15: модификация XDP-программы (сборка в CI, демо на VM)`	8
16	VI	Путь пакета в ядре Linux; итоговый опрос; защиты исследований	`Защита мини-исследований (отчёт + выступление)`	25

Карта курса по неделям

2.6. Академическая этика

Обсуждать задачи друг с другом можно и нужно; смотреть в чужой код перед сдачей — нельзя. Разрешены: общие обсуждения постановки, обмен ссылками на документацию, помощь в отладке на сессии «вживую». Запрещены: передача файлов решения, коммиты под чужим аккаунтом, использование решений прошлых лет. Скрытые тесты и устная защита делают копирование невыгодным: список его последствий — от обнуления баллов за задание до доклада в деканат. Штатные инструменты вроде ассистентов с ИИ разрешены как способ объяснения, но сданный код студент обязан уметь построчно объяснить на защите.

2.7. FAQ студентов по инфраструктуре (первая неделя)

- **«У меня Windows — подойдёт?»** Да: WSL2 (Ubuntu) закрывает всё, кроме Packet Tracer (он ставится в Windows нативно) и XDP-лабы (выполняются на серверах кафедры).
- **«GitHub Actions бесплатен?»** Для публичных и учебных репозиторий организации — да; лимиты огромны по меркам курса. Если увидите платный счёт — вы в личном аккаунте с платным раннером: вернитесь в организацию курса.
- **«Тесты падают локально, а в CI зелёные» (или наоборот)** — почти всегда разница версий компилятора или отсутствие ASan. Запустите локально ту же матрицу: cmake -B build-clang с CC=clang.
- **«Я проспал дедлайн на 20 минут»** — время фиксируется по последнему push в main перед началом семинара; «поздний дедлайн» +72 часа со штрафом 25% действует без объяснений причин.
- **«Могу ли я сдать задачу позже и без штрафа?»** — только по уважительной причине, согласованной до дедлайна (документы из деканата). Технические проблемы «у меня сломался ноутбук в ночь сдачи» — не уважительная причина: CI и тесты гоняются с первой недели, времени достаточно.
- **«Где посмотреть, что именно проверяют скрытые тесты?»** — нигде до дедлайна; после защиты набор публикуется в репозитории курса, и разбор упавших тестов проводится на следующем семинаре.
- **«Задача больше не собирается после моих правок, CI красный, что делать?»** — читайте лог до конца: 9 из 10 случаев — предупреждение, возведённое в ошибку (-Werror), или упавший один тест с сообщением-дифом. Метод: локальный запуск именно того теста, что красный.

2.8. Формат ревью кода

Ревью каждой задачи состоит из трёх частей и занимает у ассистента 5–7 минут на работу. Первое — механика: зелёный CI, отсутствие правок интерфейса, коммиты. Второе — код: читаемость имён, обработка ошибок, отсутствие лишних копий и глобального состояния; обязательный вопрос «что вернёт функция на входе X» по одному граничному случаю из скрытых тестов. Третье — понимание: студент объясняет один фрагмент по своему выбору и один — по выбору ревьюера. Типовые комментарии ревьюера оформляются по шаблону «что вижу → что ожидал → как исправить» и публикуются (обезличенно) в вики курса: за год они собираются в качественный сборник типовых ошибок, который мы используем в главе 10.

НЕДЕЛИ 1–2 · ПОСТАВИТЬ КАРТУ МИРА И ДАТЬ ИНСТРУМЕНТЫ

3. Блок I. Модель OSI, стек TCP/IP и инструментарий

Первый блок решает две задачи. Во-первых, студент получает «карту мира»: представление об уровнях модели OSI и стека TCP/IP, о том, как данные инкапсулируются при движении вниз по стеку и декапсулируются при движении вверх. Во-вторых — набор инструментов, которым он будет пользоваться все оставшиеся полгода: утилиты мониторинга и анализа трафика Linux. Обе недели насыщены живыми демонстрациями: с этого момента всё, что мы обсуждаем, можно увидеть в терминале или в Wireshark.

Семинар 1. Зачем разработчику сети: модели OSI и TCP/IP, инкапсуляция

Семинар открывает курс вопросом «что происходит между вашим send() и чужим recv()». Мы показываем, что сетевой стек — это слоёная система, в которой каждый слой решает конкретную задачу и пользуется услугами слоя ниже. Разговор намеренно идёт от проблем прикладного программиста, а не от истории протоколов: модель нужна не для заучивания, а чтобы язык «это проблема третьего уровня» стал рабочим инструментом.

Конспект теории

Модель OSI ISO описывает сеть как семь уровней: физический (передача битов средой), канальный (доставка кадров между соседними узлами), сетевой (доставка пакетов между конечными узлами через промежуточные), транспортный (доставка сообщений между процессами с нужной надёжностью), сеансовый, представления и прикладной. Ключевая идея — **инкапсуляция**: данные прикладного уровня дополняются заголовками каждого уровня сверху вниз, а принимающая сторона последовательно снимает эти заголовки. Каждому уровню соответствует свой блок данных: данные, сегменты (TCP), пакеты (IP), кадры (Ethernet), биты (физика).

Стек TCP/IP, по которому реально работает Интернет, группирует те же задачи в четыре уровня: канальный (link), сетевой (internet), транспортный и прикладной. Сеансовый и уровень представления

OSI «растворились» в приложениях и библиотеках: TLS — это уровень представления, живущий внутри прикладного стека; сериализация — задача приложения. Именно это несоответствие — главный источник путаницы у новичков, поэтому на семинаре мы составляем таблицу соответствия уровней и примеров протоколов.

OSI	Задача уровня	Типовые протоколы / технологии	TCP/IP
7	Семантика приложений	HTTP, DNS, SSH, SMTP, TLS (как надстройка)	Прикладной
6	Формат, кодирование, шифрование	TLS, JSON/Protobuf (внутри приложений)	Прикладной
5	Диалог, сессии	SIP, NetBIOS, сокеты-сессии ОС	Прикладной
4	Доставка между процессами	TCP, UDP, QUIC	Транспортный
3	Доставка между узлами, маршрутизация	IPv4, IPv6, ICMP, OSPF/BGP	Сетевой
2	Кадры, адресация соседей, медиа-доступ	Ethernet, 802.1Q, ARP, Wi-Fi MAC	Канальный
1	Биты и среда передачи	10BASE-T, оптика, радиоканалы, PHY	Канальный

Соответствие уровней OSI и TCP/IP

На примере одного HTTP-запроса мы прослеживаем полную инкапсуляцию: строки запроса превращаются в сегмент TCP (заголовок с портами и номерами последовательности), сегмент — в пакет IP (адреса отправителя и получателя, TTL), пакет — в кадр Ethernet (MAC-адреса, тип 0x0800), кадр — в биты на проводе. Каждый заголовок — это договор: что записал отправитель, то обязан понять получатель соответствующего уровня. Эта механика — фундамент всех последующих домашних заданий: половина из них — «миниатюры» обработки заголовков.

Отдельно проговариваем терминологию, без которой невозможно читать дампы: MAC-адрес — адрес «следующего соседа», IP-адрес — адрес «конечного получателя», порт — адрес программы внутри узла. Правило выбора маршрута по MAC определяется шлюзом по умолчанию; эта асимметрия (MAC меняется на каждом хопе, IP — нет) будет подробно разобрана на семинаре 6, но заложить её нужно уже сейчас.

Демонстрации на семинаре

Демонстрация ведётся в терминале в трёх окнах: браузер/curl, tcpdump, отдельный терминал. Цель — показать, что «уровни» — не схема из учебника, а наблюдаемые структуры данных.

Демо 1. Первый взгляд на сеть: интерфейсы, адреса, маршруты, соседи

```
# Краткая сводка по интерфейсам: имя, состояние, скорость
ip -br link
# Адреса в компактном виде: интерфейс, IPv4/IPv6, состояние
ip -br addr
# Таблица маршрутизации: куда идёт трафик по умолчанию
ip route show
# Кто «сосед» по канальному уровню: таблица ARP/NDP
ip neigh show
```

```
# Один эхо-запрос до DNS Google с подробностями TTL
ping -c 1 8.8.8.8
```

Демо 2. Смотрим на ping сниффером: icmp-заголовок + ip-заголовок в одном кадре

```
# В одном терминале — снифер (любой интерфейс, только ICMP, 4 пакета):
sudo tcpdump -eni any -c 4 icmp
# Во втором — сам ping:
ping -c 2 8.8.8.8
# Пример строки вывода tcpdump (поясняем по полям):
# 19:04:12.882331 eth0  Out ifindex 2 00:1a:2b:3c:4d:5e >
#   a4:2b:b0:1c:9e:70, ethertype IPv4 (0x0800), length 98:
#   192.168.1.10 > 8.8.8.8: ICMP echo request, id 3, seq 1, length 64
```

Разбирая строку дампа, преподаватель последовательно указывает: MAC-адреса (канальный уровень), ethertype (метка «внутри — IPv4»), IP-адреса и TTL (сетевой), тип ICMP и идентификаторы (эхо-протокол). Студенты видят два «слоя заголовков» в одном кадре — инкапсуляция перестаёт быть схемой.

Практика на семинаре (в парах)

- Определить у своей машины: имя основного интерфейса, его IPv4 с маской, MAC-адрес, шлюз по умолчанию и «первый» DNS-сервер (resolvectl status или /etc/resolv.conf).
- Запустить ping до шлюза и до 8.8.8.8, сравнить RTT; объяснить, почему RTT до шлюза порядка единиц миллисекунд, а до 8.8.8.8 — десятков.
- В Wireshark (или tcpdump -X) снять один ICMP-кадр и вручную, по байту, найти поля Ethernet, IPv4 и ICMP; зарисовать схему инкапсуляции.
- Обсудить: во сколько заголовков обёрнута строка «GET / HTTP/1.1», если сайт открывается по HTTPS? (Ответ подводит к TLS как уровню представления.)

Самостоятельная работа до семинара 2

- Таненбаум, гл. 1: разделы о модели OSI, стеке TCP/IP и сетевой стандартизации.
- man 8 ip, man 8 ping — прочитать секции SYNOPSIS и примеры; запустить каждую команду демо 1 у себя.
- Установить недостающие пакеты из раздела 2.2 и убедиться, что tcpdump и nc работают под вашим пользователем (с sudo).

Домашнее задание hw-1: разминка CI (4 балла, дедлайн — неделя 2)

Задача минимальна по содержанию, но проходит полный жизненный цикл сдачи: clone, реализация, локальный ctest, push, зелёный CI. Студент реализует три функции в файле src/hw1.c по заголовку include/hw1.h: парсер MAC-адреса, форматтер MAC-адреса и вычисление контрольной суммы IPv4-заголовка по RFC 1071. Это последний раз, когда boilerplate описан в методичке так подробно; далее формулировки короче, а полный эталон инфраструктуры приведён в приложении А.

include/hw1.h — интерфейс задания hw-1

```
#ifndef HW1_H
#define HW1_H
```

```
#include <stddef.h>
#include <stdint.h>
/* Разбирает MAC-адрес формата AA:BB:CC:DD:EE:FF.
 * Допустимы разделители ':' и '-', регистр любой.
 * Возвращает 0 при успехе (и заполняет out[6]), -1 при любой ошибке:
 * неверная длина, нешестнадцатеричный символ, NULL-указатель. */
int parse_mac(const char *str, uint8_t out[6]);
/* Пишет MAC в виде AA:BB:CC:DD:EE:FF (заглавные) + '\0'. out >= 18 байт. */
void format_mac(const uint8_t mac[6], char *out /* [18] */);
/* Контрольная сумма заголовка IPv4 по RFC 1071: дополнение до единицы
 * суммы дополнений 16-битных слов. Поле checksum в hdr считается
 * равным нулю; len чётное (IPv4-заголовок всегда кратен 4 байтам). */
uint16_t ipv4_checksum(const uint8_t *hdr, size_t len);
#endif /* HW1_H */
```

- **Что проверяют автотесты.** Публичные: канонические форматы, смешанные разделители, подстрочные case, эталонный заголовок из RFC 1071 (пример с чексуммой 0xB861), заголовок без поля checksum. Скрытые: NULL-указатели, обрезанные строки, строки с лишними символами, чексумма на «нечётном» буфере (требование: вернуть корректную сумму по байту-заполнителю нулём).
- **Критерии.** Тесты 80%, качество кода 20% (без malloc, без UB, понятные имена).
- **Бонус (+2 б.)**: функция ipv4_checksum_odd() для нечётной длины и тест к ней.

> **Связь с прикладной разработкой**
> Именно на этом уровне живут ошибки, которые неделями мучают прикладные команды: «приложение не видит сервис», «подключение падает раз в N минут», «в Docker-сети всё медленнее». Умение открыть tcpdump и прочитать, что реально уходит в сеть, экономит дни отладки. Чексумма из hw-1 — это не академизм: её алгоритм повторяется в IP, TCP, UDP, и ее программные реализации (RFC 1071) до сих пор встречаются в коде драйверов и ускорителях NIC.

Вопросы для обсуждения на семинаре

- Почему «чистая» OSI не стала рабочим стеком, а осталась справочной моделью? Что выиграл TCP/IP, начав с практики?
- К какому уровню OSI вы отнесёте HTTP/2-мультиплексирование? А QUIC, работающий поверх UDP?
- Где проходит граница обязанностей между ОС и приложением? Почему сокет — «стык» уровней 4 и 7?
- Если MTU канала уменьшится вдвое, что изменится для прикладного программиста? (Мостик к теме фрагментации семинара 5.)

Семинар 2. Инструментарий: ping, ip, ss, nc, traceroute, dig, tcpdump, Wireshark

Второй семинар полностью посвящён «рукам» сетевого инженера — утилитам Linux. Логика подачи: для каждой утилиты мы сначала формулируем вопрос (сколько интерфейсов? кто слушает порт? где теряются пакеты?), затем отвечаем командой. К концу семинара у студентов должен сложиться рабочий словарь: задача — инструмент. Полная шпаргалка по всем командам курса вынесена в приложение В.

Конспект теории: карта инструментов

Инструменты курса делятся на четыре группы по назначению. **Диагностика достижимости**: ping (ICMP echo, измеряет RTT и потери), traceroute/mtr (показывают путь и его нестабильность). **Конфигурация и состояние**: ip (интерфейсы, адреса, маршруты, соседи), ss (сокеты и слушающие порты), ethtool (физика линка). **Клиенты протоколов**: nc (сырые TCP/UDP-соединения — «швейцарский нож»), dig (DNS), curl (HTTP). **Анализ трафика**: tcpdump для терминала и Wireshark для интерактивного разбора. Идея курса: анализ всегда идёт в паре с конфигурацией — мы не только смотрим, но и меняем сеть, и сразу видим эффект.

Вопрос	Команда
Какие у машины интерфейсы, адреса, MAC?	ip -br link; ip -br addr
Куда уйдёт пакет до адреса X?	ip route get X
Кто сосед по L2, есть ли запись в ARP?	ip neigh show
Какие порты слушает система?	ss -tlnp (TCP), ss -ulnp (UDP)
Доступен ли узел, каков RTT и потери?	ping -c 10 X; mtr -rwbc 100 X
Где именно теряется/тормозит путь?	traceroute -n X; mtr -w X
Какой IP за именем, какие записи DNS?	dig +noall +answer example.com
Открыт ли TCP-порт, слушает ли сервис?	nc -vz X 443; ss -tlnp | grep :443
Что реально уходит в сеть?	sudo tcpdump -eni any -c 100; Wireshark
Как поговорить с сервисом вручную?	nc host port (TCP), nc -u host port (UDP)

Задача — инструмент

Особое внимание уделяем tcpdump — единственному инструменту, который сопровождает весь курс. Мы вводим фильтры BPF (tcp, udp, icmp, port 53, host X, net Y, and/or/not), флаги -i (интерфейс), -n (не резолвить имена), -e (уровень Ethernet), -c (счётчик), -w/-r (запись и чтение pcap). Правило гигиены: дамп снимается с точным фильтром и в обе стороны интересующего обмена, иначе глаз «плывёт» в тысячах посторонних пакетов.

Демонстрации на семинаре

Демо 3. Ping + tcpdump: полный цикл «отправили — увидели»

```
# Терминал 1: только ICMP от/к 8.8.8.8, не резолвить имена
sudo tcpdump -n -c 4 'icmp and host 8.8.8.8'
# Терминал 2:
```

```
ping -c 2 8.8.8.8
```

Демо 4. nc как HTTP-клиент: HTTP — это просто текстовый протокол

```
# Ручной HTTP-запрос в чистом виде; Connection: close — чтобы сервер закрыл соединение
printf 'GET / HTTP/1.1\r\nHost: example.com\r\nConnection: close\r\n\r\n' \
  | nc example.com 80
# Тот же обмен глазами tcpdump: видно handshake, PSH-пакет с запросом, FIN
sudo tcpdump -n -A -c 12 'host example.com and tcp port 80' &
```

Демо 5. traceroute: как TTL выстраивает путь

```
traceroute -n 8.8.8.8
# Объясняем механику: пакет с TTL=1 гаснет на первом роутере, тот шлёт
# ICMP time exceeded, узнав адрес хопа, повторяем с TTL=2, и так далее.
# Строка "* * *" = узел не отвечает на ICMP, это НЕ значит «пакет потерян».
```

В демо 5 обязательно проговариваем типовой заблуждение: строки «* * *» в traceroute часто означают лишь «узел не отвечает на ICMP», а не проблему пути. То же — про разницу между «нет ответа» и «нет маршрута», между таймаутом и отказом соединения: эти различия студент будет видеть всю оставшуюся жизнь в логах приложений.

Практика на семинаре: сетевой квест

Пары получают 12 заданий, ответы на которые добываются только инструментами (обе машины в одной учебной VLAN). Побеждает пара, первой собравшая корректный отчёт; затем преподаватель разбирает типовые ошибки по каждому пункту. Полный список:

№	Задание	Ожидаемый инструмент
1	Найти MTU основного интерфейса и скорость линка	ip link show; ethtool
2	Определить полное DNS-имя своей машины и домен поиска	hostname -f; resolvectl status
3	Какой процесс слушает порт 22 (PID и имя)?	ss -tlnp
4	Чем отличается соединение на закрытый порт от фильтруемого?	nc -vz: refused против timeout
5	Снять дамп одного DNS-запроса и найти вопрос/ответ	tcpdump -n port 53 + dig
6	Сколько хопов до узла X и где маршрут расходится со вторым узлом?	traceroute -n; mtr -w
7	Найти MAC шлюза и тип записи в ARP-кэше	ip neigh show
8	Отправить HTTP-запрос вручную и процитировать заголовки сервера	printf | nc host 80
9	Определить версию TLS и ALPN выбранного сайта	openssl s_client -brief
10	Какой интерфейс «выбрал» бы стек для 8.8.8.8 и с каким src?	ip route get 8.8.8.8
11	Снять icmp-кадр и найти поля L2/L3/L4 по байтам (в Wireshark)	tcpdump -c1 -w; wireshark
12	Ответить: почему traceroute показывает «* * *» на последнем хопе?	Обоснованное объяснение

Задания сетевого квеста недели 2

Самостоятельная работа до семинара 3

- Пройти квест-задания, не успевшие на семинаре; оформить ответы в gist/файл.
- Прочитать man tcpdump (секции BPF-фильтров), man ss; повторить демо 3–5 у себя.
- Таненбаум, гл. 3 (канальный уровень, разделы о Ethernet и MAC) — до семинара 3.

Домашнее задание hw-2: «сетевой снапшот» (6 баллов, дедлайн — неделя 3)

Студент пишет утилиту netshot: она читает файл-фикстуру с упрощённым выводом обследования машины (формат строго задан ниже) и печатает сводку в фиксированном формате. Задание учит точному парсингу и форматированию — навыку, без которого невозможны следующие задания на парсеры заголовков. Полная спецификация входа и вывода — в приложении Б, раздел Б.2; интерфейс:

include/hw2.h — интерфейс задания hw-2

```
#ifndef HW2_H
#define HW2_H
#include <stddef.h>
/* Читает фикс-файл (строки формата iface/route/listen, см. спецификацию)
 * и печатает сводку в stdout в каноническом формате:
 *   interfaces=<N>            — число интерфейсов со state=up
 *   addr=<iface> <ip>/<plen>  — по строке на каждый адрес, в порядке входа
 *   routes=<N>                — всего маршрутов (включая default)
 *   listen_tcp=<N>            — число слушающих TCP-сокетов
 * Возвращает 0 при успехе, -1 при ошибке чтения/формата. */
int netshot_report(FILE *in, FILE *out);
#endif /* HW2_H */
```

- **Что проверяют автотесты.** Публичные: пустой ввод, один интерфейс down, несколько адресов на интерфейсе, default-маршрут, дубли портов. Скрытые: битые строки формата (требование — пропускать их, не падать), CRLF-концы строк, большие объёмы (10 тысяч строк — утилита обязана быть потоковой, без ограничения на размер).
- **Критерии.** Тесты 80%, стиль 20%: без fscanf-подобных «прыжков», парсинг по строкам (getline/fgets + разбор), понятные сообщения об ошибках в stderr.

> **Связь с прикладной разработкой**
> Вывод ss -tlnp и ip route get — первые команды, которые просят приложить в issue на «сервис недоступен»: в 8 случаях из 10 проблема видна сразу (порт не слушается, маршрут в правильную сеть отсутствует, интерфейс down). Утилита netshot — уменьшенная копия инвентаризационных скриптов, которые пишут SRE-команды для сбора состояния тысяч машин; правила «строгий формат, потоковая обработка, устойчивость к мусору» — те же.

Вопросы для обсуждения на семинаре

- Почему nc полезнее «настоящего» HTTP-клиента при отладке протокола? Что вы теряете и что находите, опустившись на уровень ниже curl?
- Почему tcpdump требует прав root (CAP_NET_RAW)? Какие ещё возможности системы так защищены?

- Как по выводу mtr отличить потерю на последнем хопе (проблема ответов ICMP) от потери по пути (реальные потери)?
- Что изменится в демо 4, если сервер за балансировщиком с двумя сетевыми картами? (Подводка к reverse-proxy семинара 14.)

НЕДЕЛИ 3–4 · ETHERNET, MAC, ARP, ICMP, STP

4. Блок II. Физический и канальный уровни

Второй блок опускается на «дно» сетевого стека. Студент видит, как биты физически превращаются в кадры, как кадры адресуются MAC-адресами и как коммутатор их раскладывает по портам; учится читать и собирать кадры вручную; разбирает протокол ARP — первый «живой» протокол, который курс реализует в миниатюре. Завершают блок ICMP, домены коллизий и широковещания и обзор STP — протокола, который спасает Ethernet-сети от петель. Домашние задания этого блока открывают серию парсеров заголовков, к которым студент будет возвращаться весь семестр.

Семинар 3. Физический уровень и Ethernet: кадр, MAC, коммутация, VLAN

Физический уровень рассматриваем быстро и практично: среда передачи (медь, оптика, радио), понятие сигнала и кодирования, дуплекс и автосогласование, топологии звезда и шина. Главная цель — снять ореол таинственности с кабеля: 1000BASE-T — это просто договорённость о том, как группа битов кодируется напряжением/символами в линке, а «линк встал» означает успешные физический и канальный этапы согласования. Полезная проверка физики на любой машине — ethtool: скорость, дуплекс, состояние линка.

Конспект теории

Канальный уровень решает три задачи: **медиа-доступ** (кто сейчас может передавать, исторически — CSMA/CD в половинном дуплексе, сегодня практически всегда полный дуплекс на витой паре), **адресация соседей** (MAC-адреса) и **обнаружение ошибок** (контрольная сумма FCS). MAC-адрес — 48 бит; старшие 24 бита — OUI, закреплённый за производителем, младшие — серийный номер; старший бит адреса определяет unicast/multicast, а FF:FF:FF:FF:FF:FF — широковещание. Ethernet-кадр устроен так:

Структура кадра Ethernet (без преамбулы и SFD)

```
0                    6                    12      14                 1500
+--------------------+--------------------+-------+------------------+------
|  MAC получателя (6) |  MAC отправителя(6) | Ether |   Поле данных     | FCS
|                    |                    | Type  |  46 ... 1500 байт | (4)
+--------------------+--------------------+-------+------------------+------
^ Алгоритм: получатель видит свой MAC (или broadcast) -> принимает кадр,
  проверяет FCS, читает EtherType (0x0800=IPv4, 0x0806=ARP, 0x86DD=IPv6),
  и отдаёт поле данных стеку соответствующего протокола.
```

Минимальный кадр — 64 байта — наследие половинного дуплекса: передатчик обязан ещё передавать кадр, чтобы успеть заметить коллизию, отсюда и ограничение 1500 байт сверху (стандартный MTU). Коммутатор — устройство канального уровня: он строит таблицу «MAC — порт», выучивая адреса из кадров-источников; неизвестный или broadcast-кадр уходит во все порты (flooding). Хаб, в отличие от коммутатора, повторяет сигнал во все порты безадресно и создаёт один домен коллизий — на семинаре мы показываем это в Packet Tracer, переключая устройства режимом Simulation и наблюдая, кто кого «перекричал».

VLAN (IEEE 802.1Q) — деление одного физического коммутатора на несколько изолированных канальных сетей: в кадр вставляется четырёхбайтовый тег (TPID 0x8100, приоритет PCP, флаг DEI, идентификатор VID 1–4094). Порт access принадлежит одному VLAN и отдаёт хосту нетегированный трафик; порт trunk (магистраль) пропускает тегированные кадры многих VLAN — так связываются коммутаторы. VLAN — главный инструмент изоляции в корпоративных сетях; в лабораторной PT-1 (неделя 10) студент соберёт их руками, поэтому сейчас важно лишь понять механику тега.

Важно понимать принцип **сквозного владения заголовками**: поле EtherType служит «приёмником-диспетчером» кадра. Для программиста-практика это значит, что любой парсер пакетов (включая ваш будущий hw-3 и модули eBPF) начинается с одинаковой процедуры: прочитать 14 байт, определить тип нагрузки, передать дальше. Мы подробно разбираем это на листинге демонстрационного парсера.

Демо-парсер кадра на Python (используется на семинаре, 25 строк)

```
#!/usr/bin/env python3
# frame_demo.py: разбирает кадр из hex-строки (выдача tcpdump -xx)
import sys
def mac(b):  return ':'.join(f'{x:02X}' for x in b)
def parse(frame: bytes):
    dst, src, etype = frame[0:6], frame[6:12], int.from_bytes(frame[12:14], 'big')
    vlan = None
    off = 14
    while etype in (0x8100, 0x88A8) and len(frame) >= off + 4:   # 802.1Q / QinQ
        tci = int.from_bytes(frame[off:off+2], 'big')
        vlan = (tci >> 13, tci & 0x0FFF)                        # (PCP, VID)
        etype = int.from_bytes(frame[off+2:off+4], 'big')
        off += 4
    print(f'dst={mac(dst)} src={mac(src)} type=0x{etype:04X} vlan={vlan}')
    print(f'payload={len(frame)-off} B  head={frame[off:off+8].hex()}')
parse(bytes.fromhex(sys.argv[1]))
```

Демонстрации на семинаре

Демо 6. Кадр глазами tcpdump: -e показывает MAC-уровень

```
sudo tcpdump -eni any -c 2 -xx arp
# параллельно на другом узле сегмента: arping 192.168.1.1
# в выводе: MAC-адреса, ethertype 0x0806, затем hex-дамп кадра целиком
# hex-дамп скармливаем frame_demo.py и сверяем разбор поле за полем
```

Демо 7. Packet Tracer: коммутатор учится (режим Simulation)

```
Топология: PC-A, PC-B, PC-C -> 2960. Пошагово:
1) ping с PC-A на PC-B: первый кадр коммутатор флудит во все порты
   (MAC-таблица пуста), PC-C «игнорирует» кадр не для себя;
2) показываем MAC-таблицу: show mac address-table (CLI коммутатора);
```

```
3) второй ping уже идёт точечно: запись в таблице выучена;
4) меняем PC-C на hub, повторяем: коллизии в Simulation-режиме.
```

Практика на семинаре

- Ручной разбор: каждая пара получает hexdump кадра (выдача преподавателя) и заполняет таблицу полей: MAC-получатель/отправитель, EtherType, длина нагрузки; два кадра с VLAN-тегом — дополнительно PCP и VID.
- Проверить у себя ethtool eth0 (скорость, дуплекс, автосогласование) и соотнести с выводом ip -br link (состояние UP).
- В Packet Tracer собрать топологию демо 7, прогнать режим Simulation и сделать скриншот MAC-таблицы до/после ping.
- Разобраться, что произойдёт с кадром, если получатель не найден ни в одной MAC-таблице на пути (обсуждение с рисунка на доске).

Самостоятельная работа до семинара 4

- Таненбаум, гл. 3: разделы «Канальный уровень», «Ethernet», «Коммутаторы и VLAN».
- Снять свой дамп кадра ARP (tcpdump -xx -c 1 arp) и разобрать его frame_demo.py.
- Прочитать IEEE 802.1Q-часть любого конспекта о VLAN и ответить себе: чем access-порт отличается от trunk-порта в терминах тегов?

Домашнее задание hw-3: парсер Ethernet-кадра (6 баллов, дедлайн — неделя 4)

Первая «взрослая» задача курса: реализовать на C парсер и сериализатор Ethernet-кадра с поддержкой 802.1Q. Это **эталонная задача курса**: с ней студент получает полный boilerplate-репозиторий (структура, CMakeLists.txt, GoogleTest-тесты, GitHub Actions с матрицей компиляторов) — все листинги приведены в приложении А, и именно их студент копирует при следующих задачах, заменяя только предметную часть. Интерфейс:

include/ethernet.h — интерфейс задания hw-3

```
#ifndef ETHERNET_H
#define ETHERNET_H
#include <stddef.h>
#include <stdint.h>
#define ET_PAYLOAD_MIN 46
#define ET_PAYLOAD_MAX 1500
/* Результат разбора кадра. Для 802.1Q поле vlan >= 0 и хранит VID;
 * приоритет PCP возвращается отдельно. Для QinQ заполняется outer_vlan. */
typedef struct {
    uint8_t  dst[6];
    uint8_t  src[6];
    uint16_t ethertype;      /* тип нагрузки: 0x0800, 0x0806, ... */
    int      vlan;           /* -1, если тега нет */
    int      pcp;            /* приоритет 0..7, 0 если тега нет */
    int      outer_vlan;     /* внешний VID для QinQ, иначе -1 */
    const uint8_t *payload;  /* указатель ВНУТРЬ входного буфера */
    size_t   payload_len;    /* 46..1500 после проверки минимальной длины */
```

```
} eth_frame_t;
/* Разбирает кадр по буферу. Возвращает 0 при успехе, -1 при ошибке:
 * NULL, слишком короткий кадр (<14), payload вне [46,1500]
 * (кадр 1514 = 14 + 1500 максимум, FCS в буфере не передаётся). */
int eth_parse(const uint8_t *buf, size_t len, eth_frame_t *out);
/* Собирает кадр в buf (ёмкость len). Для vlan < 0 тег не пишется.
 * Возвращает итоговый размер кадра, -1 при нехватке места/ошибке. */
int eth_serialize(const eth_frame_t *f, uint8_t *buf, size_t len);
#endif /* ETHERNET_H */
```

- **Что проверяют автотесты.** Публичные: канонический ARP-кадр, IPv4-кадр, кадр с одним VLAN-тегом, ошибки на коротких буферах, round-trip parse(serialize(x)) == x. Скрытые: QinQ (два тега), граничные длины 46/1500, инъекция «второго тега» в payload при отсутствии флага, проверка что payload указывает внутрь исходного буфера (запрещено копировать).
- **Критерии.** Тесты 80%, код 20%: строгая работа с endianness (big-endian полей), никакой лишней памяти, ясные коды ошибок.
- **Бонус (+3 б.)**: поддержка jumbo-кадров до 9000 байт конфигурируемым лимитом и отдельные тесты.

> **Связь с прикладной разработкой**
> Парсеры кадров — bread-and-butter системного кода: так работают tcpdump/Wireshark, XDP-программы, сетевые драйверы и DPDK-приложения. Ошибка на один байт в оффсете — классический источник уязвимостей (out-of-bounds read). Разбор кадра в hw-3 — тот же код, что преподаватель покажет в семинаре 15 внутри XDP-программы; заметите совпадение — значит, материал связывается.

Вопросы для обсуждения на семинаре

- Почему MAC-адресация выжила 50 лет, хотя «IP единственный виден в интернете»? Что было бы, если бы кадры адресовались напрямую IP-адресами?
- Куда пропал CSMA/CD в гигабитных сетях? Почему полнодуплексный линк не имеет коллизий?
- Чем изоляция VLAN отличается от изоляции отдельными коммутаторами? Когда второе дешевле/надёжнее?
- Сколько доменов широковещания в сети из трёх коммутаторов с двумя VLAN каждый? А доменов коллизий?

Семинар 4. ARP, ICMP и архитектура доменов; STP (обзор)

Семинар закрывает канальный уровень и готовит мостик к сетевому. Центральный вопрос: как узел узнаёт MAC-адрес соседа, когда известен только IP? Ответ — протокол ARP — разбирается до деталей пакета, после чего в миниатюре реализуется в hw-4. Далее рассматриваем ICMP как «служебный голосок» сетевого уровня и архитектурные понятия доменов коллизий и широковещания, на которых держится проект корпоративной сети (блок IV).

Конспект теории

ARP (RFC 826) переводит IP-адрес следующего хопа в MAC-адрес. Узел, желающий отправить пакет, рассылает широковещательный ARP-запрос «кто имеет 192.168.1.1, скажите 192.168.1.10»; целевой узел отвечает unicast-сообщением со своим MAC. Ответы кэшируются с таймерами; состояния кэша в Linux (REACHABLE, STALE, DELAY, PROBE) воспроизводят логику соседства. Особые случаи: **gratuitous ARP** (анонс своего MAC при смене карты/мастера VRRP), **proxy ARP** (роутер отвечает за чужие адреса), атаки ARP-spoofing и защита dynamic ARP inspection — на этом примере студенты впервые видят, что «доверие канального уровня» — уязвимость.

Поле	Размер	Смысл
HTYPE	2	Тип сети-носителя: 1 = Ethernet
PTYPE	2	Тип протокола: 0x0800 = IPv4
HLEN / PLEN	1+1	Длины адресов: 6 и 4
OPER	2	1 = запрос (who has), 2 = ответ (is at)
SHA	6	MAC отправителя
SPA	4	IP отправителя
THA	6	MAC получателя (в запросе — нули)
TPA	4	Искомый IP-адрес

Формат ARP-сообщения (28 байт, Ethernet + IPv4)

ICMP (RFC 792) — служебный протокол сетевого уровня: echo request/reply (утилита ping), time exceeded (TTL=0 — так работает traceroute), destination unreachable с кодами причин, redirect (подсказка лучшего шлюза). Обсуждаем, какие типы ICMP обязательны для работы стека, почему блокировка всего ICMP ломает PMTUD (предвестие темы MTU в семинаре 5) и как ping измеряет RTT. Демонстрация: tcpdump ловит ICMP time exceeded в traceroute — студент видит механику «звёздочек» из демо 5 своими глазами.

Домены: домен коллизий — сегмент, где передатчики мешают друг другу (разрезается коммутатором/полнодуплексным линком); домен широковещания — область доставки broadcast-кадров (разрезается только маршрутизатором или VLAN). Схема на доске: «один хаб — один домен коллизий; N VLAN — N широковещательных доменов на коммутаторе». Отсюда прямой мостик к **STP**: Ethernet не имеет счётчика TTL, поэтому кадр, пойманный в кольцо, размножается бесконечно — broadcast storm. STP (IEEE 802.1D) строит дерево: выборы root bridge по идентификатору (приоритет + MAC), вычисление кратчайших путей, блокировка избыточных портов; состояния портов и ускоренный RSTP. На семинаре — анимация в Packet Tracer: замыкаем кольцо, смотрим выборы root, затем отключаем STP и наблюдаем лавину.

Демонстрации на семинаре

Демо 8. ARP живьём: чистим кэш и смотрим на запрос/ответ

```
sudo ip neigh flush all                 # опустошить кэш соседей
sudo tcpdump -eni any -c 2 arp          # терминал 1
ping -c 1 192.168.1.1                   # терминал 2
```

```
# Видно: who has 192.168.1.1? tell 192.168.1.10  (broadcast FF:FF:FF:FF:FF:FF)
#        192.168.1.1 is-at a4:2b:b0:1c:9e:70      (unicast ответ)
ip neigh show                           # состояния REACHABLE -> STALE по таймеру
```

Демо 9. ICMP time exceeded — механика traceroute

```
sudo tcpdump -eni any icmp &            # терминал 1
traceroute -n 8.8.8.8                   # терминал 2
# Каждый хоп = ICMP time exceeded от промежуточного роутера;
# финальный хоп отвечает echo reply, если отвечает на ICMP.
```

Практика на семинаре

- В парах: один студент удаляет запись соседа и молчит, второй обязан по дампу процитировать запрос и ответ ARP, включая MAC-адреса; роли меняются.
- Разобрать ARP-кадр из демо 8 полем за полем по таблице формата; посчитать длину и убедиться, что padding добивает кадр до 60 байт (46 полезных).
- Эксперимент: ip neigh show до/после ping; пронаблюдать переход REACHABLE -> STALE -> DELAY, объяснить смысл каждого состояния по man 8 ip-neighbour.
- Packet Tracer: топология из трёх коммутаторов кольцом, включённый и выключенный STP; скриншот состояния портов (альт — заблокированный порт) и «шторма».

Самостоятельная работа до семинара 5

- Таненбаум, гл. 3 (ARP — можно по man 7 arp) и гл. 5 (разделы ICMP); RFC 826 — целиком, он короткий.
- Начать лабораторную PT-1 (см. главу 6, семинар 10): выбрать схему VLAN.
- Прочитать спецификацию hw-5 (семинар 5) и прикинуть структуру парсера IPv4.

Домашнее задание hw-4: ARP-кэш с таймерами (6 баллов, дедлайн — неделя 5)

Студент реализует потокобезопасный однопоточный ARP-кэш — библиотеку, которую можно дальше встроить в учебный стек. Особенность задания — инъекция времени: тесты двигают «часы» вручную, поэтому никаких sleep и системных часов в коде быть не должно. Это стандартный приём тестируемости, на который стоит обратить внимание: он делает детерминированными любые таймерные алгоритмы (ARP-кэш, ретрансмиссии, RTO в hw-9).

include/arp_cache.h — интерфейс задания hw-4

```
#ifndef ARP_CACHE_H
#define ARP_CACHE_H
#include <stdint.h>
#include <stddef.h>
typedef struct arp_cache arp_cache_t;
/* Создаёт кэш ёмкостью capacity (>= 1). tick_now — функция времени
 * (мс, произвольная монотонная база), ttl_entry — время жизни записи.
 * Возвращает NULL при ошибке выделения/невалидных параметрах. */
arp_cache_t *arp_cache_create(size_t capacity,
                              uint64_t (*tick_now)(void *ctx), void *ctx,
                              uint64_t ttl_entry_ms);
/* Обновляет (или добавляет) запись: операция входящего ARP-пакета.
```

```
 * op: 1 = запрос, 2 = ответ. Для ответа запись становится «свежей».
 * Записи вытесняются по TTL; при переполнении — самая старая. */
int arp_cache_put(arp_cache_t *c, uint32_t ip, const uint8_t mac[6], int op);
/* Ищет MAC по IP. Возвращает 0 и заполняет mac, если запись есть и жива
 * (TTL не истёк к моменту tick_now()). -1 — нет записи или истекла. */
int arp_cache_lookup(arp_cache_t *c, uint32_t ip, uint8_t mac_out[6]);
/* Число живых записей (для тестов и статистики). */
size_t arp_cache_size(const arp_cache_t *c);
void   arp_cache_destroy(arp_cache_t *c);
#endif /* ARP_CACHE_H */
```

- **Что проверяют автотесты.** Публичные: put/lookup в обе стороны, обновление записи, вытеснение по ёмкости, истечение TTL. Скрытые: ручные часы (записи, «оживающие» при откате времени), конфликтующие записи на один IP, ёмкость 1, выравнивание структур, отсутствие утечек (тест прогоняется под ASan в CI).
- **Критерии.** Тесты 80%, код 20%: никакой системной зависимости от времени внутри библиотеки, O(1) на lookup (хэш-таблица или отсортированный массив — на выбор студента).
- **Бонус (+3 б.)**: очередь недоставленных пакетов на неопределённые в кэше IP (ожидание ARP-ответа) с ограничением глубины — как у настоящих стеков.

> **Связь с прикладной разработкой**
> ARP-кэш — прямая иллюстрация кэширования с TTL, того же паттерна, что DNS-кэши, HTTP-кэши и conntrack в файрволах. ARP-spoofing — входная точка MITM в локальной сети: любая команда, чей сервис «не должен быть виден соседям», рано или поздно наткнётся на динамический ARP. А состояния кэша соседей Linux (STALE/DELAY/PROBE) позже объяснят студенту, почему после смены IP на сервере «иногда нужно подождать пару секунд».

Вопросы для обсуждения на семинаре

- Почему ARP-запрос broadcast, а ответ unicast? Почему нельзя ответить broadcast-ом (и что бывает, когда можно — proxy ARP)?
- Как ICMP связан с фрагментацией и MTU? Что произойдёт с PMTUD, если заблокировать ICMP целиком?
- Почему в Ethernet нет TTL? Что защищает от вечных петель — и почему STP решает проблему «выключением» линков?
- Что произойдёт с сетью из 1000 хостов в одном широковещательном домене? Почему корпоративные сети режут домены на сотни VLAN?

НЕДЕЛИ 5–9 · IP, МАРШРУТИЗАЦИЯ, UDP, TCP

5. Блок III. Сетевой и транспортный уровни

Центральный и самый длинный блок курса. Пять недель подряд студент разбирает то, что чаще всего подразумевают под «сетью»: адресацию и маршрутизацию IP, дейтаграммный UDP и соединение TCP со всеми его механизмами надёжности. Блок выстроен так, что каждая тема опирается на предыдущую: CIDR нужен для таблиц маршрутизации, таблицы — для понимания пути пакета, а UDP становится фундаментом для самостоятельной реализации протокола с подтверждениями. Итог блока — hw-9: собственный «мини-TCP» поверх UDP со скользящим окном, который сдаётся через автотесты, симулирующие потери пакетов.

Семинар 5. IP: адресация, CIDR, подсети, фрагментация

Пятый семинар переводит студента с языка MAC-адресов на язык IP. Разбираем заголовок IPv4 поле за полем, учимся считать сети и подсети — навык, который потом проверяется в каждой лабораторной. Отдельно разбираем TTL и фрагментацию: это два механизма, которые объясняют половину «странных» сетевых ошибок в продакшене (петли пакетов и зависания на VPN из-за MTU).

Конспект теории

Заголовок IPv4 (RFC 791) — 20 байт без опций. Поле **version/IHL** задаёт длину заголовка в 32-битных словах; **DSCP/ECN** — приоритизация (тема семинара 12); **total length**; **identification, flags, fragment offset** — механика фрагментации; **TTL** — счётчик хопов; **protocol** — диспетчер вверх (6 = TCP, 17 = UDP, 1 = ICMP); **checksum** — только заголовка; **src/dst** — адреса. На семинаре мы читаем реальный заголовок из дампа и заполняем эту таблицу вживую.

Адресация. IPv4-адрес — 32 бита, делящиеся на часть сети и часть хоста. Классовая система мертва; современность — бесклассовая адресация (CIDR): нотация X.Y.Z.W/p, где p — число бит сети. Маска выравнивает границу; сеть определяет адрес, broadcast (все биты хоста = 1) и число узлов (2^h − 2). Частные диапазоны (RFC 1918: 10/8, 172.16/12, 192.168/16) не маршрутизируются в интернете и требуют NAT — предвестник темы недели 11. Loopback 127.0.0.0/8 никогда не выходит на интерфейс.

Поле	Размер	На что смотреть при разборе
Version/IHL	4+4 бита	IHL*4 = длина заголовка; 5 — опций нет
DSCP/ECN	1 байт	Ненулевой DSCP = трафик приоритизируется
Total Length	2	Длина всего пакета; кадр длиннее на Ethernet-заголовок
Flags/FragOff	2	DF=1 — «не фрагментировать»; иначе — фрагментация
TTL	1	Уменьшается на каждом роутере; 64 — Linux, 128 — Windows
Protocol	1	6=TCP, 17=UDP, 1=ICMP — «кому отдать» payload
Checksum	2	Только заголовок; пересчитывается на каждом хопе (TTL!)
src / dst	4+4	Конечные адреса; не меняются в пути (без NAT)

Заголовок IPv4: тренировочная карточка для разбора дампа

Диапазон	CIDR	Типичное применение
10.0.0.0 — 10.255.255.255	10.0.0.0/8	большие корпоративные сети, облака
172.16.0.0 — 172.31.255.255	172.16.0.0/12	средние сети, кластеры, докер по умолчанию (172.17)
192.168.0.0 — 192.168.255.255	192.168.0.0/16	домашние и малые офисные сети

Частные диапазоны (RFC 1918) — знать наизусть

Фрагментация и MTU. Если пакет больше MTU исходящего канала и DF=0, роутер режет его на фрагменты; собирает только получатель. При DF=1 (по умолчанию в TCP) вместо резки отправляется ICMP fragmentation needed — так работает Path MTU Discovery. Классическая болезнь: туннели/VPN «съедают» MTU, ICMP закрыт — и большие передачи зависают при работающем ping. Демонстрация ping -M do показывает это за 30 секунд. IPv6 фрагментацию на роутерах убрал вовсе: только PMTUD и минимальный MTU 1280 — полезно сравнить философию версий.

Расчёт подсетей — практический навык, который тренируем на семинаре до автоматизма: сколько адресов даёт /26? (62 хоста). Как разрезать 192.168.10.0/24 на четыре равные сети? (/26: 0, 64, 128, 192). Какому адресу принадлежит узел 10.1.2.3 при маске 255.255.0.0? (10.1.0.0/16). Разбираем на доске двоичное представление, а затем показываем, что iproute2 умеет валидировать ответы: ipcalc-подобные расчёты средствами ip route get.

Префикс	Маска	Хостов	Пример сети/широковещания
/24	255.255.255.0	254	192.168.1.0 — .1.255
/25	255.255.255.128	126	0–127 и 128–255 при делении /24 пополам
/26	255.255.255.192	62	.0, .64, .128, .192 — деление /24 на четыре
/27	255.255.255.224	30	шаг сети = 32 адреса
/30	255.255.255.252	2	классика p2p-линков (шлюз + узел)
/31	255.255.255.254	2*	RFC 3021: точка-точка без broadcast

Справка по префиксам для блица (запомнить порядок!)

Демонстрации на семинаре

Демо 10. Разбор IP-заголовка по байтам

```
sudo tcpdump -n -c 1 -xx 'tcp port 80 and host example.com'
# Копируем первые 20 байт кадра (после 14 байт Ethernet) в разбор:
# 45 00 00 3c | 1c 46 40 00 | 40 06 | ac 11 0f 19 | ac 11 0f 02
# 45 = IPv4, IHL=5 (20 байт); 00 = DSCP 0; 003c = длина 60;
# 40 = TTL 64; 06 = TCP; ac11 0f19 = 172.17.15.25; 0f02 = 172.17.15.2
```

Демо 11. MTU и PMTUD на практике

```
ip -br link show eth0                 # mtu 1500
ping -c 2 -M do -s 1472 8.8.8.8       # 1500 = 1472 payload + 28 IP+ICMP: ок
ping -c 2 -M do -s 1473 8.8.8.8       # «message too long»: DF=1, не влезает
```

```
ping -c 2 -M do -s 1372 8.8.8.8       # через туннель mtu 1400: тоже не влезет
sudo tcpdump -n icmp                  # увидеть fragmentation needed
```

Практика на семинаре

- Решить 8 задач на CIDR (выдача преподавателя): деление сетей, определение принадлежности адреса, broadcast, число хостов; сверка в парах.
- Снять дамп своего пакета и заполнить таблицу заголовка IPv4; объяснить TTL и протокол верхней стороны.
- Эксперимент с -M do: найти максимальный непрофрагментированный payload до узла в локальной сети и во внешней; объяснить разницу.
- Обсудить сценарий «сайт не открывается по HTTPS, но ping идёт»: составить гипотезы (MTU/PMTUD, TLS, DPI) и план проверки инструментами.

Самостоятельная работа до семинара 6

- Таненбаум, гл. 5: адресация IPv4, CIDR, фрагментация; RFC 1918 — просмотреть.
- Прорешать 12 задач на подсети (выдача) — на семинаре 6 будет пятиминутный блиц.
- Прочитать man 8 ip-route: что выведет ip route get 8.8.8.8 у вас дома и почему?

Домашнее задание hw-5: парсер IPv4-заголовка (6 баллов, дедлайн — неделя 6)

Второй парсер серии. Студент реализует разбор заголовка IPv4 с валидацией чексуммы и минимальной семантикой фрагментации. Задача закрепляет big-endian, битоориентированные поля и RFC 1071 — теперь уже на «настоящем» заголовке.

include/ipv4.h — интерфейс задания hw-5

```
#ifndef IPV4_H
#define IPV4_H
#include <stddef.h>
#include <stdint.h>
typedef struct {
    uint8_t  ihl;            /* в 32-битных словах: 5..15 */
    uint8_t  dscp;
    uint16_t total_len;
    uint16_t id;
    int      df;             /* флаг Don't Fragment: 0/1 */
    int      mf;             /* флаг More Fragments: 0/1 */
    uint16_t frag_off;       /* смещение в 8-байтовых единицах */
    uint8_t  ttl;
    uint8_t  protocol;       /* 1 ICMP, 6 TCP, 17 UDP */
    uint16_t hdr_checksum;   /* как лежит в пакете */
    uint32_t src, dst;       /* в сетевом порядке байт */
    const uint8_t *options;  /* указатель на опции (ihl > 5), иначе NULL */
    size_t   options_len;    /* (ihl - 5) * 4 */
} ipv4_hdr_t;
/* Разбирает заголовок. Возвращает 0 и заполняет out при успехе;
 * -1 при: NULL, len < 20, version != 4, ihl < 5, total_len > len.
 * Чексумма НЕ проверяется здесь — отдельной функцией. */
int ipv4_parse(const uint8_t *buf, size_t len, ipv4_hdr_t *out);
```

```
/* Возвращает 1, если чексумма заголовка корректна (RFC 1071);
 * 0 — если нет; -1 — при ihl < 5 или нечётной длине заголовка. */
int ipv4_checksum_ok(const uint8_t *buf, size_t len);
#endif /* IPV4_H */
```

- **Что проверяют автотесты.** Публичные: эталонный заголовок из демо 10, заголовок с опциями (IHL=6), DF/MF-комбинации, битая чексумма. Скрытые: version=6 в IPv4-парсере, total_len больше буфера, граница ihl=15, проверка что options указывает внутрь входного буфера.
- **Критерии.** Тесты 80%, код 20%; чтение полей строго по сдвигам, никаких reinterpret_cast на невыровненные структуры.
- **Бонус (+3 б.)**: сборщик фрагментов (до 64 одновременных дейтаграмм, детект перекрывающихся фрагментов) с тестами на разрезанном/склеенном трафике.

> **Связь с прикладной разработкой**
> «Сервис тормозит только у клиентов за VPN» — в 80% случаев это PMTUD: заголовки IP и механика фрагментации объясняют зависание, которое не видно в логах. Понимание TTL спасает при диагностике петель маршрутизации (traceroute зацикливается). А CIDR — язык всякой сетевой настройки: от безопасности подов в Kubernetes (NetworkPolicy по /24) до правил файрвола, которые прикладной разработчик согласовывает с инфраструктурой.

Вопросы для обсуждения на семинаре

- Почему checksum IP считает только заголовок? Что это экономит и чем платим?
- Куда пропал broadcast в IPv6 и что пришло вместо него?
- Как NAT ломает «сквозность» IP и какие протоколы от этого страдают сильнее всего?
- Почему у фрагментов нет своей чексуммы payload — как тогда ловить битые данные?

Семинар 6. Маршрутизация: таблицы, longest prefix match, мини-роутер на netns

Шестой семинар отвечает на вопрос «как пакет находит дорогу». Мы разбираем таблицу маршрутизации как структуру данных, вводим алгоритм longest prefix match — и реализуем его в hw-6. Половина семинара — живая лаборатория: из трёх namespaces Linux студенты вместе с преподавателем собирают маршрутизатор и смотрят, как пакет проходит путь, о котором только что шла речь в теории.

Конспект теории

Маршрутизатор пересылает пакеты между сетями: читает dst, ищет в таблице наиболее специфичный префикс (longest prefix match), уменьшает TTL, пересчитывает чексумму заголовка и меняет канальную адресацию: MAC-адреса всегда «на один хоп», IP — сквозной. Запись таблицы: префикс/длина, next-hop, интерфейс, метрика. Специальные записи: маршрут по умолчанию 0.0.0.0/0 (совпадает со всем), маршруты подключенных сетей (connected), host-маршруты /32 (самые специфичные). ip route get — встроенный «предсказатель» решения о forwarding: он показывает, через какой интерфейс и next-hop уйдёт пакет, не отправляя его.

Таблицы заполняются либо статически (администратором), либо динамическими протоколами: внутри автономных систем — IGP (RIP — исторический, OSPF — link-state, обзорно), между ними — BGP, протокол маршрутизации всего интернета (тема одного из мини-исследований). Студенту важно понять принцип, а не конкретные протоколы: маршрутизация — распределённая система, обменивающаяся достижимостью, а решение о forwarding принимается локально за одну операцию поиска. Отдельно — ICMP redirect: «ошибка конфигурации, оформленная как подсказка».

Longest prefix match — единственная нетривиальная структура данных в этом блоке. Наивный подход «проверить все записи и взять самый длинный совпавший префикс» работает, но при тысячах маршрутов нужен trie (префиксное дерево, в сетевом мире — Patricia/DIR-24-8). Мы рисуем битовое дерево на доске: запись 10.0.0.0/8 — ветка длины 8; /24 уточняет её; /32 — лист. Lookup идёт по битам адреса от корня и запоминает последний пройденный «реальный» узел. hw-6 реализует именно этот алгоритм.

Демонстрации: мини-роутер на namespaces

Демо 12. Три namespace, veth-пары и маршрутизация между ними

```
#!/bin/bash
# nsmesh.sh: ns-left --- [router] --- ns-right
ip netns add left; ip netns add right; ip netns add router
ip link add v-l type veth peer name v-l-r; ip link set v-l netns left
ip link add v-r type veth peer name v-r-r; ip link set v-r netns right
ip link set v-l-r netns router; ip link set v-r-r netns router
ip netns exec left    ip addr add 10.0.1.2/24 dev v-l;    ip netns exec left    ip link
        set v-l up
ip netns exec right   ip addr add 10.0.2.2/24 dev v-r;    ip netns exec right   ip link
        set v-r up
ip netns exec router  ip addr add 10.0.1.1/24 dev v-l-r;  ip netns exec router  ip link
        set v-l-r up
ip netns exec router  ip addr add 10.0.2.1/24 dev v-r-r;  ip netns exec router  ip link
        set v-r-r up
# Роутер обязан пересылать: иначе он только «терминальный» узел
ip netns exec router sysctl -w net.ipv4.ip_forward=1
ip netns exec left   ip route add default via 10.0.1.1
ip netns exec right  ip route add default via 10.0.2.1
ip netns exec left  ping -c 2 10.0.2.2       # работает: пакет идёт через router
ip netns exec left  traceroute -n 10.0.2.2   # видно хоп 10.0.1.1
```

Демо 13. Ломаем маршрут и смотрим

```
ip netns exec left ip route del default        # убрать шлюз
ip netns exec left ping -c 2 10.0.2.2          # «Network is unreachable»
```

```
ip netns exec left ip route add 10.0.2.0/24 via 10.0.1.1   # вернуть точечно
ip netns exec left ping -c 2 10.0.2.2          # снова работает
ip netns exec left ip route get 10.0.2.2       # показать решение forwarding
```

В демо 13 преподаватель намеренно создаёт «плохую конфигурацию» и предлагает студентам диагно́стировать её только командами: это миниатюра реального разбора инцидента. namespaces — рабочая лошадка всех последующих лабораторных (QoS, прокси, eBPF), поэтому команды ip netns стоит запомнить: они есть и в шпаргалке приложения В.

Практика на семинаре

- Воспроизвести демо 12 у себя; добавить третий namespace и провернуть «треугольник» маршрутизации.
- На бумаге: таблица из 7 маршрутов (выдача) — посчитать LPM для 5 адресов; затем проверить себя, построив trie на доске.
- Блиц-опрос по подсетям из самостоятельной работы (5 минут, оценка в активность).
- Обсудить: почему default route есть почти у всех, но почти никогда — у роутеров провайдеров?

Самостоятельная работа до семинара 7

- Таненбаум, гл. 5: разделы о маршрутизации; man 8 ip-route; RFC 1918 повторно.
- Собрать топологию демо 12 в виде скрипта и сохранить — пригодится в лабе QoS.
- Прикинуть структуру trie для hw-6: как хранить /8, /24 и /32 в одном дереве?

Домашнее задание hw-6: таблица маршрутизации с LPM (6 баллов, дедлайн — неделя 7)

Студент реализует потокобезопасную (однопоточную, но с чистым интерфейсом) таблицу маршрутизации на префиксном дереве с операциями вставки, удаления и поиска по longest prefix match. Ключевое требование — корректное разрешение перекрывающихся префиксов и детерминированное поведение.

include/lpm.h — интерфейс задания hw-6

```
#ifndef LPM_H
#define LPM_H
#include <stdint.h>
#include <stddef.h>
typedef struct lpm_table lpm_table_t;
lpm_table_t *lpm_create(size_t max_rules);
void          lpm_destroy(lpm_table_t *t);
/* Добавляет правило prefix/plen -> value (plen 0..32; 0 = default route).
 * Повторная вставка того же префикса ОБНОВЛЯЕТ value.
 * Возвращает 0, -1 при ошибке (NULL, plen > 32, переполнение таблицы,
 * префикс «шире маски»: биты после /plen должны быть нулями). */
int lpm_insert(lpm_table_t *t, uint32_t prefix, uint8_t plen, int value);
/* Удаляет правило. Возвращает 0, если правило было; -1 если нет. */
int lpm_remove(lpm_table_t *t, uint32_t prefix, uint8_t plen);
/* Longest prefix match. Возвращает 0 и value_out, если есть совпадение
```

```
 * (самый длинный префикс); -1, если маршрута нет (таблица пуста/только
 * несовпадающие префиксы). Default 0.0.0.0/0 ловит всё, что не точнее. */
int lpm_lookup(const lpm_table_t *t, uint32_t ip, int *value_out);
size_t lpm_size(const lpm_table_t *t);   /* число активных правил */
#endif /* LPM_H */
```

- **Что проверяют автотесты.** Публичные: вставка/удаление, перекрытие /8 и /24 (побеждает /24), default route, обновление значения. Скрытые: масштабный тест на 10 000 правил с контролем времени (lookup обязан быть не хуже O(32)), случайные таблицы против эталонной наивной реализации (property-based), вставка префикса с «грязными» битами после маски.
- **Критерии.** Тесты 80%, код 20%: память O(N*32) максимум, отсутствие утечек (ASan), понятная структура узла.
- **Бонус (+3 б.)**: оптимизация типа DIR-24-8 (двухуровневая таблица) с бенчмарком lookup/сек и сравнением с trie в отчёте-README.

Связь с прикладной разработкой

LPM живёт не только в роутерах: маршрутизация подов в Kubernetes, сегментация в облаках (VPC-таблицы), файрволы с тысячами правил, CDN-выбор региона. Даже «просто бэкенд-разработчик» встречает LPM, когда читает security groups или отлаживает «почему трафик уходит не в тот peering». Дерево префиксов — заодно прекрасный пример того, как структура данных диктует производительность системы.

Вопросы для обсуждения на семинаре

- Почему пересылающий роутер меняет MAC-адреса, но не IP? Что изменится, если бы менял IP?
- Что опаснее для стабильности сети: петля маршрутизации (routing loop) или петля кадров (L2 loop)? Где цикл обнаружится быстрее?
- Как бы вы искали LPM, если маршрутов миллионы? Что придумали в железе (TCAM) и в софте (DIR-24-8)?
- Почему ip route get не отправляет пакет — как устроен «сухой прогон» forwarding?

Семинар 7. UDP: дейтаграммы, порты, и первый протокол поверх UDP

Седьмой семинар открывает транспортный уровень. UDP — минимальный транспорт: «IP плюс порты плюс опциональная чексумма». Мы разбираем, зачем он существует, как работает мультиплексирование по портам, и пишем первый сетевой сервис на сокетах вживую. Домашнее задание — протокол TFTP в миниатюре: первый протокол курса с состояниями и таймаутами.

Конспект теории

Заголовок UDP (RFC 768) — 8 байт: порты источника и получателя, длина, чексумма. Порт — 16-битный идентификатор процесса: транспортный уровень демультиплексирует дейтаграммы по паре (IP, порт). Чексумма в IPv4 опциональна (нулевое поле), в IPv6 обязательна; считается по псевдозаголовку (адреса + протокол + длина) — защита от «пакет не туда». UDP не устанавливает соединение, не гарантирует доставку, порядок и отсутствие дублей, не контролирует перегрузку — и именно поэтому он идеален там, где важна латентность или протокол сам управляет надёжностью: DNS, DHCP, NTP, голос/видео, QUIC, современные игры.

Свойство	TCP	UDP
Соединение	handshake, состояние у обеих сторон	нет состояния
Доставка	гарантированная, упорядоченная	at-most-once, порядок не обещан
Контроль потока/перегрузки	есть (окна, congestion control)	нет — «и так сойдёт»
Единица данных	поток байт, сегменты	дейтаграмма = сообщение
Головной блокировкой	страдаем при потерях	нет
Типовое применение	HTTP, SSH, почта, БД	DNS, DHCP, NTP, VoIP, QUIC

TCP vs UDP: «что выбираем и чем платим»

Сокеты UDP: socket(AF_INET, SOCK_DGRAM, 0), bind, sendto/recvfrom; connect() на UDP-сокете не устанавливает соединение, а лишь фиксирует «постоянного адресата» (и позволяет получать ICMP-ошибки типа port unreachable). Важная практика — размер дейтаграммы: всё, что больше MTU пути, фрагментируется IP; фрагмент потерян — потеряна вся дейтаграмма. Правило: данные UDP-запроса держат в пределах ~1200 байт (стандарт де-факто DNS и QUIC). Мы разбираем сокетный код echo-сервера построчно — это первый листинг курса, который студент компилирует вживую.

Демонстрации на семинаре

Демо 14. UDP echo-сервер на C — 25 строк, компилируем на семинаре

```
#include <arpa/inet.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>
int main(void) {
    int fd = socket(AF_INET, SOCK_DGRAM, 0);
    struct sockaddr_in a = {0}, c; socklen_t cl = sizeof c;
    a.sin_family = AF_INET; a.sin_addr.s_addr = htonl(INADDR_ANY);
    a.sin_port = htons(7777);
    bind(fd, (struct sockaddr *)&a, sizeof a);          /* без listen! */
    char buf[1500];
    for (;;) {
        ssize_t n = recvfrom(fd, buf, sizeof buf, 0, (struct sockaddr *)&c, &cl);
        if (n < 0) { perror("recvfrom"); return 1; }
        sendto(fd, buf, (size_t)n, 0, (struct sockaddr *)&c, cl);  /* эхо */
    }
}
```

Демо 15. Говорим с echo-сервером: nc и tcpdump

```
# терминал 2: интерактивный клиент
nc -u 127.0.0.1 7777
# терминал 3: дамп с разбором UDP-заголовка
sudo tcpdump -n -X -i lo udp port 7777
# Эксперимент: отправить 2000 байт одной дейтаграммой и увидеть
# фрагментацию (не на lo!); потерять кусок при -M do — дейтаграмма пропала
```

Практика на семинаре

- Собрать и запустить демо 14; обменяться клиентами между парами; посмотреть UDP-заголовок в дампе, найти порт и чексумму.
- Эксперимент: nc -u и отправка сообщения, превышающего буфер recvfrom — что обрезается и почему (нет потока, есть граница сообщений!).
- Убить сервер и послать дейтаграмму на закрытый порт: получить ICMP port unreachable; обсудить, как это видит connect()-нутый UDP-сокет (ECONNREFUSED).
- Спор-обсуждение: почему DNS до сих пор на UDP и что изменил DNS over TLS/QUIC.

Самостоятельная работа до семинара 8

- Таненбаум, гл. 6 (транспорт): начало главы, UDP; RFC 768 — 3 страницы.
- Дописать echo-сервер: аргумент порта из argv, обработка SIGINT, лог каждой дейтаграммы (время, адрес, длина).
- Прочитать RFC 1350 (TFTP): пакет RRQ, DATA, ACK, ERROR — это будет hw-7.

Домашнее задание hw-7: мини-TFTP с инъекцией транспорта (6 баллов, дедлайн — неделя 8)

Студент реализует клиентскую часть чтения по TFTP (RFC 1350, режим octet) — но не поверх реальных сокетов, а поверх **инъектируемого транспорта**: структура функций send/recv передаётся извне. Автотесты подставляют «виртуальный сервер» со скриптованными потерями и задержками, поэтому тесты детерминированы и быстры. Задание тренирует: протокольные состояния, таймауты/ретрансмиссии, упаковку полей в буферы.

include/tftp.h — интерфейс задания hw-7

```
#ifndef TFTP_H
#define TFTP_H
#include <stddef.h>
#include <stdint.h>
/* Транспорт, который подставляют тесты (и который вы оборачиваете
 * вокруг UDP для запуска «по-настоящему»). */
typedef struct {
    void *ctx;
    /* Отправляет пакет; возвращает 0 или -1. */
    int  (*send)(void *ctx, const uint8_t *buf, size_t len);
    /* Ждёт пакет до timeout_ms; возвращает число принятых байт,
     * 0 — таймаут, -1 — ошибка транспорта. */
    int  (*recv)(void *ctx, uint8_t *buf, size_t cap, unsigned timeout_ms);
    uint64_t (*now_ms)(void *ctx);   /* инъекция часов, как в hw-4 */
```

```
} tftp_transport_t;
/* Скачивает файл filename с сервера (режим octet), вызывает on_data
 * для каждого блока (block_num начинается с 1). Возвращает суммарное
 * число принятых байт или -1 при ошибке протокола/превышении попыток.
 * Лимиты: 5 ретрансмиссий на блок, MAX_BLOCK_SIZE = 512 (RFC 1350). */
long tftp_download(const tftp_transport_t *tr, const char *filename,
                   void (*on_data)(void *ud, uint32_t block_num,
                                   const uint8_t *data, size_t len),
                   void *user_data);
/* Сборка пакетов для тестов: RRQ и ACK; возвращает длину или -1. */
int tftp_build_rrq(uint8_t *buf, size_t cap, const char *filename);
int tftp_build_ack(uint8_t *buf, size_t cap, uint16_t block_num);
#endif /* TFTP_H */
```

- **Что проверяют автотесты.** Публичные:happy path на 3 блока, потеря одного DATA (ретрансмиссия после таймаута), потеря ACK. Скрытые: дубли блоков, ответ ERROR (функция возвращает -1 с кодом), чередование потерь в 30% пакетов (счётчик ретрансмиссий), контроль, что транспорт вызывается ровно согласно состояниям (mock-счётчики).
- **Критерии.** Тесты 80%, код 20%: таблица состояний вместо вложенных if, нулевое использование глобальных переменных, чтение полей по big-endian.
- **Бонус (+3 б.)**: вариант blocksize=1024 (опция RFC 2348) и тесты на согласование размера блока.

> **Связь с прикладной разработкой**
> TFTP-клиент — это в миниатюре любой протокол поверх «ненадёжного» канала: ретрансмиссии и дедлайны так же пишутся в сигнальных протоколах, игровых клиентах и системах телеметрии. Инъекция транспорта и часов — тот же принцип, на котором построены фейковые клиенты в юнит-тестах реальных сервисов: тестируемость протокола проектируется заранее, в интерфейсе.

Вопросы для обсуждения на семинаре

- Почему у UDP нет connect в понимании TCP, но системный вызов connect() для UDP существует? Что он делает?
- Дейтаграмма 60 КБ через линк MTU 1400: сколько фрагментов, что случится при потере любого из них?
- Почему QUIC строит надёжность поверх UDP вместо «правки» TCP? Что это даёт прикладным разработчикам?
- Как бы вы оценили «грузоподъёмность» UDP-сервиса, если он не контролирует перегрузку? (Подводка к congestion control.)

Семинар 8. TCP, часть 1: заголовок, handshake, машина состояний

Восьмой семинар — самый насыщенный протокольно. Мы разбираем TCP как машину состояний: handshake, передачу с подтверждениями, корректное и аварийное завершение. Разбор подкреплён дампами: студенту показывают SYN, SYN-ACK, ACK, FIN-обмен и RST в одном pcap. Домашнее задание — парсер TCP-заголовка и симулятор машины состояний по событиям.

Конспект теории

TCP (RFC 9293, обновивший RFC 793) — надёжный поток байт между парами сокетов. Заголовок 20 байт без опций: порты, **sequence number** и **acknowledgment number** (счёт в байтах, а не в пакетах!), флаги (SYN, ACK, FIN, RST, PSH, URG), окно получателя, чексумма, указатель срочности, опции (MSS, window scale, timestamps, SACK). Начальный номер последовательности ISN выбирается случайно — защита от наложения старых соединений и подделки.

Установление соединения — трёхшаговый handshake: клиент шлёт SYN(seq=ISN_c), сервер отвечает SYN-ACK(seq=ISN_s, ack=ISN_c+1), клиент подтверждает ACK. Три шага нужны, чтобы обе стороны обменялись начальными номерами и убедились в двунаправленной достижимости; два шага недостаточны из-за дубликатов старых SYN. Сервер хранит полусоединение в очереди SYN backlog — отсюда типовая атака SYN flood и механизм SYN cookies. **Завершение** — четырёхшаговый обмен FIN/ACK в обе стороны (полузакрытие возможно), либо мгновенный RST при ошибке. TIME_WAIT на закрытой стороне — страховка от «заблудившихся» сегментов; поэтому интенсивные клиенты видят исчерпание локальных портов.

Машина состояний (диаграмма RFC 793) — свод всех фаз: LISTEN, SYN_SENT, SYN_RCVD, ESTABLISHED, FIN_WAIT_1/2, CLOSE_WAIT, CLOSING, LAST_ACK, TIME_WAIT, CLOSED. Мы проходим её на доске с двумя «жизнями» — клиента и сервера — и сверяем с реальностью: ss -tan показывает эти состояния на живой системе. Анализ «кто виноват в CLOSE_WAIT» — вечная тема код-ревью: это состояние означает, что приложение не закрыло сокет после получения FIN.

Из состояния	Событие → куда	Комментарий
CLOSED	OPEN_PASSIVE → LISTEN	сервер объявил порт
CLOSED	OPEN_ACTIVE → SYN_SENT	клиент инициирует
LISTEN	SEG_SYN → SYN_RCVD	полусоединение в backlog
SYN_SENT	SEG_SYNACK → ESTABLISHED	клиент закончил handshake
SYN_RCVD	SEG_ACK → ESTABLISHED	сервер закончил handshake
ESTABLISHED	CLOSE → FIN_WAIT_1	активное закрытие (мы первые)
ESTABLISHED	SEG_FIN → CLOSE_WAIT	пассивное: FIN пришёл первым
FIN_WAIT_1	SEG_ACK → FIN_WAIT_2	наш FIN подтвердили
FIN_WAIT_2	SEG_FIN → TIME_WAIT	полузакрытие завершено
CLOSE_WAIT	CLOSE → LAST_ACK	приложение закрыло своё
CLOSING	SEG_ACK → TIME_WAIT	одновременное закрытие

Из состояния	Событие → куда	Комментарий
любое актив.	SEG_RST → CLOSED	аварийный разрыв немедленно
TIME_WAIT	TIMEOUT (2*MSL) → CLOSED	страховка от заблудившихся сегментов

Ключевые переходы FSM TCP (подмножество для hw-8)

Демонстрации на семинаре

Демо 16. Handshake и завершение глазами tcpdump

```
# терминал 1: дамп обмена с example.com:80
sudo tcpdump -n -S 'host example.com and tcp port 80'
# -S показывает абсолютные (не относительные) seq/ack — честные числа
# терминал 2:
printf 'GET / HTTP/1.0\r\n\r\n' | nc example.com 80
# в дампе: SYN -> SYN-ACK -> ACK -> PSH,ACK(запрос) -> ACK -> FIN,ACK ...
```

Пакет	Флаги и seq	ack	Что доказывает
1	SYN, seq=1000 (ISN_c)	–	клиент инициирует, ISN случайный
2	SYN+ACK, seq=5000 (ISN_s)	1001	сервер подтвердил ISN_c+1: SYN «весит» байт
3	ACK	5001	клиент подтвердил ISN_s+1; соединение ESTABLISHED
4	PSH+ACK, seq=1001	5001	запрос: 20 байт, seq вырос на длину
5	FIN+ACK, seq=5001	1021	сервер закончил: FIN съел ещё байт
6	ACK, seq=1021	5002	клиент подтвердил FIN; обе стороны CLOSED

Разбор дампа демо 16: считать seq/ack по строкам

Демо 17. Состояния TCP на живой системе

```
ss -tan state established | head      # активные соединения
ss -tan state time-wait | wc -l        # сколько «хвостов» закрытий
ss -tan state close-wait               # утилита-детектор утечки сокетов
# Ручной RST: послали байт в соединение, сервер убит SIGKILL -> RST
```

Практика на семинаре

- Снять handshake с -S, записать seq/ack каждого шага; доказать, что ack = seq+1 (SYN и FIN «весит» один байт последовательности).
- Сымитировать полузакрытие: shutdown(SHUT_WR) в маленькой C-программе; найти FIN без ответного FIN в дампе, объяснить смысл.
- Написать 10-строчный «спамер соединений» на C и понаблюдать TIME_WAIT; обсудить SO_REUSEADDR и когда он безопасен.
- Разобрать (на доске) сцену: сервер в CLOSE_WAIT, клиент ушёл — кто и что должен сделать, чтобы освободить сокет?

Самостоятельная работа до семинара 9

- Таненбаум, гл. 6: TCP — установление, завершение, управление потоком (до конгестии — она будет на семинаре 9).
- RFC 9293 — только разделы о handshake и FIN (структура RFC найдётся по TOC).
- Прогнать демо 16–17 у себя и сохранить pcap — понадобится в hw-8 и на защите.

Домашнее задание hw-8: парсер TCP-заголовка + FSM (6 баллов, дедлайн — неделя 9)

Задание из двух частей. Первая — разбор заголовка TCP (включая длину опций и выделенные флаги). Вторая — детерминированный симулятор машины состояний: функция получает текущее состояние и событие (входящий сегмент с флагами / вызов приложения close), возвращает новое состояние или ошибку. Это ядро, которое пригодится в hw-9 для управления «мини-TCP».

include/tcpfsm.h — интерфейс задания hw-8

```
#ifndef TCPFSM_H
#define TCPFSM_H
#include <stddef.h>
#include <stdint.h>
typedef struct {
    uint16_t src_port, dst_port;
    uint32_t seq, ack;
    uint8_t  data_off;      /* смещение данных в 32-битных словах: 5..15 */
    int      f_syn, f_ack, f_fin, f_rst, f_psh, f_urg;
    uint16_t window;
    const uint8_t *options; size_t options_len;  /* как в hw-5 */
    const uint8_t *payload; size_t payload_len;  /* указатели внутрь буфера */
} tcp_hdr_t;
/* Разбор заголовка TCP: 0 при успехе; -1 при NULL/len<20/data_off<5. */
int tcp_parse(const uint8_t *buf, size_t len, tcp_hdr_t *out);
/* Машина состояний: состояния и события — перечисления ниже.
 * Возвращает новое состояние; TCP_FSM_ERR — недопустимый переход. */
typedef enum { TCP_CLOSED, TCP_LISTEN, TCP_SYN_SENT, TCP_SYN_RCVD,
               TCP_ESTABLISHED, TCP_FIN_WAIT_1, TCP_FIN_WAIT_2,
               TCP_CLOSE_WAIT, TCP_CLOSING, TCP_LAST_ACK, TCP_TIME_WAIT,
               TCP_STATE_MAX } tcp_state_t;
typedef enum { EV_OPEN_PASSIVE, EV_OPEN_ACTIVE, EV_CLOSE,
               EV_SEG_SYN, EV_SEG_SYNACK, EV_SEG_ACK,
               EV_SEG_FIN, EV_SEG_FINACK, EV_SEG_ACK, EV_SEG_RST,
               EV_TIMEOUT } tcp_event_t;
tcp_state_t tcp_fsm_step(tcp_state_t s, tcp_event_t e);
#endif /* TCPFSM_H */
```

- **Что проверяют автотесты.** Публичные: эталонные заголовки (с опциями MSS, без опций), полные жизненные циклы: активное открытие, пассивное, корректное закрытие обеими сторонами. Скрытые: RST из любого состояния, FIN в SYN_SENT (обязан закрыть), двойной SYN-ACK, TIME_WAIT-таймаут через EV_TIMEOUT.
- **Критерии.** Тесты 80%, код 20%: FSM обязана быть таблицей переходов (массив/switch по состоянию) — «лапша if» снижает балл за стиль.
- **Бонус (+2 б.)**: RFC-заметка в README: 3 состояния, где ваша FSM проще настоящей, и почему (например, одновременное открытие).

Связь с прикладной разработкой
CLOSE_WAIT-утечки и «зависшие» полусоединения — результат непонимания FSM TCP. Разработчик, знающий переходы, по выводу ss за секунду диагностирует, какая сторона виновата. SYN backlog и SYN cookies объясняют странное «сервер не принимает новые соединения при атаке». Понимание seq/ack — фундамент для чтения Wireshark при любом споре с сетевой командой о «кто закрыл первым».

Вопросы для обсуждения на семинаре

- Почему handshake именно из трёх шагов? Какой баг позволил бы двухшаговый?
- Что хуже для сервера: тысячи соединений в TIME_WAIT или в CLOSE_WAIT? Почему?
- Как RST отличается от FIN по семантике и по последствиям для буферов приложения?
- Зачем TCP нужны опции (MSS, window scale) в SYN, если их можно было бы «зашить» в протокол?

Семинар 9. TCP, часть 2: окно, контроль перегрузки, Nagle; старт hw-9

Девятый семинар завершает транспортный блок. Если семинар 8 отвечал «как TCP устанавливает и закрывает соединения», то этот — «как TCP гонит данные быстро и одновременно бережно»: управление потоком со стороны получателя, контроль перегрузки со стороны сети, ретрансмиссии по таймеру и микрофлэги вроде Nagle. Здесь же выдаётся самое большое задание курса — hw-9, собственный протокол надёжной доставки поверх UDP.

Конспект теории

Управление потоком: получатель анонсирует свободное место в окне (rwnd) в каждом сегменте; отправитель не смеет отправлять больше, чем окно. Нулевое окно останавливает передачу (и требует зондов window probe). **Контроль перегрузки** (RFC 5681) защищает сеть, а не получателя: окно перегрузки cwnd растёт экспоненциально в slow start (до порога ssthresh), затем линейно в congestion avoidance (+1 MSS на RTT, AIMD). Потери трактуются как сигнал перегрузки: таймаут RTO режет cwnd до 1 MSS и заново запускает slow start; три дублирующих ACK вызывают fast retransmit и fast recovery (половина cwnd). Реализации: CUBIC — дефолт Linux с 2006, кубический рост cwnd по времени; BBR — измеряет пропускную способность и RTT, а не ждёт потерь (подробности — одно из мини-исследований).

Таймеры: RTO оценивается по RTT (SRTT со сглаживанием, deviation), экспоненциальный откат при повторных таймаутах, правило Карна — не измерять RTT по ретранслированным сегментам. **Микрорешения, влияющие на latency**: алгоритм Нейгла группирует мелкие записи (TCP_NODELAY отключает — первый совет любому разработчику чата/игры), delayed ACK копит подтверждения, keepalive держит NAT-трансляции живыми. Все эти механизмы видны в Wireshark: показываем графики cwnd и retransmissions на реальном pcap с искусственными потерями.

Демонстрации на семинаре

Демо 18. Потери и их последствия: netem + iperf3

```
# Пара netns с veth (из демо 12), на роутере добавляем потери и задержку:
ip netns exec router tc qdisc add dev v-l-r root netem delay 30ms loss 2%
ip netns exec left  iperf3 -s &                 # сервер
ip netns exec right iperf3 -c 10.0.1.2 -t 10    # клиент: ~столько Мбит/с
# Сравниваем три конфигурации: без netem, loss 0%, loss 2%, loss 5%:
# студент видит, как потери «съедают» пропускную способность TCP
```

Демо 19. Стрём-трафик глазами Wireshark

```
# Пишем pcap в netns и открываем в Wireshark:
ip netns exec left tcpdump -w /tmp/loss.pcap 'tcp port 5201'
# В Wireshark: Statistics -> TCP Stream Graphs -> Time Sequence (tcptrace):
# ступеньки = ретрансмиссии, «полка» = RTO-таймаут, красные точки = дубли ACK
```

Практика на семинаре

- Прогнать демо 18 с тремя уровнями потерь; записать throughput и RTT; объяснить расхождение через AIMD.
- Сравнить iperf3 с -Z zero copy и без; обсудить, что ограничивает скорость: сеть, CPU или буферы (введение в неделю 16).
- На живом соединении включить/выключить TCP_NODELAY в маленьком клиенте и посмотреть разницу в дампе (два PSH вместо одного).
- Разбор постановки hw-9: протокол, ожидаемые состояния, тестовый харнесс; формируем пары добровольцев для бонусной реализации SACK-подобного ACK.

Самостоятельная работа до семинара 10

- Таненбаум, гл. 6: контроль перегрузки; RFC 5681 — разделы slow start/AIMD.
- Начать hw-9 (дедлайн — неделя 11, две недели на реализацию).
- Подготовить тему для мини-исследования: выбрать из каталога главы 9 и сдать заявку преподавателю до недели 12.

Домашнее задание hw-9: мини-TCP поверх UDP — ARQ со скользящим окном (10 баллов, дедлайн — неделя 11)

Флагманское задание транспортного блока: надёжный протокол с подтверждениями, окном и ретрансмиссиями поверх дейтаграмм UDP. Как и в hw-7, транспорт и время инъектируются — тесты симулируют потери, дубли, перестановки и задержки. Задание прокачивает: алгоритмы ARQ, работу с буферами, продумывание взаимодействия отправителя и получателя. Бонусом — «перегрузка» в миниатюре.

include/mtcp.h — интерфейс задания hw-9 (фрагмент)

```
#ifndef MTCP_H
#define MTCP_H
#include <stddef.h>
#include <stdint.h>
#include "tftp.h"        /* переиспользуем tftp_transport_t из hw-7 */
```

```
typedef struct mtcp_conn mtcp_conn_t;
/* Конфигурация протокола. */
typedef struct {
    uint16_t window;        /* размер окна в байтах (кратно MSS) */
    uint16_t mss;           /* максимум полезной нагрузки кадра */
    unsigned rto_ms;        /* базовый таймаут ретрансмиссии */
    unsigned max_retrans;   /* попыток на кадр, 0 = бесконечно */
} mtcp_config_t;
/* Создаёт соединение-отправитель поверх транспорта. */
mtcp_conn_t *mtcp_open(const tftp_transport_t *tr, const mtcp_config_t *cfg);
/* Отправляет len байт (можно больше окна: функция блокируется,
 * «двигая» окно по подтверждениям из recv-обратного вызова транспорта).
 * Возвращает len, -1 при исчерпании max_retrans/ошибке. */
long mtcp_send(mtcp_conn_t *c, const uint8_t *data, size_t len);
/* Приёмная сторона: собирает поток из дейтаграмм, отдаёт данные
 * вызывающему строго по порядку, подтверждает кумулятивно. */
long mtcp_recv(mtcp_conn_t *c, uint8_t *out, size_t cap);
void mtcp_close(mtcp_conn_t *c);
#endif /* MTCP_H */
```

- **Что проверяют автотесты.** Публичные: доставка 1 байта и 100 КБ без потерь, потери 5% и 25% (функция обязана довести до конца), дубли и перестановки кадров. Скрытые: «буря» потерь в 40% с чередованием задержек, окно 1×MSS (стоп-энд-вэйт), исчерпание max_retrans (корректный -1), дедлок-тест: приёмник «молчит» N мс — отправитель обязан ретранслировать.
- **Критерии.** Тесты 70%, код 20%, отчёт-README с описанием протокола 10%: формат кадра, ack-семантика, обработка краёв.
- **Бонус (+5 б.)**: имитация контроля перегрузки (экспоненциальный рост окна после старта, разрезание окна вдвое на потере) + тест «пропускная способность при 2% потерь выросла/не упала».

> **Связь с прикладной разработкой**
> После hw-9 студент больше никогда не скажет «TCP сам всё сделает» без понимания цены: почему первый байт HTTPS-запроса едет сотни мс (handshake + TLS), почему lossy-канал рушит throughput экспоненциально (AIMD), почему разработчику realtime- приложения советуют TCP_NODELAY и маленькие сообщения, и почему BBR подарил YouTube прирост на «толстых, но потерянных» каналах. Это тот случай, когда реализация протокола меняет чтение документации навсегда.

Вопросы для обсуждения на семинаре

- Почему TCP трактует потерю как перегрузку и всегда прав ли он? (Wi-Fi, сотовые сети — где этот постулат ломается.)
- Чем BBR принципиально отличается от CUBIC? Почему BBR иногда «грабит» соседние потоки?
- Nagle + delayed ACK: почему их совместное действие даёт классические 40 мс задержки и как этого избегать?
- Как бы выглядел congestion control для hw-9? Какие сигналы перегрузки вообще доступны протоколу поверх UDP?

НЕДЕЛИ 10–12 · VLAN, DHCP/NAT/ACL, ПРИОРИТИЗАЦИЯ

6. Блок IV. Корпоративные сети, изоляция и QoS

Третий блок переносит студентов из «одной локалки» в реальный мир: корпоративная сеть строится из множества сегментов, изолированных друг от друга и от интернета. Три недели идут парами «теория — лабораторная»: сначала семинар объясняет принцип на схемах и живых конфигурациях, затем лабораторная (Cisco Packet Tracer или Linux netns) требует собрать это руками. Формат сдачи лабораторных — отчёт со скриншотами и выводами; главное в отчёте — объяснение «почему работает», а не набор картинок.

Семинар 10. Корпоративные сети: иерархия, VLAN, router-on-a-stick

Десятый семинар отвечает на вопрос «почему корпоративные сети устроены сложно». Простая сеть из одного коммутатора перестаёт работать, когда узлов становятся сотни: широковещательный трафик съедает канал, отделы видят чужие данные, отказ любого узла валит всех. Решения — иерархия и сегментация. Мы разбираем трёхуровневую модель (access — подключения, distribution — агрегация и маршрутизация, core — магистраль) и переходим к инструментам сегментации: VLAN на access-портах, транки между коммутаторами, маршрутизация между VLAN.

Конспект теории

Иерархия сети. Уровень доступа — коммутаторы, к которым подключены хосты (access-порты, по VLAN на отдел). Уровень распределения — коммутаторы/маршрутизаторы агрегации: здесь живёт маршрутизация между VLAN, ACL и QoS-политики. Ядро — магистральный транспорт (в учебных топологиях часто сливается с распределением). Правило проектирования: широковещательный домен не должен пересекать уровень распределения; каждый VLAN — один сегмент одного отдела.

Маршрутизация между VLAN. Классика — router-on-a-stick: один физический интерфейс роутера с подынтерфейсами (Cisco: interface gi0/0.10, encapsulation dot1Q 10), подключённый транком к коммутатору; трафик между VLAN гуляет через один линк. Современный вариант — L3-коммутатор с SVI-интерфейсами: маршрутизация в кремнии, без отдельного роутера. Мы проходим кадр по всей цепочке: хост отдела А шлёт пакет в другой VLAN → access-порт тегирует 802.1Q → транк → подынтерфейс → решение LPM → обратный транк → получатель. То же самое студент соберёт в лабе PT-1 и в netns (netns-версия — семинар 6, демо 12: там veth-пары уже сыграли роль «роутера на палочке»).

Конфигурация router-on-a-stick (Cisco IOS, разбираем построчно)

```
! Роутер R1: подынтерфейсы под VLAN 10 (офис) и VLAN 20 (серверы)
interface GigabitEthernet0/0
 no shutdown
!
interface GigabitEthernet0/0.10
```

```
 encapsulation dot1Q 10
 ip address 192.168.10.1 255.255.255.0
!
interface GigabitEthernet0/0.20
 encapsulation dot1Q 20
 ip address 192.168.20.1 255.255.255.0
! Коммутатор SW1: access-порты и транк в сторону роутера
interface range FastEthernet0/1 - 10
 switchport mode access
 switchport access vlan 10
interface GigabitEthernet0/1
 switchport mode trunk
 switchport trunk allowed vlan 10,20
```

Обсуждаем подводные камни: native VLAN и его безопасность, allowed vlan на транке (минимизация доверия), VTP (только упоминанием — в проде обычно выключен), двойная инкапсуляция как вектор атаки. Каждый механизм привязываем к его инженерной причине: почему транк один, почему подынтерфейсы, почему SVI быстрее.

Лабораторная работа PT-1: VLAN + маршрутизация между VLAN (6 баллов)

Задание: собрать в Packet Tracer сеть кампуса из трёх отделов (VLAN 10 «Офис», VLAN 20 «Серверы», VLAN 30 «Гости») на двух коммутаторах 2960 + роутер 2911 (router-on-a-stick). Гости — изолированно от серверов; маршрутизация между офисом и серверами разрешена. Сдача — отчёт (PDF/Markdown) с пунктами:

1. Схема сети с подписанными VLAN, портами и адресацией (скриншот + таблица).
2. Выводы show vlan brief, show interfaces trunk с обоих коммутаторов.
3. Таблица маршрутизации роутера (show ip route) и объяснение каждой connected-строки.
4. Успешные ping: офис → серверы, серверы → офис; и отрицательный результат: гости → серверы (с объяснением, почему он отрицательный — предвестие ACL недели 11).
5. Режим Simulation: скриншот кадра между VLAN с раскрытыми заголовками (802.1Q тег виден на транке).
6. Выводы: три предложения о том, что вы бы изменили в дизайне при росте сети вдвое.

Критерии: работоспособность 50%, полнота отчёта 30%, качество выводов 20%. Бонус (+2 б.): добавить DHCP для VLAN 30 (пролог недели 11) и проследить DORA-обмен в Simulation.

> **Связь с прикладной разработкой**
> Даже в «чисто софтовой» команде разработчик сталкивается с этим блоком: поды в Kubernetes живут в изолированных сетях (NetworkPolicy — тот же ACL), сервисы разделяют на VLAN/сегменты ради PCI DSS и изоляции баз данных, а «ping до базы не идёт» в 9 случаях из 10 — это ACL/сегментация, а не баг приложения. Умение читать схему сегментов и правила между ними — часть собеседования любого backend-инженера.

Вопросы для обсуждения на семинаре

- Почему не делают «одну большую плоскую сеть» с ACL на хостах? Что выигрывает сегментация на L2?
- Где в вашей учебной/рабочей инфраструктуре прячутся VLAN? Как это увидеть? (traceroute, ip addr, лабораторные стенды.)
- Router-on-a-stick vs L3-коммутатор: когда узким местом станет первый?
- Как бы вы нарезали VLAN для студенческого кампуса: сколько, по какому принципу?

Семинар 11. DHCP, NAT и ACL: адресация, выход наружу, разграничение

Одиннадцатый семинар достраивает корпоративную сеть до «полётной»: узлы получают адреса автоматически (DHCP), сеть выходит в интернет через один публичный адрес (NAT), а политика «кто к кому может ходить» реализуется ACL. Всё три темы закрепляются лабораторной PT-2. Особое внимание — conntrack: без понимания таблицы трансляций невозможно объяснить ни работу NAT, ни состояние в файрволах.

Конспект теории

DHCP (RFC 2131) выдаёт адреса арендой. Обмен DORA: Discover (клиент, broadcast), Offer (сервер предлагает адрес+параметры), Request (клиент просит предложенное), Ack (сервер подтверждает, lease-time). Параметры: маска, шлюз, DNS, lease/механизм продления (T1 = 50% срока). DHCP relay: в больших сетях сервер один, а broadcast не пересекает роутер — relay-агент пересылает запросы в unicast. Разбираем security-следствия: rogue DHCP-сервер, DHCP snooping.

NAT: сеть с приватными адресами выходит наружу через трансляцию. SNAT/masquerade заменяет адрес источника и порт, записывая трансляцию в таблицу conntrack; обратные пакеты разворачиваются по ней. DNAT/port-forwarding пробрасывает входящие порты внутрь. Следствия NAT: снаружи нет «прямых» адресов (P2P-проблема, STUN/TURN — тема мини-исследования), таймауты трансляций (почему нужен TCP keepalive), FTP/ALG-страдания. В Linux NAT — обычное правило nftables/iptables в таблице nat; в Cisco PT — overload (PAT).

NAT на Linux: минимальный nftables-набор для шлюза netns-лабы

```
table ip nat {
  chain postrouting {
    type nat hook postrouting priority srcnat; policy accept;
    oifname "wan0" ip saddr 10.0.0.0/8 masquerade
  }
  chain prerouting {
    type nat hook prerouting priority dstnat; policy accept;
    iifname "wan0" tcp dport 8080 dnat to 10.0.2.2:80   # port-forward
  }
}
# conntrack: посмотреть живые трансляции на шлюзе
sudo conntrack -L | head
```

ACL: упорядоченный список правил «разрешить/запретить» с первого совпадения; standard-ACL фильтрует по адресу источника, extended — по «источник/получатель/протокол/порт». Правила прикладываем близко к точке запрета (extended — ближе к источнику, чтобы не гонять трафик зря), неявный deny any в конце. Разбираем типовые кейсы: бухгалтерия изолирована от серверов, гости — только в интернет, админ-доступ только с bastion. Сопоставляем с nftables: там же идея «порядок правил решает», плюс состояния (ct state established,related accept) — файрвол с состоянием короче и надёжнее.

Лабораторная работа PT-2: DHCP + NAT + ACL (6 баллов)

1. Сеть: продолжение PT-1 (или новая): VLAN 10/20/30, роутер, внешний «интернет» (симулированный ISP + сервер 8.8.8.8-подобный).
2. DHCP: пул для VLAN 30 (гости) и VLAN 10 (офис) на роутере; серверы — статика. Скриншоты ipconfig / DHCP-запросов в Simulation.
3. NAT: настройка overload, доказательство: ping с офисного ПК на «интернет», show ip nat translations на роутере.
4. ACL: (а) гости → только интернет, запрет к 192.168.10.0/24 и 20.0/24; (б) офис → серверы только по HTTPS; (в) блок telnet на роутер извне VLAN 30. Скриншоты show access-lists с счётчиками совпадений.
5. Отрицательные проверки: ping/HTTP из VLAN 30 в VLAN 20 должны падать с объяснением.
6. Выводы: что произойдёт с сетью при выходе из строя NAT-шлюза и как это исправить (мостик к HA — неделя 14).

Критерии те же: работоспособность 50%, отчёт 30%, выводы 20%. Бонус (+2 б.): DHCP relay через подынтерфейс вместо локальных пулов.

Вопросы для обсуждения на семинаре

- Почему NAT «сломал» сквозность интернета, но никто от него не отказывается? Что IPv6 меняет в этом сюжете?
- Как conntrack отличает «ответ» от «нового входящего»? Что будет при SYN от снаружи на немаркированный порт?
- Почему порядок ACL-правил критичен? Сформулируйте правило «как не писать мёртвые правила».
- DHCP snooping и dynamic ARP inspection: какие угрозы они закрывают и чем платим за включение?

Семинар 12. QoS: приоритизация трафика, очереди, netem

Двенадцатый семинар — про то, что «скорость» — не единственная метрика сети. Голос важнее закачки; резервное копирование не должно мешать звонкам. Мы разбираем полный конвейер QoS: классификация → маркировка (DSCP) → очереди → police/shape, и закрепляем лабораторной в netns с tc. Это единственная лаба блока, где студенты измеряют результат: до/после на графиках.

Конспект теории

Классификация и маркировка. Пакет классифицируется по 5-кортежу или по приложению и маркируется DSCP (6 бит в поле TOS IPv4 / Traffic Class IPv6): EF (46) — голос, AF41 — видеоконференции, AF11 — «деловая» закачка, CS0/BE — всё прочее. Метка ставится ближе к источнику и уважается транзитом. **Очереди**: FIFO — всем поровну и всем плохо; priority queuing — строгий приоритет (голодание низких классов без лимитов); CBWFQ/WFQ — гарантированные доли полосы; fq_codel — современный дефолт Linux, борется с bufferbloat. **Shape vs police**: shaper задерживает сверхурочные пакеты (сглаживает, добавляет latency), policer выбрасывает (жёсткий потолок, потери).

Инструментальная часть: tc qdisc — дисциплина обслуживания на egress; классовые qdisc (htb) создают дерево классов с гарантиями полосы и приоритетами; netem эмулирует задержку/потери/джиттер — наш тестовый стенд. В PT — priority queueing на интерфейсе роутера. Ключевая идея для разработчика: QoS-разметка может идти прямо из приложения (setsockopt IP_TOS) — и её видит tcpdump, а значит её можно проверять автотестом стенда.

Дисциплина	Идея	Плюсы	Минусы
FIFO	одна очередь, кто первым — тот и ушёл	нулевая сложность	bufferbloat, приоритетов нет
PQ (strict)	строгий приоритет классов	минимальная латентность EF	голодание низких классов
CBWFQ/WFQ	гарантированные доли полосы	справедливость, предсказуемость	настройка классов сложнее
fq_codel	на каждую очередь — AQM по времени стояния	борьба с bufferbloat из коробки	меньше контроля по классам
TBF	token bucket: средняя скорость + всплеск	простой лимитер	не про приоритеты

Сравнение дисциплин очередей

Лаба-скелет: HTB с двумя классами в netns (фрагмент лабораторной)

```
#!/bin/bash
# qdemo.sh: канал 10 Мбит, голос EF получает приоритет, bulk — остаток
NETNS=router; IF=v-l-r
ip netns exec $NETNS tc qdisc replace dev $IF root handle 1: htb default 30
ip netns exec $NETNS tc class add dev $IF parent 1: classid 1:1 htb rate 10mbit
ip netns exec $NETNS tc class add dev $IF parent 1:1 classid 1:10 htb rate 8mbit ceil
        10mbit prio 0
ip netns exec $NETNS tc class add dev $IF parent 1:1 classid 1:30 htb rate 1mbit ceil
        10mbit prio 7
# Фильтр по DSCP EF (0x2e<<2 = 0xb8) в класс голоса:
ip netns exec $NETNS tc filter add dev $IF parent 1: protocol ip prio 1 u32 \
   match ip tos 0xb8 0xfc flowid 1:10
# Замеры: iperf3 без маркировки vs с TOS=EF (см. задание лабы)
```

Лабораторная работа «QoS в netns»: замеры и отчёт (6 баллов)

1. Собрать топологию демо 12 (left–router–right) + qdemo.sh; убедиться в канале 10 Мбит.
2. База: iperf3 одним потоком — записать throughput/RTT.

3. Конкуренция: два параллельных iperf3 (по 8 параллельных соединений) — записать распределение полосы без QoS.
4. Включить классы HTB; повторить конкуренцию, один поток помечен EF (iperf3 -S 0xB8); записать распределение.
5. Задержка: ping во время закачки до/после QoS (эффект bufferbloat/очередей).
6. Отчёт: схема, команды, таблица замеров, 2–3 графика (можно в gnuplot/LibreOffice) и выводы: кто выиграл, кто заплатил, за счёт чего.

Критерии: корректность стенда и замеров 50%, отчёт и графики 30%, выводы 20%. Бонус (+2 б.): добавить netem delay 20ms в класс bulk и показать, что EF-поток латентность не почувствовал.

> **Связь с прикладной разработкой**
> Приложения сами участвуют в QoS: пометка DSCP для realtime-трафика (Zoom и игры это делают), размер буферов и интервалов отправки, реакция на ECN. Разработчик, который понимает shape vs police, по-другому проектирует облой трафика: вместо «валить всё в сеть и надеяться» — сегментировать и маркировать. А явление bufferbloat объясняет, почему «пинг 300 мс при закачке» — не всегда вина провайдера.

Вопросы для обсуждения на семинаре

- Почему strict priority без лимитов «морит голодом» низкие классы? Как спасают ceil/guarantees в HTB?
- Если весь трафик вдруг помечен EF — что даст QoS? (Ответ: ничего. QoS — про различие, а не про магию.)
- Чем policer страшнее shaper для TCP? Вспомните AIMD: что делают потери с окном?
- Где заканчивается юрисдикция QoS приложения и начинается сеть? Что должно быть в SLA, чтобы инженер мог «честно» ответить за голос?

НЕДЕЛИ 13–14 · ОТ РЕЗОЛВИНГА ДО STICKY-SESSION

7. Блок V. Прикладная инфраструктура: DNS, HTTP, прокси, балансировка

Четвёртый блок поднимается к прикладному уровню — но не в сторону веб-разработки, а в сторону инфраструктуры, на которой живут все сервисы: как имя превращается в адрес (DNS), как устроен HTTP и что изменил HTTP/2, что такое TLS-рукопожатие, и как reverse-proxy с балансировщиком собирают из нескольких слабых бэкендов один отказоустойчивый сервис. Обе недели — с живыми лабораториями; неделя 13 завершается программным заданием hw-13 (мини-HTTP-сервер).

Семинар 13. DNS и HTTP: резолвинг, кэши, версии протокола, TLS

Семинар начинается с вопроса «что происходит, когда вы ввели example.com и нажали Enter» и честно доходит до TLS: students сами находят все уровни курса в одном обмене. DNS разбираем как первую распределённую БД с кэшированием (TTL), HTTP — как язык, на котором говорят сервисы между собой.

Конспект теории: DNS

DNS (RFC 1034/1035) — иерархическое пространство имён: корень, TLD, зоны, авторитетные серверы. Запрос бывает итеративным (сервер говорит «иди к NS такой-то») и рекурсивным (резолвер сам обходит иерархию) — в жизни клиент всегда рекурсивно спрашивает свой резолвер. Кэширование по TTL на каждом уровне; отрицательное кэширование тоже существует. Типы записей: A/AAAA, CNAME, MX, NS, TXT, PTR. Прикладной интерес: время жизни TTL прямо управляет скоростью миграций сервисов; DNS-балансировка — грубый, но рабочий инструмент. Современность: DNS over TLS/HTTPS, stub vs recursive vs authoritative — все три мы видим в dig +trace.

Демо 20. Путь DNS-запроса до корня и ответ

```
dig +trace example.com            # обход иерархии: root -> tld -> ns
dig +noall +answer example.com A  # сам ответ с TTL
dig +noall +answer www.example.com CNAME example.com A  # цепочка алиасов
sudo tcpdump -n port 53 &         # и тот же запрос глазами Wireshark
```

Тип	Что хранит	Где встретится
A / AAAA	IPv4 / IPv6 узла	любой сервис
CNAME	алиас на другое имя	балансировка облаков, CDN
MX	почтовый сервер домена (с приоритетом)	почта
NS	авторитетные серверы зоны	делегирование, dig +trace
TXT	произвольный текст	SPF/DKIM, проверка владения доменом
PTR	обратное соответствие IP → имя	почтовые серверы, логи
SOA	параметры зоны (серийный номер, таймеры)	администрирование зоны

Типы DNS-записей, нужные разработчику

Конспект теории: HTTP и TLS

HTTP/1.1 (RFC 9110/9112): текстовые сообщения, методы, коды статусов, заголовки, cookies, keep-alive (переиспользование соединения — тема, которая станет важной на неделе 14). HTTP/2 (RFC 9113): бинарное фрейминг-представление, **мультиплексирование** многих потоков в одном TCP-соединении (решение head-of-line blocking на прикладном уровне, но не на транспортном — почему появился HTTP/3/QUIC — тема мини-исследования), сжатие заголовков HPACK, server push (историческая страница). TLS (RFC 8446, TLS 1.3): рукопожатие обменивается криптопараметрами и сертификатами, 1-RTT (и 0-RTT для повторов), SNI «открывает» имя сайта во время handshake. Для разработчика важно: TLS шифрует payload, но не всё метаданные; handshake — лишний RTT, отсюда keep-alive и connection pooling в клиентах.

Код	Смысл	Типовая причина в сервисе
200/204	успех (с телом/без)	норма
301/302	перенаправление	смена домена, HTTP→HTTPS
304	не изменилось (кэш)	ETag/If-Modified-Since сработали
400	битый запрос	нарушен синтаксис, мусор в заголовках (hw-13)
401/403	нет аутентификации/доступа	токен не передан; выход за docroot (../)
404	нет ресурса	опечатка в пути; файл удалён
405	метод не поддерживается	POST на статике
429	слишком много запросов	сработал rate-limit на прокси
500	внутренняя ошибка	исключение в бэкенде
502	плохой шлюз	прокси не смог поговорить с бэкендом (упал/закрыл)
503	недоступен	переполнен maxconn/очередь; деплой в процессе
504	таймаут шлюза	бэкенд молчит дольше timeout server

Коды статусов HTTP, которые разработчик обязан узнавать в лицо

Демо 21. HTTP и TLS вживую

```
curl -sv http://example.com/ -o /dev/null        # -v: заголовки, версии,Timing
curl -sv --http2 https://example.com/ -o /dev/null  # ALPN выбирает h2
openssl s_client -connect example.com:443 -brief </dev/null  # handshake: TLS1.3,
                                                    # сертификат, шифры, ALPN
```

Практика на семинаре

- Зарисовать по выводу dig +trace цепочку «stub → recursive → root → TLD → authoritative»; пометить, где кэш сработал бы.
- Сравнить curl -v по HTTP/1.1 и HTTP/2: найти мультиплексирование в выводе; объяснить, зачем одна страница открыла 1 соединение вместо 6.

- Разобрать TLS-handshake из openssl s_client -brief: версия, набор шифров, сертификат, ALPN; обсудить, что видит и не видит DPI.
- Убить резолвер (в netns) и наблюдать таймауты curl — диагностика «сервис есть, имя не резолвится».

Самостоятельная работа до семинара 14

- Таненбаум, гл. 7: DNS и WWW-разделы; RFC 9110 — только обзорная часть.
- Прочитать man haproxy (вводная часть) и подготовиться к лабе недели 14.
- Сдать заявку на тему мини-исследования (каталог — глава 9).

Домашнее задание hw-13: мини-HTTP-сервер (8 баллов, дедлайн — неделя 14)

Студент реализует HTTP/1.1-сервер на сокетах: принимает соединения, разбирает запрос, отдаёт файлы из docroot, корректно закрывает соединения. Тесты в CI запускают сервер на 127.0.0.1:0 (случайный порт из вывода) и общаются нативным клиентом на сокетах — так проверяется и парсинг, и сетевая часть. Сервер обязан выдерживать параллельные запросы (thread/fork/epoll — на выбор).

include/httpd.h — интерфейс задания hw-13 (фрагмент)

```
#ifndef HTTPD_H
#define HTTPD_H
#include <stddef.h>
#include <stdint.h>
/* Разбор запроса из буфера (может содержать больше одного запроса:
 * возвращается число СЪЕДЕННЫХ байт первого запроса).
 * Возвращает 0 при успехе; -1 — незавершённый запрос; -2 — синтаксис. */
typedef struct {
    char method[8];            /* GET, HEAD */
    char path[1024];           /* без query string; проверяется на '..' */
    char version[9];           /* "HTTP/1.1" */
    /* заголовки: до 32 пар, ключи в нижнем регистре */
    struct { char key[64]; char val[256]; } headers[32];
    int  n_headers;
} http_request_t;
long http_request_parse(const uint8_t *buf, size_t len, http_request_t *out);
/* Сервер: слушает порт (0 = случайный, фактический — через out_port),
 * отдаёт файлы из docroot по GET/HEAD. 404 — нет файла, 400 — мусор,
 * 403 — выход за пределы docroot (../), 405 — не GET/HEAD.
 * Content-Type по расширению: html/txt/png/jpg/jpeg/css/js.
 * Заголовки ответа: Content-Length, Connection. Поддержка keep-alive
 * для HTTP/1.1 без "Connection: close". Работает, пока не вернёт
 * ошибку фата; тесты запускают его в отдельном потоке. */
int httpd_run(uint16_t port, const char *docroot, uint16_t *out_port);
#endif /* HTTPD_H */
```

- **Что проверяют автотесты.** Публичные: GET существующего файла, 404, запрос с «Connection: close», заголовки ответа (Content-Length обязателен). Скрытые: обход пути /../etc/passwd (обязан быть 403), два запроса в одном пакете (keep-alive), HEAD без тела, параллельные 50 клиентов (threads), битые запросы (обязан ответить 400 и не упасть), запрос с разбиением заголовков по двум пакетам (TCP-фрагментация прикладного сообщения!).

- **Критерии.** Тесты 70%, код 20%, README с описанием архитектуры 10%: как обрабатываются соединения, где границы запроса.
- **Бонус (+3 б.)**: sendfile-подобная отдача больших файлов без лишних копий и тест на файл 50 МБ.

> **Связь с прикладной разработкой**
> hw-13 — уменьшенный nginx. После него студент понимает, почему nginx «вдруг» отдаёт 400 (между запросами сломали keep-alive), почему 502 означает «прокси не смог поговорить с бэкендом», и зачем нужны Content-Length и chunked encoding. Анализ «HEAD vs GET» и путей с «..» — это, кстати, первые строки настоящего security-ревью любого HTTP-сервера.

Вопросы для обсуждения на семинаре

- Почему TTL DNS-записи — инженерное решение, а не техническая деталь? Что происходит при миграции сервиса с TTL=300 против TTL=86400?
- HTTP/2 решил head-of-line blocking на уровне сообщений — почему TCP-блокировка осталась и чем её лечит HTTP/3?
- Что TLS скрывает, а что — нет? Какие данные остаются видимыми провайдеру?
- Keep-alive: почему переиспользование соединения ускоряет даже «мелкие» API в разы? Посчитайте RTT.

Семинар 14. Reverse-proxy, кэширование, HAProxy, балансировка, sticky-session

Четырнадцатый семинар — сборочный: из всего, что изучено, собирается архитектура реального веб-сервиса. Reverse-proxy закрывает сервисы от прямого доступа, терминирует TLS и кэширует; балансировщик распределяет нагрузку и следит за здоровьем; sticky-session лечит «сессии в памяти бэкенда». Всё это студент собирает в лаборатории и измеряет.

Конспект теории

Reverse-proxy — сервер-представитель перед бэкендами: принимает трафик клиентов, по правилам передаёт внутрь. Функции: терминация TLS (бэкенды говорят по HTTP внутри), кэширование (nginx proxy_cache, уважение Cache-Control и Vary), сжатие, лимиты, безопасность (скрытие бэкендов, WAF перед входом), единая точка логирования. Forward-proxy, в отличие, представляет клиентов (корпоративный выход наружу).

HAProxy — эталонный L4/L7-балансировщик. Конфиг: frontend (где слушаем) → default_backend → backend (список серверов, алгоритм, health-чеки). Алгоритмы: roundrobin (равные доли), leastconn (менее загруженному — для неравномерных запросов), source (хэш адреса клиента — «стикiness» без cookies), uri (кэш-френдли). Health-чеки выводят сервер из ротации (check, inter, fall/rise); страница статистики — витрина состояния. **Sticky-session**: cookie-вставка (HAProxy выдаёт cookie с ID сервера) или source-hash (по IP клиента) — нужны, когда состояние сессии живёт в бэкенде; правильное решение вообще-то выносить состояние в Redis/БД — проговариваем это как инженерный принцип.

Отказоустойчивость: timeout connect/server/client, retries, maxconn, очереди при переполнении; сценарий «бэкенд умер» студент увидит вживую: health-check снимает узел, трафик перераспределяется, «убитый» возвращается rise-ами. Разбираем уровни балансировки: L4 (по TCP, быстрее, слепее) vs L7 (по HTTP, умнее: маршрутизация по path/заголовкам, sticky, сжатие).

Алгоритм	Как распределяет	Применять, когда
roundrobin	по кругу, равные доли	бэкенды одинаковые, запросы однородные
leastconn	тому, у кого меньше активных соединений	долгие соединения, неравномерные запросы
source	хэш адреса клиента	простая «липкость» без cookie (за NAT — перекосы)
uri	хэш URL	кэш на бэкендах: один URL — один бэкенд
first	первый свободный, остальные — в резерв	экономия энергии, тёплые апгрейды
random	случайно, опционально power-of-two-choices	массовые короткие запросы, простота

Алгоритмы балансировки HAProxy: когда какой

Минимальный HAProxy-конфиг лабы (разбираем построчно)

```
global
    log stdout format raw local0
defaults
    mode http
    timeout connect 3s
    timeout client  30s
    timeout server  30s
frontend fe_main
    bind *:8080
    default_backend be_app
backend be_app
    balance roundrobin
    cookie SRV insert indirect nocache      # sticky по cookie
    option httpchk GET /health
    http-check expect status 200
    server app1 10.200.1.2:8000 check cookie app1
    server app2 10.200.2.2:8000 check cookie app2
listen stats
    bind *:8404
    stats enable
    stats uri /stats
```

Лабораторная работа «HAProxy: балансировка и отказоустойчивость» (6 баллов)

1. Топология в netns (или docker-compose): два «бэкенда» (python3 -m http.server с разным index.html или маленький собственный сервер из hw-13), nginx как reverse-proxy при желании, HAProxy впереди.
2. Проверить roundrobin: несколько curl — ответы чередуются; сменить на leastconn и source — описать разницу поведения.
3. Sticky: включить cookie-вставку, убедиться, что один клиент «прилипает» (curl -c/-b cookie jar); объяснить, когда sticky вреден.

4. Отказы: убить app1 (SIGKILL) — трафик уходит на app2 до и после закрытия; показать на /stats момент вывода и возврата сервера.
5. Замеры: wrk/ab на 30 сек: один бэкенд vs два vs два с sticky; записать RPS и p99 latency.
6. Отчёт: конфиги, скриншоты /stats, таблица замеров, выводы: где упёрлись и почему.

Критерии: работающий стенд 40%, сценарии отказов 30%, замеры и выводы 30%. Бонус (+2 б.): кэширующий nginx перед HAProxy с Cache-Control и демонстрация попаданий/промахов в логах.

> **Связь с прикладной разработкой**
> Это самый «бэкендерский» семинар курса: 502/504 из логов — это timeouts HAProxy; «пользователя выкинуло из сессии» — балансировка без sticky или stateless-бэкенд, у которого забыли общий стор; «двое видят разный контент» — кэш и Vary. Понимание слоёв «клиент → CDN → прокси → балансировщик → бэкенды» — то, что спрашивают на любом собеседовании по backend/DevOps, и то, что отличает разработчика от «человека, который пишет контроллеры».

Вопросы для обсуждения на семинаре

- Почему L7-балансировка медленнее L4, но почти всегда используется в вебе? Где выигрывает L4?
- Какой алгоритм выбрать для: (а) долгих WebSocket-соединений, (б) коротких API-запросов, (в) Mix-кэша по URL?
- Что случится с sticky-сессиями при выходе бэкенда из строя? Как проектируют сессии, чтобы их не бояться?
- Сколько «девяток» даёт ваш стенд из лабы? Что нужно, чтобы посчитать это честно (какие отказы моделировали, какие нет)?

НЕДЕЛИ 15–16 · ПРОГРАММИРУЕМОЕ ЯДРО И ПОЛНЫЙ ПУТЬ ПАКЕТА

8. Блок VI. eBPF/XDP и сетевое ядро Linux

Финальный блок курса соединяет всё изученное с устройством операционной системы. Пятнадцатая неделя знакомит с eBPF — технологией, изменившей сетевой и наблюдаемый стек Linux: программы в ядре, безопасно проверяемые верификатором, работающие на скорости сетевой карты. Шестнадцатая неделя собирает весь курс в одну картину: полный путь пакета от сетевой карты до сокета приложения и обратно, влияние этого пути на разработку — и завершается итоговым опросом и защитами мини-исследований.

Семинар 15. eBPF и XDP: программируемое ядро

Семинар начинается с мотивирующего вопроса: «как отбросить миллион пакетов в секунду до того, как ядро затратит на них память?». Ответ — XDP: программа, прикреплённая к сетевому интерфейсу и решающая судьбу каждого пакета на самой ранней стадии. Мы разбираем архитектуру eBPF, ограничения верификатора, карты как способ общения с ядром — и вживую грузим и модифицируем XDP-программы из учебного boilerplate.

Конспект теории

eBPF — виртуальная машина ядра: программа на ограниченном ассемблере (из C через clang), которая загружается в ядро, проходит **верификацию** (статический анализ: нет бесконечных циклов без ограничения, все обращения к памяти проверены, нет разыменования произвольных указателей), затем JIT-компилируется в машинный код. Программы прикрепляются к «крючкам»: XDP (пакет на входе интерфейса), tc (классификация трафика), tracepoints/kprobes (трассировка ядра), сокеты. Обмен данными с userspace — через **карты** (maps): массивы, хэш-таблицы, LPM-таблицы (знакомая структура из hw-6!), кольца событий. Программы не могут звать libc — только хелперы ядра; поэтому код короткий, специализированный и жёстко проверяемый.

XDP работает до выделения sk_buff — прямо на страницах DMA-кольца драйвера, поэтому это самый быстрый путь обработки: типовые цифры на учебном железе — миллионы решений в секунду на ядро. Программа возвращает **действие**: XDP_PASS (продолжить стек), XDP_DROP (отбросить), XDP_TX (отправить обратно тем же интерфейсом), XDP_REDIRECT (другому интерфейсу/в userspace). Кейсы: анти-DDoS (Cloudflare отбрасывает атаки на линуксах так), балансировка katran (XDP-версия того же LPM+hash, что мы изучали), NAT, телеметрия. Ограничения честно проговариваем: нет доступа к потокам TCP (пакет за пакетом), ограничение на инструкции, зависимость фич от драйвера (native vs generic режим).

Инструментарий: clang компилирует C в объектный файл eBPF; загрузка через libbpf или bpftool; разбор состояния — bpftool prog show, bpftool map dump; прицепление — ip link set dev eth0 xdp obj prog.o sec xdp. Курс использует подготовленный boilerplate: Makefile, скелет программы, тест-карты. Студенту важно освоить цикл «правим C → грузим → смотрим счётчики в карте → снимаем программу» — это и есть повседневная работа с eBPF.

Демонстрации на семинаре

Демо 22. Учебная XDP-программа: фильтр ICMP со счётчиком (boilerplate курса)

```
#include <linux/bpf.h>
#include <bpf/bpf_helpers.h>
#include <linux/if_ether.h>
#include <linux/ip.h>
struct {                    /* карта-счётчики: одно значение на всё */
    __uint(type, BPF_MAP_TYPE_ARRAY);
    __uint(max_entries, 2);
    __type(key, __u32);     /* 0 = пропущено, 1 = отброшено */
    __type(value, __u64);
} counters SEC(".maps");
SEC("xdp")
```

```
int xdp_icmp_filter(struct xdp_md *ctx) {
    void *data = (void *)(long)ctx->data;
    void *data_end = (void *)(long)ctx->data_end;
    struct ethhdr *eth = data;
    if ((void *)(eth + 1) > data_end)          /* обрезанный кадр */
        return XDP_DROP;
    if (eth->h_proto != __constant_htons(ETH_P_IP))
        return XDP_PASS;                       /* не IPv4 — мимо фильтра */
    struct iphdr *ip = (void *)(eth + 1);
    if ((void *)(ip + 1) > data_end)
        return XDP_DROP;
    __u32 key = (ip->protocol == IPPROTO_ICMP) ? 1 : 0;
    __u64 *cnt = bpf_map_lookup_elem(&counters, &key);
    if (cnt) __sync_fetch_and_add(cnt, 1);
    return key ? XDP_DROP : XDP_PASS;
}
char LICENSE[] SEC("license") = "GPL";
```

Демо 23. Жизненный цикл: собрать, загрузить, проверить, снять

```
make                                  # clang -target bpf -> xdp_icmp.o
sudo ip link set dev eth0 xdp obj xdp_icmp.o sec xdp
ping -c 3 8.8.8.8                     # 100% packet loss — ICMP отброшен
curl http://example.com -o /dev/null  # а HTTP жив: XDP_PASS
sudo bpftool prog show                # увидеть нашу программу и stats
sudo bpftool map dump name counters   # counters[1] растёт от ping
sudo ip link set dev eth0 xdp off     # сняли — всё вернулось
```

На защите задания студенты модифицируют программу: заменяют жёсткий ICMP-фильтр на политику из карты-списка адресов (hash map с префиксами) и добавляют счётчики по адресам. Разбор демо 22 на семинаре подводит к этой модификации почти дословно — увидеть разницу между «скопировать» и «понять» поможет устный вопрос «почему тут XDP_DROP, а не XDP_PASS при обрезанном кадре».

Практика на семинаре

- Собрать и загрузить демо 22; снять дамп до/после (tcpdump): ICMP исчезает из эфира целиком — обсудить, где «живёт» DROP.
- bpftool prog show / map dump: прочитать run_time_ns/run_cnt — сколько ядра стоит фильтр; сравнить с выводом iperf3 до/после.
- Модификация в парах: перенести порог фильтра в карту (hash), перезагрузить программу без падения интерфейса.
- Обсудить: где XDP не поможет (анализ TCP-состояний, после-роутинг), и почему тогда существуют tc-хуки.

Самостоятельная работа до семинара 16

- Дописать hw-15 (см. ниже) и записаться на слот демонстрации.
- Прочитать: документация ядра Documentation/bpf (вводные разделы XDP) — 20 страниц о том, что мы сегодня руками делали.
- Подготовить выступление по мини-исследованию (7 минут): слайды или живой стенд.

Домашнее задание hw-15: политика XDP из карты (8 баллов, демо на VM)

Задание выполняется в предоставленной VM (root, ядро 5.15+). GitHub Actions проверяет сборку (clang -target bpf, чистые предупреждения) и наличие обязательных секций; работа программы демонстрируется на виртуальной машине преподавателя. Требования:

- Политика фильтрации хранится в BPF_MAP_TYPE_LPM_TRIE: префиксы, которые надо отбрасывать (например, 10.0.0.0/8), загружаются из userspace-утилиты.
- Две карты-счётчика: passed/dropped — с приращением атомарно; утилита в userspace печатает их раз в секунду (аналог bpftool map dump).
- Граничные проверки байтов обязаны покрывать все чтения заголовков — верификатор должен пропустить программу без признаков «speculative»-хаков.
- README: как собрать, как загрузить, три команды проверки и вывод счётчиков.
- **Бонус (+3 б.)**: режим XDP_TX-эхо для определённого порта (отдельная карта) и демонстрация «сервера» прямо в ядре.

> **Связь с прикладной разработкой**
> eBPF — среда, где «системное программирование» снова стало массовым: балансировщики (Katran в Meta), анти-DDoS (Cloudflare), наблюдаемость (bpftrace, Prometheus-экспортёры), сервисные сети (Cilium). Прикладному разработчику eBPF даёт суперспособность — отвечать на вопрос «что реально делает моя программа в ядре» без перекомпиляции: задержки syscalls, очередь сокета, потери в TCP. Порог входа выше, чем у strace, но потолок диагностики — несравнимый.

Вопросы для обсуждения на семинаре

- Почему верификатор строже компилятора? Какие классы ошибок он исключает в принципе, а какие — только «усложняет»?
- Почему XDP быстрее, чем iptables DROP? Нарисуйте путь пакета для обоих случаев.
- Что произойдёт с картами и программой при перезапуске userspace-процесса? А при выгрузке модуля драйвера?
- Как бы вы применили eBPF в сервисе, который пишете сейчас? Что измеряли бы?

Семинар 16. Путь пакета в ядре Linux; итоги курса; защиты исследований

Заключительный семинар — синтез. Мы рисуем на доске один большой путь: пакет приходит на NIC, попадает в DMA-кольцо, его встречает XDP, затем NAPI, затем sk_buff проходит tc, IP-стек, conntrack/nftables, TCP-стек и ложится в буфер сокета, откуда приложение читает recv(). Каждый пункт этого пути — семинар нашего курса, и студент должен уметь назвать, что происходит и как это проверить. После разбора — итоговый опрос и защиты мини-исследований.

Конспект теории: путь пакета и что из него видно приложению

Приём. NIC пишет кадр DMA-ом в кольцо описателей; контроллер прерывает процессор только раз за пачку (прерывание-коалесинг), а остальное добирает поток NAPI: мягкое прерывание (softirq) вычитывает кольцо, создаёт sk_buff. Ранний хук XDP (мы владеем им практически — неделя 15), затем tc-классификация, маршрутное решение (наш LPM из hw-6), conntrack (таблица трансляций из недели 11), nftables-фильтры (ACL-семантика недели 11), TCP-стек (FSM недели 8, окна недели 9) и, наконец, буфер приёмного сокета. **Отправка**: send() копирует данные в буфер сокета, TCP пакует их в сегменты (Nagle, cwnd), IP маршрутизирует, tc ставит в очереди qdisc (наш HTB из недели 12), драйвер пишет в TX-кольцо.

Отсюда — практические следствия для разработчика. **Буферы**: recv/send возвращают управление не тогда, когда «сеть передала», а когда ядро скопировало байты — поэтому SO_SNDBUF/SO_RCVBUF и backpressure реальные. **Порты и масштабирование**: SO_REUSEPORT позволяет нескольким процессам слушать один порт — ядро распределяет соединения (C10K решается ядерно). **Ноль копий**: sendfile/splice экономят копирование — поэтому nginx отдаёт статику быстро. **Латентность**: каждый слой добавляет очереди; измерять нужно end-to-end, а не «внутри приложения» (eBPF-трассировка делает это честно). Все эти вопросы — финальный опрос, и все они уже встречались студентам по ходу курса.

Итоговый опрос (20 минут, письменно, до 6 баллов активности)

Десять коротких вопросов, проверяющих «карту мира» курса, а не детали: перечислить уровни TCP/IP и их задачи; объяснить, почему MAC меняется, а IP нет; описать LPM; нарисовать handshake TCP и объяснить TIME_WAIT; сравнить UDP и TCP для голосового трафика; объяснить, что делает NAT и что такое conntrack; рассказать, как DSCP помогает голосу; объяснить работу reverse-proxy и sticky-сессий; перечислить действия XDP; перечислить пять причин, почему «приложение тормозит», глядя на вывод ss и ip route.

Защиты мини-исследований (остаток семинара, ~10 минут на доклад)

Формат: 7 минут выступления + 3 минуты вопросов. Оценивается: корректность методики (для прикладных тем — измерений), глубина понимания (для теоретических — умение объяснить «простыми словами» без утраты смысла), качество ответов на вопросы. Темы и критерии — в главе 9. Жюри: преподаватель + две пары «экспертов» из студентов, оценивающих по заранее объявленной шкале.

> **Что дальше**
> Курс оставляет три «двери» для продолжения: сетевое программирование (Стивенс, эпиллинг-серверы, DPDK), сетевая инфраструктура (мини-исследования перерастают в курсовые по BGP/SDN) и наблюдаемость (eBPF-трассировка как рабочая профессия). Литература — в приложении Г; репозиторий курса с задачами остаётся доступным, скрытые тесты публикуются после защиты.

Вопросы для обсуждения на семинаре (закрывающие)

- За полгода мы прошли путь от MAC-адреса до eBPF. Какой механизм surprised больше всего и почему?
- Какие два знания курса вы примените в ближайшие месяцы в своих проектах? Конкретно: команда/настройка/подход.
- Что в курсе стоило бы углубить, а что — сократить? (Обратная связь идёт в планирование следующего года.)

ОДНО ИССЛЕДОВАНИЕ НА КАЖДОГО · ЗАЯВКА ДО НЕДЕЛИ 12 · ЗАЩИТА НА НЕДЕЛЕ 16

9. Мини-исследования: каталог тем

Мини-исследование — индивидуальная (или парная, по согласованию) работа, заменяющая экзамен: она проверяет способность применить материал курса самостоятельно. Студент выбирает тему из каталога ниже или предлагает свою (при согласовании преподавателя). До недели 12 сдаётся заявка: тема + план в трёх предложениях + ожидаемый результат. Итог — отчёт на 4–6 страниц (введение, методика, результаты с числами и графиками, выводы, список источников) и выступление 7 минут с 3 минутами вопросов.

Оценивание: отчёт — до 15 баллов (методика 5, данные и их корректная интерпретация 5, выводы 5), выступление — до 10 баллов (содержание 5, ответы на вопросы 5). Сложность темы помечена звёздами: одна звезда — достаточно аккуратной работы по инструкции, три — требуется собственная проектная часть. Прикладные темы проверяются замерами, теоретические — качеством объяснения: «простыми словами» не значит «грубо», а значит «без потери смысла».

9.1. Прикладные темы: конфигурации, замеры, отчёт о сравнении

Тема 1. HAProxy против nginx как reverse-proxy (2 звезды)

- **Цель.** Сравнить два reverse-proxy по пропускной способности, latency (p50/p99) и поведению под нагрузкой в одинаковой роли.
- **Что делать.** Стенд из лабы недели 14 (два бэкенда); две конфигурации: nginx proxy_pass и HAProxy с эквивалентной логикой; замеры wrk: 4 конфигурации нагрузки (лёгкая, средняя, перегрузка, перегрузка + медленный бэкенд).
- **Инструменты.** netns или docker-compose, nginx, HAProxy, wrk, tcpdump для контроля честности стенда.
- **Что сдать.** Отчёт: конфиги, таблицы замеров, графики RPS/latency, вывод «какой и когда выгоднее», список факторов, не вошедших в замеры.

Тема 2. Алгоритмы балансировки и sticky-сессии (2 звезды)

- **Цель.** Показать различие roundrobin / leastconn / source / cookie-вставки на измеримом поведении, включая сценарии отказов и неравномерных запросов.

- **Что делать.** Бэкенды с разной скоростью обработки (один «тормозит» через sleep); измерить распределение запросов и p99 для каждого алгоритма; сценарий: убить бэкенд под нагрузкой, вернуть — проследить пере-распределение.
- **Инструменты.** HAProxy, wrk, скриптованные бэкенды (Flask/Go/собственный сервер из hw-13), страница /stats.
- **Что сдать.** Отчёт с матрицей «алгоритм × сценарий», вывод о связи выбора алгоритма с типом нагрузки и состоянием сессий.

Тема 3. CUBIC против BBR на канале с потерями (3 звезды)

- **Цель.** Экспериментально показать разницу классического loss-based и модельного congestion control на «плохих» каналах.
- **Что делать.** netns + netem (loss 0/1/2/5%, delay 10/50 мс); iperf3 с sysctl net.ipv4.tcp_congestion_control=cubic и bbr; снять throughput и RTT; объяснить результаты через AIMD против BBR-модели.
- **Инструменты.** iperf3, tc/netem, ss -ti (cwnd в живую!), gnuplot.
- **Что сдать.** Отчёт с матрицей замеров, графиком «throughput от loss%», объяснением механизма и честным разделом «где BBR не поможет».

Тема 4. Потери, джиттер и HTTP: цена keep-alive (2 звезды)

- **Цель.** Измерить, как потери/джиттер и режим соединений (keep-alive против Connection: close) влияют на время выполнения пачки HTTP-запросов.
- **Что делать.** Стенд с netem; скрипт из N=100 запросов к серверу двумя режимами; вариации loss 0–3%, jitter 0–20 мс; посчитать суммарное время и число рукопожатий в дампе.
- **Инструменты.** curl (многократный), python-клиент или wrk, tcpdump, netem.
- **Что сдать.** Отчёт: таблица режимов, объяснение «стоимости» одного handshake (в RTT), выводы для разработчика API.

Тема 5. MTU, фрагментация и jumbo frames (2 звезды)

- **Цель.** Показать влияние MTU на throughput и поведение PMTUD; разобрать классическую «чёрную дыру» при туннелировании.
- **Что делать.** netns-стенд с настраиваемым MTU на промежуточном линке; замеры iperf3 при MTU 1500/1400/1200 и с jumbo 9000 (veth позволяет); эксперимент с DF=1 и закрытым ICMP fragmentation needed — воспроизвести «зависание» и доказать причину дампом.
- **Инструменты.** ip link set mtu, iperf3, tcpdump, ping -M do.
- **Что сдать.** Отчёт: таблица «MTU → throughput», воспроизведённая PMTUD- дыра с дампом, рекомендации по MSS clamp.

Тема 6. QoS-очереди: HTB против fq_codel в борьбе с bufferbloat (3 звезды)

- **Цель.** Сравнить поведение классового шейпера HTB и fq_codel по latency под нагрузкой (bufferbloat-эффект).
- **Что делать.** Канал с узким горлом (htb rate); фоновая закачка; параллельно замерять RTT (ping) и интерактивный поток; три конфигурации: без управления, htb, fq_codel; построить графики RTT во времени.
- **Инструменты.** tc (htb, fq_codel), iperf3, ping с интервалом 0.2 с, gnuplot.
- **Что сдать.** Отчёт с графиками «latency under load», объяснением queuing- механики и рекомендацией, когда что применять.

9.2. Смешанные темы: инструмент + небольшая разработка

Тема 7. XDP против nftables: скорость отбрасывания пакетов (3 звезды)

- **Цель.** Сравнить pps и CPU при отбрасывании потока UDP на XDP и в nftables.
- **Что делать.** VM с двумя NIC/veth; генератор потока (iperf3 в UDP-режиме или pktgen-скрипт); три конфигурации: без фильтра, nftables drop, XDP drop (программа из недели 15); мерить pps на получателе и загрузку CPU (mpstat); снять счётчики run_cnt программы.
- **Что сдать.** Отчёт: методика генерации (чтобы честно), таблица pps/CPU, объяснение, откуда берётся разница (sk_buff!), границы применимости.

Тема 8. eBPF-трассировка собственного сервера (3 звезды)

- **Цель.** Построить профиль сетевых задержек hw-13-сервера с помощью eBPF: где «сидит» время — в приложении, в ядре или в очередях.
- **Что делать.** bpftrace-скрипты: latency connect(), latency send/recv, счётчик qdisc-drops; нагрузка wrk; собрать распределения (гистограммы bpftrace print @latency).
- **Что сдать.** Отчёт: скрипты, гистограммы, интерпретация «где узко» и что изменить (буферы, epoll, keep-alive), комментарии по верификатору/лимитам.

Тема 9. Сравнение DNS-резолверов (2 звезды)

- **Цель.** Сравнить systemd-resolved, dnsmasq и unbound по латентности и поведению кэша, включая отказ upstream.
- **Что делать.** Стенд: локальный «авторитетный» сервер (или публичный); замеры: холодный запрос, тёплый запрос (TTL), burst 1000 запросов, отказ upstream (поведение и таймауты); tcpdump для подтверждения числа реальных внешних запросов.
- **Что сдать.** Отчёт: таблица латентностей, анализ стратегий кэширования и рекомендация для корпоративного и для клиентского сценария.

9.3. Теоретические темы: обзор протокола «простыми словами»

Тема 10. QUIC и HTTP/3 простыми словами (2 звезды)

- **Задача.** Объяснить аудитории курса: зачем QUIC строит надёжность поверх UDP, что такое 0-RTT, как мультиплексирование потоков решает head-of-line blocking и чем HTTP/3 отличается от HTTP/2 на практике.
- **План.** Обзор RFC 9000 (архитектурная часть), сравнительная таблица TCP+TLS vs QUIC, разбор одного измерения из сети (например, curl --http3) как иллюстрации; обязательный раздел «почему это важно бэкенд-разработчику».
- **Что сдать.** Отчёт-обзор 4–6 страниц + выступление с 2–3 диаграммами.

Тема 11. STP/RSTP простыми словами (1 звезда)

- **Задача.** Объяснить проблему петель L2, алгоритм выбора root bridge, роли портов и то, что именно ускорил RSTP; связать с broadcast storm из недели 4.
- **План.** Схемы-диаграммы (минимум три: кольцо, дерево, отказ линка), демонстрация в Packet Tracer с таймингами сходимости STP vs RSTP.
- **Что сдать.** Отчёт-обзор + видеоскринкаст или скриншоты PT-эксперимента.

Тема 12. BGP: как маршрутизируется интернет (2 звезды)

- **Задача.** Объяснить модель автономных систем, что такое eBGP/iBGP, как выбирается маршрут (атрибуты в упрощении), почему возможны route leaks и hijacks; показать рост таблицы маршрутов.
- **План.** Обзор RFC 4271 (концептуально), схема «интернет из AS», разбор одного публичного инцидента (по публичным пост-мортемам), мини-лаба: BGP в Packet Tracer между двумя AS (опционально FRRouting в netns).
- **Что сдать.** Отчёт-обзор с диаграммами и разбором инцидента.

Тема 13. TLS 1.3 handshake простыми словами (2 звезды)

- **Задача.** Объяснить этапы рукопожатия TLS 1.3, что такое 1-RTT и 0-RTT, почему удалены старые наборы шифров и что остаётся видимым для наблюдателя.
- **План.** Разбор вывода openssl s_client -brief (с недели 13), сравнение с TLS 1.2 (число round-trip), раздел «что видит DPI, а что нет»; опционально — снятие handshake в Wireshark с расшифровкой ключей (SSLKEYLOGFILE).
- **Что сдать.** Отчёт-обзор с поэтапной диаграммой handshake.

Тема 14. NAT traversal: почему P2P за NAT так тяжёл (2 звезды)

- **Задача.** Объяснить, как NAT ломает прямые соединения, что делают STUN, TURN и ICE, и почему WebRTC без релеев не всегда возможен.

- **План.** Классификация NAT по поведению, схемы «кто кого вызывает», мини-эксперимент в netns: две «сети за NAT» и попытка прямого соединения; анализ, какие типы NAT проходимы.
- **Что сдать.** Отчёт-обзор со схемами и результатами эксперимента.

9.4. Общие требования к отчёту и выступлению

- Отчёт: 4–6 страниц (теоретические темы — до 8), структура: введение с целью; методика (что и как измеряли/изучали — чтобы читатель смог воспроизвести); результаты (таблицы, графики, выдержки конфигов); обсуждение (что это значит); выводы (3–5 пунктов); источники (RFC, документация, статьи — обязательно с указанием, что именно из них взято).
- Все замеры — минимум три прогона, в отчёт идут медианы; оборудование и версии ПО указаны обязательно («на моей машине было быстрее» не является результатом).
- Выступление: 7 минут, не «прочитанный отчёт», а история: проблема → подход → главный результат → неожиданная находка. Слайды опциональны; живая демонстрация приветствуется.
- Парная работа допускается только для тем со звёздами не ниже двух и требует от каждого участника отдельного раздела «мой вклад» в отчёте.

КАК ВЕСТИ СЕМИНАРЫ И ТИПОВЫЕ ГРАБЛИ

10. Методические заметки преподавателю

Эта глава — накопленный опыт ведения курса: приёмы, которые оживляют семинар, и типовые ошибки, которые студенты совершают из года в год. Ассистенту, ведущему курс впервые, достаточно прочитать эту главу и соответствующий раздел семинара накануне — методичка рассчитана на это.

10.1. Приёмы активного ведения семинара

- **Вопрос-пауза.** После каждого ключевого утверждения — 20–30 секунд тишины и открытый вопрос («почему здесь так?»). Студенты охотно отвечают на вопросы, на которые им разрешено молчать первые полминуты.
- **Испорченный конфиг.** Перед семинаром сломайте на демонстрационной машине заранее подготовленную конфигурацию (маршрут, ACL, MTU); разбор «инцидента» силами аудитории — лучшая тренировка диагностики. Скрипт поломки готовится заранее, откат — одной командой.
- **Парный разбор дампа.** tcpdump-вывод разбирают пары: один читает поля, второй «озвучивает» смысл; затем смена ролей. Работает лучше, чем разбор с места преподавателя.
- **Микродоклады 3–5 минут.** Два-три студента в неделю готовят короткий материал к семинару (например, «что такое SYN cookies», «куда делся CSMA/CD»); засчитывается в активность. К неделе 16 это превращается в готовность к защите исследования.
- **Живая ведомость.** Откройте в конце семинара страницу CI организации: студенты, чьи hw уже зелёные, показывают решение на экране — это снимает страх «сложного» кода и дисциплинирует по срокам.

10.2. Типовые заблуждения по блокам

Блок	Заблуждение	Как разобрать
I	«OSI — то, что реально работает, TCP/IP — теория»	Перевернуть: показать, что стек TCP/IP — реализация, а OSI — язык описания; на дампе найти, где «уровни» физически живут.
I	«MAC и IP — два имени одного»	Демонстрация traceroute: MAC меняется на каждом хопе, IP нет; рисуем два «стека адресов» на схеме пути.
II	«Коммутатор роутует»	Схема доменов: коммутатор режет домен коллизий, но не широковещательный; показать ping broadcast на стенде.
II	«ARP — исторический рудимент»	Показать живой ARP-обмен и атаки на него: без ARP не работает ни один Ethernet-линк.
III	«ping измеряет скорость»	Отделить RTT от пропускной способности; iperf3 против ping на одном стенде.
III	«TCP гарантированно доставит — значит, приложение ничего не делать не должно»	Разбор TIME_WAIT, RST, полузакрытия; сценарий «сервер умер, клиент не заметил».
IV	«VLAN = безопасность»	Обсуждение VLAN-hopping и роли ACL: сегментация — архитектурный приём, безопасность — совокупность мер.
IV	«QoS ускорит мою сеть»	Показать на стенде: QoS перераспределяет, а не добавляет полосу; выигрыш есть только при разнородном трафике.
V	«reverse-proxy нужен, чтобы скрывать IP»	Разобрать полный список функций (TLS, кэш, лимиты, отказоустойчивость) на конфиге из лабы.
VI	«eBPF — для системщиков, нам не надо»	Показать bpftrace-профиль «медленного» приложения: разработчик бэкенда диагностирует свою проблему за минуты.

Частые заблуждения студентов и способ их разбора

10.3. Типовые ошибки в домашних заданиях

Задача	Частая ошибка	Реакция на ревью
hw-1	Чексумма считается по 16-битным словам в host-endianness	Дать декомпозицию: сначала собрать байты в big-endian слова, потом суммировать; ссылка на RFC 1071, раздел 2.
hw-2	Парсинг «прыжками» scanf по форматным строкам	Показать getline+разбор по разделителям; тест на мусорных строках сразу ломает scanf-решение.
hw-3	Забыт big-endian в EtherType/TCI; payload копируется вместо указателя внутрь	Обе ошибки видны в round-trip тесте; прогнать с ASan — копирование найдёт valgrind-джоба.
hw-4	Системное время вместо инъектированного tick_now	Указать на требование детерминизма; показать, что скрытые тесты двигают время вручную.
hw-5	Чтение полей структурой поверх буфера (alignment/padding)	Мини-лекция про alignment: вывести offsetof для IP-структуры и сравнить с RFC-картой.

Задача	Частая ошибка	Реакция на ревью
hw-6	Наивный перебор правил вместо префиксного дерева	Бенчмарк скрытых тестов не пройдёт; обсудить bit-trie, предложить DIR-24-8 как бонус.
hw-7	Бесконечное ожидание recv без таймаута	Показать, что транспорт обязан возвращать 0 по timeout; ретрансмиссия — единственный путь прогресса.
hw-8	FSM «лапшей» if и без обработки RST	Требовать таблицу переходов; RST-тест из любого состояния обязателен.
hw-9	Кумулятивный ACK посчитан по кадрам, а не по байтам	Самая частая концептуальная ошибка; дать счётный пример на доске: кадры 100+200 байт, ACK после второго.
hw-13	Content-Length не выставлен; чтение запроса одним recv	TCP — поток: запрос приходит кусками; показать тест с разбиением заголовков по пакетам.
hw-15	Верификатор ругается на чтение без границы (void* + data_end)	Показать паттерн if ((void*)(ip+1) > data_end); ссылка на Documentation/bpf.

Что ломают студенты и как об этом говорить на ревью

10.4. Инфраструктурные грабли

- **FetchContent без интернета** (аудитории без сети): заранее скачайте googletest-архив и раздайте через локальный зеркальный путь FETCHCONTENT_SOURCE_DIR_GOOGLETEST; инструкция — в README шаблона (A.2).
- **Разные версии gcc/clang** ловят разные ошибки: студенты должны проверять локально обе, иначе «у меня работало» заканчивается красной матрицей CI.
- **WSL2**: netns работает, но с нюансами сетевых интерфейсов; tcpdump требует прав; Packet Tracer — только Windows-часть. Рекомендация: лабы WSL-студенты делают на серверах кафедры.
- **Права на tcpdump**: CAP_NET_RAW не выдаётся в контейнерах по умолчанию; на серверах кафедры настроен sudo без пароля для конкретных утилит.
- **Rate-limit GitHub Actions** в пиковые дедлайны: очередь матриц gcc×clang ×Debug/Release ×4 на 200 студентов может занять минуты; если organization ограничена — сократить матрицу до gcc/Debug для повторных push.

10.5. Регулировка темпа курса

Ситуация	Регулировка
Блок III слишком затянулся	Сократить практику семинара 9 за счёт переноса экспериментов с netem в самостоятельную работу; hw-9 не сокращать — он ядро курса.
Группа сильная, всё успевает	Добавить в семинары 6 и 12 мини-исследования из тем 6 и 7 как семинарные прогоны; бонусные части объявить обязательными для претендующих на автомат.
Мало студентов выбрали темы исследований к неделе 12	Провести «ярмарку тем» в начале семинара 12: преподаватель представляет три прикладные темы с готовым стендом из лаб.

Ситуация	Регулировка
CI-инфраструктура «съедает» неделю 1	Сдвинуть дедлайн hw-1 на 10 дней, hw-2 — совместить с неделей 3; тема недели 2 инвариантна.
Нет доступа к Packet Tracer у части студентов	Дать резервную лабу в netns: VLAN- подобную изоляцию собрать на namespace с бриджами и vlan-фильтрами; критерии те же.

Что менять, если группа идёт быстрее/медленнее плана

ПОЛНЫЕ ЛИСТИНГИ ЗАДАЧИ hw-3 · КОПИРУЙТЕ КАК ШАБЛОН

Приложение A. Эталонный boilerplate с автотестами

Это приложение — полный образец инфраструктуры любого «программистского» задания курса на примере hw-3 (парсер Ethernet-кадра). Задачи hw-2, hw-5…hw-9, hw-13 отличаются только предметной частью: заголовком интерфейса, тестами и списком проверок. Преподаватель готовит репозиторий-шаблон по этому приложению один раз; студент копирует структуру, не изменяя сборку и CI. Раздел A.6 объясняет механику скрытых тестов.

A.1. Структура репозитория задания

Дерево репозитория hw-ethernet

```
hw-ethernet/
├── CMakeLists.txt          # сборка и подключение тестов
├── README.md               # задание, критерии, как собрать/сдать
├── include/
│   └── ethernet.h          # интерфейс (менять запрещено)
├── src/
│   └── ethernet.c          # ЗДЕСЬ пишет студент
├── tests/
│   └── test_ethernet.cpp   # публичные тесты (GoogleTest)
└── .github/
    └── workflows/
        ├── ci.yml          # публичные тесты: gcc+clang, Debug/Release, ASan
        └── hidden.yml      # скрытые тесты преподавателя (репо faculty/*)
```

Правило неизменности интерфейса — центральное: функция tests/ и hidden-репозиторий собираются против include/ethernet.h как есть. Любое изменение интерфейса ломает проверку и расценивается как непринятая сдача. Если студенту нужен внутренний хелпер — он кладётся в src/ или в отдельный приватный заголовок src/*.h.

A.2. CMakeLists.txt (полный листинг)

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.16)
project(hw-ethernet LANGUAGES C CXX)
set(CMAKE_C_STANDARD 11)
set(CMAKE_C_STANDARD_REQUIRED ON)
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
if(NOT CMAKE_BUILD_TYPE)
  set(CMAKE_BUILD_TYPE Debug)
endif()
add_compile_options(-Wall -Wextra -Wpedantic)   # -Werror добавим в CI-джобе
include(FetchContent)
FetchContent_Declare(googletest
  URL      https://github.com/google/googletest/archive/refs/tags/v1.14.0.zip
  URL_HASH SHA256=1f357c27ca988c3f7c85b0b29560726855eae608db7dc2a9c7265afe849d294a
)
# Для офлайн-аудиторий: FetchContent поддерживает локальный зеркальный путь
# через FETCHCONTENT_SOURCE_DIR_GOOGLETEST (см. README курса).
FetchContent_MakeAvailable(googletest)
add_library(ethernet src/ethernet.c)
target_include_directories(ethernet PUBLIC include)
enable_testing()
add_executable(test_ethernet tests/test_ethernet.cpp)
target_link_libraries(test_ethernet PRIVATE ethernet gtest gtest_main)
include(GoogleTest)
gtest_discover_tests(test_ethernet)
```

A.3. Публичные тесты tests/test_ethernet.cpp (репрезентативный фрагмент)

tests/test_ethernet.cpp

```
#include <gtest/gtest.h>
#include "ethernet.h"
#include <cstring>
#include <string>
#include <vector>
// hex-строка -> байты;fixture-хелпер, общий для всех задач курса
static std::vector<uint8_t> hex_bytes(const std::string &s) {
  std::vector<uint8_t> out;
  for (size_t i = 0; i + 1 < s.size(); i += 2)
    out.push_back((uint8_t)std::stoul(s.substr(i, 2), nullptr, 16));
  return out;
}
TEST(Parse, ArpBroadcastPadded) {
  // ARP-запрос: dst ff:ff:ff:ff:ff:ff, src 00:1a:2b:3c:4d:5e, type 0x0806;
  // полезная нагрузка 28 байт + 18 байт padding = 46 (минимум IEEE 802.3).
  auto f = hex_bytes(
    "ffffffffffff001a2b3c4d5e0806"
    "00010800060400010001a2b3c4d5ec0a8010a000000000000c0a80101"
    "00000000000000000000000000000000000000");
  ASSERT_EQ(f.size(), 60u);
  eth_frame_t fr;
```

```
  ASSERT_EQ(eth_parse(f.data(), f.size(), &fr), 0);
  EXPECT_EQ(fr.ethertype, 0x0806);
  EXPECT_EQ(fr.vlan, -1);
  EXPECT_EQ(fr.payload_len, 46u);
  EXPECT_EQ(fr.payload[0], 0x00);          // HTYPE hi
  EXPECT_EQ(fr.payload[24], 0xc0);         // TPA первый байт (192.168.1.1)
}
TEST(Parse, VlanTag) {
  // IPv4 в VLAN 100, PCP=5: тип 0x8100, TCI = (5<<13)|100 = 0xA064
  auto f = hex_bytes(
    "0123456789ab" "fedcba987654" "8100" "a064" "0800" +
    std::string(92, '7'));                  // 46 байт нагрузки
  eth_frame_t fr;
  ASSERT_EQ(eth_parse(f.data(), f.size(), &fr), 0);
  EXPECT_EQ(fr.vlan, 100);
  EXPECT_EQ(fr.pcp, 5);
  EXPECT_EQ(fr.ethertype, 0x0800);
}
TEST(Parse, Errors) {
  std::vector<uint8_t> short_frame(10, 0x11);
  eth_frame_t fr;
  EXPECT_EQ(eth_parse(short_frame.data(), short_frame.size(), &fr), -1);
  // payload меньше 46 байт (14 + 30) — ошибка минимальной длины
  auto f = hex_bytes("0123456789ab" "fedcba987654" "0800" +
                     std::string(30, '7'));
  EXPECT_EQ(eth_parse(f.data(), f.size(), &fr), -1);
}
TEST(RoundTrip, SerializeParse) {
  eth_frame_t f{};
  uint8_t payload[50];
  for (int i = 0; i < 50; ++i) payload[i] = (uint8_t)i;
  memcpy(f.dst, "\x01\x02\x03\x04\x05\x06", 6);
  memcpy(f.src, "\x11\x12\x13\x14\x15\x16", 6);
  f.ethertype = 0x86DD; f.vlan = -1; f.pcp = 0; f.outer_vlan = -1;
  f.payload = payload; f.payload_len = sizeof payload;
  std::vector<uint8_t> buf(200);
  int n = eth_serialize(&f, buf.data(), buf.size());
  ASSERT_EQ(n, 14 + 50);
  eth_frame_t back;
  ASSERT_EQ(eth_parse(buf.data(), (size_t)n, &back), 0);
  EXPECT_EQ(back.ethertype, 0x86DD);
  EXPECT_EQ(0, memcmp(back.payload, payload, sizeof payload));
}
```

Обратите внимание на два стилистических приёма, обязательных для задач курса: фикстуры задаются hex-строками (их легко сверить с tcpdump -xx), а round-trip тест проверяет парность parse/serialize — типовой источник ошибок на один бит. Полный файл тестов hw-3 (24 теста) входит в репозиторий-шаблон.

A.4. GitHub Actions: .github/workflows/ci.yml (полный листинг)

.github/workflows/ci.yml

```
name: ci
on: [push, pull_request]
jobs:
  build:
    name: ${{ matrix.compiler }} / ${{ matrix.build }}
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        compiler: [gcc, clang]
        build: [Release, Debug]
    env:
      CC: ${{ matrix.compiler }}
      CXX: ${{ matrix.compiler == 'gcc' && 'g++' || 'clang++' }}
    steps:
      - uses: actions/checkout@v4
      - name: Configure
        run: cmake -B build -DCMAKE_BUILD_TYPE=${{ matrix.build }}
                   -DCMAKE_C_FLAGS="-Werror"
      - name: Build
        run: cmake --build build -j
      - name: Tests
        run: ctest --test-dir build --output-on-failure
  sanitizers:
    name: ASan + UBSan
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Configure
        run: cmake -B build -DCMAKE_BUILD_TYPE=Debug
             -DCMAKE_C_FLAGS="-fsanitize=address,undefined -fno-omit-frame-pointer -g"
             -DCMAKE_CXX_FLAGS="-fsanitize=address,undefined -fno-omit-frame-pointer -g"
             -DCMAKE_EXE_LINKER_FLAGS="-fsanitize=address,undefined"
      - run: cmake --build build -j
      - run: ctest --test-dir build --output-on-failure
```

Матрица gcc/clang × Release/Debug ловит неопределённое поведение и компиляторо-зависимые ошибки; job sanitizers — утечки и out-of-bounds. Студент видит красную галочку с логом конкретного теста — ревью начинается только после зелёного CI. Для задач с потоками добавляется флаг -pthread в целевой библиотеке (hw-13).

A.5. Шаблон README.md

README.md (шаблон задания)

```
# hw-3: парсер Ethernet-кадра
Дедлайн: начало семинара 4. Поздняя сдача: +72 часа, -25%.
Оценка: публичные тесты 40% · скрытые тесты 40% · качество кода 20%.
Бонус: jumbo-кадры до 9000 байт (+3 б.).
## Что сделать
Реализовать eth_parse/eth_serialize по include/ethernet.h.
Интерфейс менять запрещено. Код писать только в src/.
```

```
## Как проверить локально
    cmake -B build -DCMAKE_BUILD_TYPE=Debug
    cmake --build build -j
    ctest --test-dir build --output-on-failure
## Что проверяют тесты
- канонические кадры (ARP/IPv4), padding до 46 байт;
- VLAN 802.1Q (PCP/VID), QinQ — в скрытых;
- ошибки: короткий кадр, payload вне [46, 1500], NULL;
- round-trip parse(serialize(x)) == x.
## Сдавать
push в main; зелёный CI обязателен; ссылка на PR — в ведомость.
```

A.6. Скрытые тесты: механика для преподавателя

Скрытые тесты живут в **приватном** репозитории организации (faculty/hidden-ethernet и т.п.) и содержат: дополнительные тесты на граничные случаи, анти-чит проверки (подмена возвращаемых констант, проверка отсутствия memcpy-обёрток там, где запрещено) и эталонные решения. У студента в репозитории лежит workflow hidden.yml, который подтягивает приватный репозиторий по токену с правом read:packages (finer-grained PAT, срок 1 год, хранится в secrets.HIDDEN_TESTS_TOKEN) и прогоняет скрытый набор. Токен выдаётся через GitHub Classroom-механизм организации.

.github/workflows/hidden.yml (в репозитории студента)

```yaml
name: hidden
on: [push, workflow_dispatch]
jobs:
  hidden-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Загрузить скрытый набор (приватное репо курса)
        uses: actions/checkout@v4
        with:
          repository: faculty/hidden-ethernet
          ref: main
          token: ${{ secrets.HIDDEN_TESTS_TOKEN }}
          path: hidden
      - name: Сборка + скрытые тесты
        run: |
          cmake -B build -DCMAKE_BUILD_TYPE=Debug
          cmake --build build -j
          ctest --test-dir build --output-on-failure
        env:
          # тесты подключаются CMake-опцией из hidden/CMakeLists.txt
          HIDDEN_TESTS_DIR: ${{ github.workspace }}/hidden
```

Альтернатива без токенов (для аудиторий без GitHub-организации): преподаватель клонирует студенческие репозитории скриптом (gh repo clone org/hw-N-student), накатывает скрытые тесты локально и прогоняет ctest; результат попадает в ведомость CSV. Механика та же, CI не обязателен. В обоих случаях студент видит только факт прохождения и имя упавшего теста, но не сам тест до публикации набора после дедлайна.

A.7. Чек-лист преподавателя: добавить новую задачу за 30 минут

1. Скопировать репозиторий hw-ethernet как шаблон (Use this template).
2. Заменить include/*.h, src/*.c (заглушка: функции возвращают -1), README, название проекта в CMakeLists.txt.
3. Написать публичные тесты (8–12 штук: happy path, границы, ошибки, round-trip если есть сериализация).
4. Создать приватный hidden-репозиторий (10–20 тестов + анти-чит) и прицепить hidden.yml в шаблон.
5. Создать задание в GitHub Classroom, выдать приглашение, проверить сборку на «нулевом» студенческом репозитории.
6. Добавить задачу в карту курса (глава 2) и в сводную таблицу приложения Б.

СВОДНЫЕ ТЗ: ФОРМАТЫ, ПРОВЕРКИ, ДЕДЛАЙНЫ, БАЛЛЫ

Приложение Б. Спецификации домашних заданий

Интерфейсы задач приведены в главах 3–8 при соответствующих семинарах; здесь — сводная таблица и единственная задача, чей формат требует полной спецификации (hw-2), плюс общие требования к коду, одинаковые для всех заданий.

ID	Задача	Сдать к	Баллы	Бонус
hw-1	Разминка CI: parse_mac, format_mac, ipv4_checksum	неделя 2	4	+2
hw-2	netshot: отчёт по фикстурам (Б.1)	неделя 3	6	–
hw-3	Ethernet-парсер/сериализатор (эталон)	неделя 4	6	+3
hw-4	ARP-кэш с инъекцией времени	неделя 5	6	+3
hw-5	IPv4-парсер + чексумма (+ сборщик фрагментов)	неделя 6	6	+3
hw-6	LPM-таблица на префиксном дереве	неделя 7	6	+3
hw-7	Мини-TFTP с инъекцией транспорта	неделя 8	6	+3
hw-8	TCP-парсер + FSM-симулятор	неделя 9	6	+2
hw-9	Мини-TCP поверх UDP (ARQ, окно, RTO)	неделя 11	10	+5
hw-13	Мини-HTTP-сервер на сокетах	неделя 14	8	+3
hw-15	XDP-политика из LPM-карты (демо на VM)	неделя 16	8	+3

Сводная таблица программных заданий

Б.1. Полная спецификация hw-2: формат входа и вывода

Вход — текстовый файл фиксированного формата (LF или CRLF, кодировка ASCII). Строки других форматов игнорируются молча. Пустые строки и строки, начинающиеся с «#», игнорируются. Форматы строк:

Формат входного файла hw-2

```
iface <имя> <up|down> [ip/<plen> ...]   # ноль и более адресов
route (default|<cidr>) via <ip> dev <имя>   # или без via: connected
route <cidr> dev <имя>
listen (tcp|udp) <addr>:<port>
# Пример:
iface lo up 127.0.0.1/8
iface eth0 up 192.168.1.10/24
iface wlan0 down
route default via 192.168.1.1 dev eth0
route 10.0.0.0/8 dev tun0
listen tcp 0.0.0.0:22
listen udp 127.0.0.53:53
```

Канонический вывод netshot_report для примера выше

```
interfaces=2
addr lo 127.0.0.1/8
addr eth0 192.168.1.10/24
routes=2
listen_tcp=1
listen_udp=1
```

- Строка interfaces — число интерфейсов с состоянием up (wlan0 не считается).
- Строки addr — по одному адресу на строку, в порядке появления во входе; интерфейсы down адресов не имеют и в addr не попадают.
- routes — всего строк route (включая default).
- listen_tcp / listen_udp — число слушающих сокетов по протоколам.
- Вывод заканчивается переводом строки; ничего лишнего в stdout быть не должно (диагностика — только в stderr).
- Скрытые тесты: 10 000 строк (потоковая обработка), мусорные строки, строки без пробелов, дублирующиеся адреса (выводить как есть), порт без адреса (`listen tcp :80` — считается).

Б.2. Общие требования к коду всех заданий

- Сборка без предупреждений (-Werror в CI), стандарт C11 (серверные задачи — допускается C++17 для тестов).
- Отсутствие утечек и UB: ASan/UBSan-джоба зелёная; valgrind — по желанию.
- Ошибки возвращаются кодами (0/-1), диагностика — в stderr с префиксом имени функции; никаких exit() из библиотек.
- Запрещены глобальные изменяемые состояния; всё состояние — в структурах из интерфейса.

- Чтение полей сетевых заголовков — строго по сдвигам и big-endian (ntohs/htonl или явные сдвиги);reinterpret_cast на невыровненные данные запрещён.
- README каждого задания: как собрать, что сделано, что не успело, ответы на вопросы «для размышления» из постановки (если есть).

РАБОЧИЙ СПРАВОЧНИК КОМАНД КУРСА

Приложение B. Шпаргалка по утилитам

Команда	Что делает
`ip -br link; ip -br addr`	Кратко: интерфейсы, состояние, адреса
`ip route show; ip route get`	Таблица маршрутов; «сухой прогон» решения о маршруте
`ip neigh show; ip neigh flush all`	ARP/NDP-кэш: состояния REACHABLE/STALE; очистка
`ip link set dev eth0 mtu 1400`	Изменить MTU (эксперименты с фрагментацией)
`ping -c N -i 0.2 -M do -s`	N эхо-запросов, интервал 0.2 с, запрет фрагментации
`ss -tlnp / ss -ulnp / ss -tan`	Слушающие TCP/UDP; все TCP-соединения с состояниями
`nc -l 8080; nc host 8080`	TCP-сервер и клиент; -u — UDP-режим
`printf "GET / HTTP/1.1\r\nHost: h\r\n\r\n" | nc h 80`	Ручной HTTP-запрос
`mtr -rwbc 100`	Отчёт о пути: 100 пакетов, статистика потерь
`traceroute -n`	Путь без резолвинга имён
`dig +trace ; dig +noall +answer`	Полный обход DNS; краткий ответ с TTL
`curl -v ; curl -sv --http2`	Подробный обмен; форс HTTP/2
`ethtool eth0`	Скорость, дуплекс, состояние линка

Диагностика и конфигурация (ip, ping, ss, nc, mtr, dig, curl)

Команда/фильтр	Смысл
`tcpdump -i any -n -c 100`	Все интерфейсы, без резолвинга, 100 пакетов
`tcpdump -eni any icmp`	С MAC-уровнем (-e), только ICMP
`tcpdump -n tcp port 80 and host X`	HTTP-обмен с узлом X
`tcpdump -n "net 10.0.0.0/8 and not host Y"`	Подсеть без узла Y
`tcpdump -w file.pcap / -r file.pcap`	Запись/чтение дампа
`tcpdump -A / -X`	ASCII-дамп / hex+ASCII полезной нагрузки
`tcpdump -n arp and ether broadcast`	Широковещательные ARP-запросы
`tcpdump -n "icmp[icmptype] = icmp-unreach"`	Только ICMP unreachable

tcpdump: фильтры и флаги

Команда	Смысл
`ip netns add ns1; ip netns exec ns1`	Создать namespace; выполнить в нём
`ip link add v0 type veth peer name v1`	Пара виртуальных интерфейсов
`ip link set v0 netns ns1`	Перенести интерфейс в namespace
`tc qdisc add dev eth0 root netem delay 30ms loss 2%`	Эмуляция задержки и потерь
`tc qdisc add dev eth0 root handle 1: htb default 30`	Классовый шейпер HTB (см. неделю 12)
`tc -s qdisc show dev eth0`	Статистика очередей (сбросы, длины)
`nft add table ip nat; nft add chain ... masquerade`	NAT-шлюз (неделя 11); man nft
`conntrack -L`	Живая таблица трансляций NAT
`haproxy -c -f haproxy.cfg`	Проверить конфиг без запуска
`systemctl reload haproxy`	Перечитать конфиг без разрыва соединений
`bpftool prog show; bpftool map dump name`	Загруженные eBPF-программы; содержимое карты
`ip link set dev eth0 xdp obj prog.o sec xdp`	Загрузить XDP-программу; off — снять

netns, tc/netem, nftables, HAProxy, bpftool

КНИГИ, RFC, ОНЛАЙН-ИСТОЧНИКИ

Приложение Г. Литература и ресурсы

Г.1. Книги

- **А. Таненбаум, Д. Уэзеролл. «Компьютерные сети» (6-е изд.)** — базовый учебник курса; главы 1, 3, 5, 6, 7 обязательны в объёме разделов, указанных в недельных самостоятельных работах.
- **Дж. Куроуз, К. Росс. «Компьютерные сети. Нисходящий подход»** — альтернативное изложение «сверху вниз»; полезно тем, кому проще начинать с прикладного уровня.
- **В. Олифер, Н. Олифер. «Компьютерные сети. Принципы, технологии, протоколы»** — русскоязычный справочник с большим числом схем; удобен для повторения.
- **У. Ричард Стивенс. «UNIX: разработка сетевых приложений»** — настольная книга для hw-7, hw-9, hw-13 (сокеты, epoll, серверные архитектуры).
- **У. Ричард Стивенс. «TCP/IP Illustrated, Vol. 1»** — протоколы с картинками и дампами; идеальное дополнение к блоку III.
- **Д. Эванджелос и др. (Borkmann et al.) «Linux Networking» / документация Documentation/bpf ядра Linux** — по eBPF/XDP недели 15; плюс книга «Learning eBPF» (L. Pickering) как введение.

Г.2. Ключевые RFC (по темам курса)

- RFC 791 — IPv4; RFC 8200 — IPv6; RFC 792 — ICMP; RFC 1071 — алгоритм чексуммы.
- RFC 826 — ARP; RFC 894 — IP поверх Ethernet; IEEE 802.1Q — VLAN (стандарт).
- RFC 9293 — TCP (актуальная редакция; ранее RFC 793); RFC 5681 — контроль перегрузки TCP; RFC 6298 — RTO.
- RFC 768 — UDP; RFC 1350 — TFTP; RFC 2131 — DHCP.
- RFC 1034/1035 — DNS; RFC 9110/9112 — HTTP (семантика/HTTP 1.1); RFC 9113 — HTTP/2; RFC 8446 — TLS 1.3; RFC 9000 — QUIC.
- RFC 1918 — частные адреса; RFC 4271 — BGP-4.

Г.3. Онлайн-ресурсы

- man-страницы разделов 7 (протоколы), 8 (администрирование): man 7 arp, man 7 tcp, man 8 ip — первоисточник, всегда соответствующий вашей системе.
- Documentation/networking и Documentation/bpf исходников ядра Linux — авторитетное описание NAPI, sk_buff, XDP.
- Wireshark Sample Captures (wiki.wireshark.org/SampleCaptures) — готовые дампы практически для всех протоколов курса; полезны для самостоятельного разбора.
- Официальная документация HAProxy (cbonte.github.io/haproxy-dconv) и nginx (nginx.org/ru/docs) — по блоку V; учебный конфиг — неделя 14.
- bpfilter, Cilium и Cloudflare-блоги по XDP — по неделе 15; статьи katran (github.com/facebookincubator/katran) — промышленный пример из семинара.
- Репозиторий курса (задачи, скрытые тесты после защит, этот файл методички) — ссылка выдаётся преподавателем в начале семестра.

ПОЛНЫЕ ЛИСТИНГИ ДЛЯ СЕМИНАРОВ И САМОСТОЯТЕЛЬНОЙ РАБОТЫ

Приложение Д. Демонстрационные программы

Здесь собраны полные листинги программ, на которые ссылаются семинары. Все они компилируются без внешних зависимостей (кроме отмеченного libbpf) и рассчитаны на копирование в терминал/редактор прямо во время семинара. Репозиторий курса содержит те же файлы в каталоге demos/ с Makefile.

Д.1. UDP-эхо-клиент (семинар 7, дополнение к демо 14)

demos/udp_echo_client.c

```
#include <arpa/inet.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
/* ./udp_client <host> <port> <message>
 * Шлёт дейтаграмму, ждёт эхо до 2 секунд, печатает ответ. */
int main(int argc, char **argv) {
    if (argc != 4) { fprintf(stderr, "usage: %s host port msg\n", argv[0]); return 2; }
    int fd = socket(AF_INET, SOCK_DGRAM, 0);
    struct sockaddr_in a = {0};
    a.sin_family = AF_INET;
    a.sin_port = htons((uint16_t)atoi(argv[2]));
    if (inet_pton(AF_INET, argv[1], &a.sin_addr) != 1) {
        fprintf(stderr, "bad ip\n"); return 2;      /* без DNS — честно */
    }
    struct timeval tv = { .tv_sec = 2, .tv_usec = 0 };
    setsockopt(fd, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof tv);
    sendto(fd, argv[3], strlen(argv[3]), 0, (struct sockaddr *)&a, sizeof a);
    char buf[2048];
    ssize_t n = recvfrom(fd, buf, sizeof buf, 0, NULL, NULL);
    if (n < 0) { perror("recvfrom (timeout?)"); return 1; }
    buf[n < (ssize_t)sizeof buf ? n : (ssize_t)sizeof buf - 1] = 0;
    printf("echo (%zd b): %s\n", n, buf);
    return 0;
}
```

Клиент намеренно лишён DNS: адрес только в виде IP — так студент не путает сети с резолвингом при диагностике. Эксперименты с ним: уронить сервер (получим ICMP unreachable и errno ECONNREFUSED при connect-версии), послать больше буфера (обрезка), проверить поведение при потере (timeout).

Д.2. bpftrace: сетевые однострочники (семинар 16, наблюдаемость)

demos/bpftrace_net.txt — готовые к запуску команды

```
# Гистограмма длительности connect() по процессам
bpftrace -e 'tracepoint:syscalls:sys_enter_connect /comm != "bpftrace"/
  { @start[tid] = nsecs; }
tracepoint:syscalls:sys_exit_connect /@start[tid]/
  { @us = hist((nsecs - @start[tid]) / 1000); delete(@start[tid]); }'
# TCP: кто и куда открывает соединения (адреса в сетевом порядке)
bpftrace -e 'tracepoint:sock:inet_sock_set_state
  /args->newstate == 1 || args->newstate == 2/
  { printf("%s %s %s:%d -> %s:%d\n", comm, args->protocol == 6 ? "tcp":"other",
           ntop(args->saddr), args->sport, ntop(args->daddr), args->dport); }'
# Размер recv-вызовов: есть ли «мелкие» чтения (микро-пакеты)
bpftrace -e 'tracepoint:syscalls:sys_enter_recvfrom
  /args->count > 0/ { @bytes = hist(args->count); }'
# Очередь qdisc: дропы на egress (см. лабу QoS)
```

```
bpftrace -e 'kprobe:dev_queue_xmit { @[kstack(2)] = count(); }' | head
```

Однострочники выполняются без правок кода приложения — это главный аргумент в споре «наблюдаемость важнее логирования». На семинаре 16 мы запускаем первые два против сервера из hw-13 и читаем гистограммы: где и кто проводит время.

Д.3. Полный скрипт стенда netns (лабы недель 6, 9, 12, 14)

demos/nsmesh_full.sh — сборка, проверка, разбор

```
#!/bin/bash
# Стенд "left --- router --- right" с teardown и проверками.
set -euo pipefail
up() {
  ip netns add left; ip netns add right; ip netns add router
  ip link add v-l type veth peer name v-l-r
  ip link add v-r type veth peer name v-r-r
  ip link set v-l netns left; ip link set v-l-r netns router
  ip link set v-r netns right; ip link set v-r-r netns router
  ip -n left   addr add 10.0.1.2/24 dev v-l;    ip -n left   link set v-l up
  ip -n right  addr add 10.0.2.2/24 dev v-r;    ip -n right  link set v-r up
  ip -n router addr add 10.0.1.1/24 dev v-l-r;  ip -n router link set v-l-r up
  ip -n router addr add 10.0.2.1/24 dev v-r-r;  ip -n router link set v-r-r up
  ip -n router sysctl -qw net.ipv4.ip_forward=1
  ip -n left   route add default via 10.0.1.1
  ip -n right  route add default via 10.0.2.1
  check
}
check() {
  ip -n left ping -c1 -W1 10.0.2.2 || { echo "FAIL: ping через роутер"; exit 1; }
  echo "OK: сквозная связность left->right через router"
}
down() {
  for ns in left right router; do ip netns del $ns 2>/dev/null || true; done
  echo "Стенд разобран"
}
case "${1:-up}" in
  up) up ;; down) down ;; *) echo "usage: $0 [up|down]"; exit 2 ;;
esac
```

Форма ip -n NAME заменяет ip netns exec NAME ip ... — короче и безопаснее в скриптах. set -euo pipefail обязателен: без него молчаливая ошибка настройки проявится часом позже «загадочным» ping. Стенд используется лабами QoS и HAProxy-лабой (там добавляются два namespace-бэкенда и HAProxy в router).

Д.4. Userspace-загрузчик XDP со счётчиками (к hw-15)

demos/xdp_util.c (фрагмент: загрузка и чтение карт через libbpf)

```
#include <bpf/libbpf.h>
#include <net/if.h>
#include <stdio.h>
#include <unistd.h>
/* ./xdp_util eth0 xdp_policy.o   — прицепить программу и читать счётчики
 * В worker: bpf_map_lookup_elem + sleep(1) раз в секунду. */
int main(int argc, char **argv) {
    if (argc != 3) { fprintf(stderr, "usage: %s <ifname> <obj>\n", argv[0]); return 2; }
    int ifindex = if_nametoindex(argv[1]);
    struct bpf_object *obj = bpf_object__open_file(argv[2], NULL);
    if (libbpf_get_error(obj)) { perror("open"); return 1; }
    if (bpf_object__load(obj)) { perror("load"); return 1; }   /* верификатор тут */
    struct bpf_program *prog =
        bpf_object__find_program_by_name(obj, "xdp_policy");
    int prog_fd = bpf_program__fd(prog);
    /* SKB-режим работает на любом драйвере; native быстрее, но зависит от NIC */
    if (bpf_xdp_attach(ifindex, prog_fd, XDP_FLAGS_SKB_MODE, NULL)) {
        perror("attach"); return 1;
    }
    int map_fd = bpf_object__find_map_fd_by_name(obj, "stats");
    for (;;) {
        __u64 pass = 0, drop = 0;
        __u32 k_pass = 0, k_drop = 1;
        bpf_map_lookup_elem(map_fd, &k_pass, &pass);
        bpf_map_lookup_elem(map_fd, &k_drop, &drop);
        printf("pass=%llu drop=%llu\n",
               (unsigned long long)pass, (unsigned long long)drop);
        sleep(1);
    }
}
```

Фрагмент намеренно сокращён до минимума, который студент расширяет в hw-15: загрузка политики из LPM-карты, обработка SIGINT для корректного отсоединения программы, атомарная печать дельт. Ошибки верификатора появляются на этапе bpf_object__load — читать их нужно с bpf_verifier_log, что включено в шаблон Makefile задания.

ДЛЯ МИКРОКВИЗОВ И САМОПРОВЕРКИ

Приложение Е. Контрольные вопросы с эталонными ответами

Вопросы сгруппированы по блокам и годятся для микроквизов начала семинара (раздел 2.1) и самопроверки студента перед сдачей задания. Эталонные ответы — краткие эталоны, а не полные тексты: на семинаре достаточно услышать их ядро. Ответы приводятся сразу после вопроса — ассистенту рекомендуется копировать вопросы в отдельный лист без ответов.

E.1. Блоки I–II: модели, утилиты, канальный уровень

Вопрос 1. Почему TCP/IP остался из четырёх уровней, а OSI описывают семью?

Стек TCP/IP сложился исторически как работающее решение и группирует задачи уровней 5–6 OSI в прикладной уровень: сессии и представление (TLS, сериализация) обслуживаются самим приложением. OSI осталась справочной моделью — удобным языком, на котором инженеры называют части системы («проблема на L2»), но никто не реализует её уровни буквально.

Вопрос 2. Что покажет ip route get 10.0.0.5 и чем это отличается от ping?

ip route get выполняет «сухой» поиск маршрута: покажет интерфейс, next-hop и src-адрес, которые выбрал бы стек, но пакет не отправит. ping отправляет ICMP и измеряет RTT. Первое — про решение о пересылке, второе — про достижимость и время; диагностировать маршрут быстрее и безопаснее первым.

Вопрос 3. Зачем в Ethernet минимальный размер кадра 64 байта?

Наследие полудуплекса с CSMA/CD: передатчик обязан передавать не менее времени прохождения сигнала по максимальному сегменту, чтобы успеть услышать коллизию и повторить передачу. Минимум 64 байта (и максимум 1500) закрепили эту геометрию; в полном дуплексе коллизий нет, но лимиты сохранены для совместимости.

Вопрос 4. Кому и зачем нужен broadcast в ARP-запросе?

Запрос адресован всем узлам сегмента, потому что отправитель не знает, где живёт целевой IP: broadcast — единственный способ «спросить всех» на L2. Ответ уже unicast — получатель теперь знает MAC отправителя (записал его из запроса), и широковещать ответ не нужно.

Вопрос 5. Чем домен коллизий отличается от широковещательного домена?

Домен коллизий — сегмент, где передатчики мешают друг другу; его режет любой коммутатор (и исчезает вовсе в полнодуплексе). Широковещательный домен — область доставки broadcast-кадров; его режут только маршрутизатор или VLAN. Коммутатор из 24 портов создаёт 24 домена коллизий, но один широковещательный.

Вопрос 6. Что случится с сетью, если два коммутатора соединить двумя линками без STP?

Возникает петля: broadcast-кадр бесконечно циркулирует и размножается (broadcast storm), MAC-таблицы «мигают», сеть умирает за секунды. STP логически блокирует избыточный линк, оставляя физическое резервирование на случай аварии основного пути.

E.2. Блок III: IP, маршрутизация, UDP, TCP

Вопрос 7. Сколько хостов в сети 172.16.4.0/22 и какой broadcast?

/22 оставляет 10 бит на хост: 1024 адреса, 1022 хостовых. Broadcast — 172.16.7.255 (все биты хоста = 1); последняя хостовая — .7.254. Диапазон сети: 172.16.4.0 — 172.16.7.255.

Вопрос 8. Почему TTL уменьшается на каждом роутере и что будет при TTL=0?

TTL — страховка от вечных пакетов в петлях маршрутизации: каждый роутер уменьшает его на 1; при нуле пакет выбрасывается и отправителю шлётся ICMP time exceeded. Именно этот механизм, эксплуатируемый намеренно, даёт traceroute.

Вопрос 9. Что такое longest prefix match и почему таблица ищет именно так?

При поиске маршрута выбирается совпавший префикс максимальной длины: более специфичный маршрут точнее описывает получателя (host-маршрут /32 сильнее /8). Это позволяет «вытягивать» трафик отдельным узлам и сетям, не ломая общие правила; алгоритмически поиск реализуют префиксные деревья.

Вопрос 10. Почему UDP-дейтаграмма сохраняет границы сообщений, а TCP-поток нет?

UDP — обёртка над IP-дейтаграммой: один sendto — одна дейтаграмма, получатель читает её целиком (обрезая до буфера). TCP — поток байт: запись дробятся и склеиваются по усмотрению стека, границы между send() в общем случае не сохраняются — приложение само вводит фрейминг (длина, разделители).

Вопрос 11. Зачем handshake TCP из трёх шагов?

Стороны должны обменяться начальными seq и синхронизироваться в обе стороны; SYN и ACK сервера совмещены в SYN-ACK, поэтому минимум — три сообщения. Двух недостаточно из-за дубликатов старых SYN: без третьего шага сервер может открыть «мёртвое» полусоединение. Три шага также позволяют согласовать опции (MSS, window scale).

Вопрос 12. Что означает состояние CLOSE_WAIT на сервере и кто виноват?

CLOSE_WAIT значит: FIN от клиента получен, TCP подтвердил, но приложение ещё не закрыло сокет (не вызвало close). Виновато приложение: типовая утечка ресурсов. Тысячи CLOSE_WAIT — знак, что код не обрабатывает удалённую сторону соединения.

Вопрос 13. Как именно потери пакетов «убивают» throughput TCP?

Потеря трактуется как сигнал перегрузки: по таймауту cwnd режется до 1 MSS и заново стартует slow start, по трём дублирующим ACK — halving (fast recovery). Через AIMD средняя скорость обратно пропорциональна корню из частоты потерь, поэтому уже 2% потерь на длинном канале катастрофически снижают скорость — студенты видят это в демо 18.

Вопрос 14. Почему Nagle и delayed ACK вместе дают ~40 мс задержки?

Nagle ждёт ACK или полного сегмента, чтобы отправить мелкую запись; delayed ACK держит подтверждение до 40 мс, надеясь «поймать» с ним ответные данные. Оба ждут друг друга — интерактивное приложение зависает. Лечение: TCP_NODELAY, конкатенация записей на прикладном уровне, включение TCP_QUICKACK где нужно.

E.3. Блоки IV–VI: корпоративные сети, прокси, ядро

Вопрос 15. Зачем trunk-порту allowed vlan, если можно пропустить все?

Минимизация доверия и области аварии: транк между этажами не должен носить гостевой VLAN, а инженерная ошибка в одной VLAN не должна транслироваться везде. Список allowed — часть политики сегментации, как ACL для маршрутов.

Вопрос 16. Как NAT решает, какому внутреннему узлу адресован входящий пакет?

По таблице conntrack: исходящий SNAT-запись (внутренний IP:порт → внешний IP:порт) хранит трансляцию; обратный пакет сопоставляется по пятикортежу и адресату порта. Пакет без записи (неответ на исходящий запрос) отбрасывается или требует отдельного DNAT/port-forwarding.

Вопрос 17. Почему policer для TCP-трафика часто хуже shaper?

Policer мгновенно выбрасывает сверхурочные пакеты, а TCP трактует потерю как перегрузку: cwnd режется, скорость обрушивается и восстанавливается медленно (AIMD). Shaper лишь задерживает пакеты, сглаживая всплески; потери — только при переполнении буфера. Отсюда правило: полиция — на границе провайдера, шейпинг — у себя.

Вопрос 18. Что такое sticky-сессия и когда она вредит?

Механизм прикрепления клиента к бэкенду (cookie-вставка, хэш источника), нужный, когда состояние сессии живёт в памяти бэкенда. Вредит при отказе бэкенда (сессии теряются, клиенты перепригают), при неравномерном клиентском трафике (перекос) и при масштабировании. Правильное решение — выносить состояние в общий стор и делать бэкенды взаимозаменяемыми.

Вопрос 19. Какие действия может вернуть XDP-программа и где она исполняется?

XDP_PASS (продолжить сетевой стек), XDP_DROP (отбросить), XDP_TX (отправить в тот же интерфейс), XDP_REDIRECT (другой интерфейс/в userspace). Программа исполняется в драйвере на самом раннем этапе приёма, до выделения sk_buff — по-этому она быстрее любого фильтра выше по стеку.

Вопрос 20. Почему SO_REUSEPORT критичен для многоядерных серверов?

Он позволяет нескольким независимым процессам/потокам связать один порт, и ядро распределяет входящие соединения между ними (по хэшу 4-кортежа). Это снимает конфликт bind и даёт линейное масштабирование accept-а по ядрам — стандартная архитектура nginx-style воркеров и любых высоконагруженных серверов на Linux.

Вопрос 21. Где в пути пакета задержки, которых «не видно» приложению?

Очереди qdisc на egress, буферы сокета (send буфер — «отправили» не значит «улетело»), delayed ACK/Nagle, кольца NIC и NAPI на приёме, очереди conntrack. Приложение видит только syscall; разницу показывает end-to-end замер и eBPF-трассировка ядра.

Вопрос 22. Почему «приложение тормозит только через VPN» — чаще всего про MTU?

Инкапсуляция VPN уменьшает эффективный MTU пути; TCP при DF=1 полагается на PMTUD, и если ICMP fragmentation needed отфильтрован, большие сегменты тихо теряются: маленькие пакеты (handshake, ping) ходят, большие зависают. Лечение — MSS clamp, открытие ICMP type 3 code 4, настройка MTU туннеля.

E.4. Сквозные вопросы: связать блоки курса

Вопрос 23. Проследите один и тот же ping от send() до ответа, назвав задействованные структуры каждой недели курса.

Приложение формирует дейтаграмму (сокеты, неделя 7/16); стек TCP/IP строит IP-пакет, выбирает маршрут LPM-поиском (неделя 6) и разрешает соседа через ARP-кэш (неделя 4); кадр уходит в qdisc (неделя 12) и в кольцо NIC; XDP-программа (неделя 15), если есть, может отбросить его на входе; ответ проходит те же слои в обратном порядке. Названия структур: sk_buff, conntrack-запись, ARP-кэш, таблица маршрутов — полный «словарь» курса в одном ответе.

Вопрос 24. Почему один и тот же сервис «работает локально и не работает в корпоративной сети»? Назовите пять причин из разных блоков.

Пример ответа: (1) ACL/сегментация режет порт (неделя 11); (2) MTU/PMTUD-дыра на VPN (неделя 5); (3) NAT без DNAT для входящих (неделя 11); (4) sticky/прокси кэширует старый контент (неделя 14); (5) DNS раздаёт внутренний адрес (неделя 13). Ценность ответа — в умении перечислять причины по уровням, а не «гадать».

Вопрос 25. Что общего у ARP-кэша, DNS-кэша, HTTP-кэша и conntrack?

Все — кэши с TTL и проблема недействительности: ARP-запись устаревает при смене карты (gratuitous ARP), DNS — при миграции сервиса (TTL), HTTP — при обновлении файла (Cache-Control, ETag), conntrack — при таймауте трансляции (keepalive). Одна и та же инженерная идея в четырёх слоях — курс специально показывает её повторение.

Вопрос 26. Какой механизм курса вы внедрили бы в свой учебный проект первым и как бы измерили эффект?

Правильных ответов нет; оценивается связка «механизм — метрика — инструмент»: например, TCP_NODELAY для чата (метрика: p99 латентности, инструмент: eBPF-гистограммы send), SO_REUSEPORT для воркеров (RPS до/после, wrk), keep-alive для HTTP-клиента (число handshake в дампе). Ответ «настроил бы всё сразу» — неправильный: изменение нужно измерять по одному.

ТЕРМИНЫ В АЛФАВИТНОМ ПОРЯДКЕ

Приложение Ж. Глоссарий курса

Термин	Определение
ARP	Протокол разрешения IP-адреса в MAC-адрес в пределах сегмента L2 (RFC 826).
BGP	Протокол маршрутизации между автономными системами; «система навигации» интернета.
bufferbloat	Рост задержки из-за длинных очередей на узком канале; лечится fq_codel/AQM.
cwnd	Окно перегрузки TCP — сколько байт отправитель может «в полёте»; управляется AIMD/CUBIC/BBR.
CIDR	Бесклассовая адресация: запись сети X.Y.Z.W/p с явной длиной префикса.
conntrack	Таблица соединений ядра Linux; основа NAT и stateful-файрволов.
DSCP	6-битное поле QoS-метки в IP-заголовке (EF, AFxy, CSx).
eBPF	Виртуальная машина ядра для безопасных программ: фильтрация, трассировка, телеметрия.
EtherType	Поле кадра Ethernet, определяющее тип нагрузки (0x0800 IPv4, 0x0806 ARP).
FSM (TCP)	Машина состояний TCP-соединения: LISTEN, SYN_SENT, ESTABLISHED, TIME_WAIT и др.
handshake	Трёхшаговое установление TCP-соединения: SYN → SYN-ACK → ACK.
hop (хоп)	Один транзитный узел на пути пакета; счёт хопов ведёт TTL.
ICMP	Служебный протокол сетевого уровня: echo, time exceeded, unreachable.
инкапсуляция	Оборачивание данных заголовками уровней при движении вниз по стеку.
keep-alive (TCP)	Зонды TCP для проверки живости простаивающего соединения; не путать с HTTP keep-alive.
LPM	Longest prefix match — поиск наиболее специфичного совпавшего маршрута.
MAC-адрес	48-битный адрес узла на канальном уровне; OUI + серийный номер.
maps (BPF)	Структуры данных для обмена между eBPF-программами и userspace (array, hash, LPM).
MSS	Maximum segment size — максимум полезной нагрузки TCP-сегмента (обычно MTU−40).

Термин	Определение
MTU	Максимум полезной нагрузки кадра канального уровня (Ethernet: 1500).
Nagle	Алгоритм группировки мелких TCP-записей; отключается TCP_NODELAY.
NAPI	Механизм ядра: обработка пакетов из кольца NIC в softirq вместо потока прерываний.
NAT	Трансляция адресов/портов на границе сетей; SNAT/DNAT, таблица conntrack.
netem	Дисциплина tc для эмуляции задержки, потерь, джиттера в тестах.
PMTUD	Path MTU Discovery — подбор MTU пути через ICMP fragmentation needed.
reverse-proxy	Сервер-представитель перед бэкендами: TLS, кэш, маршрутизация, лимиты.
RTO	Таймаут ретрансмиссии TCP; оценивается по SRTT и отклонению RTT.
RTT	Время кругового прохода пакета; измеряется ping и стеком TCP.
rwnd	Окно получателя TCP — управление потоком со стороны принимающей стороны.
sk_buff	Буфер пакета в ядре Linux; основная структура сетевого стека.
SO_REUSEPORT	Опция сокета: несколько процессов слушают один порт, ядро балансирует.
sticky-session	Прикрепление клиента к одному бэкенду (cookie/source-hash) при балансировке.
SYN cookies	Безсерверное (без состояния) установление соединения против SYN flood.
TCP_NODELAY	Отключение Nagle: отправка мелких записей немедленно.
TFTP	Простейший файловый протокол поверх UDP с блоками и ACK (RFC 1350).
TIME_WAIT	Состояние TCP после активного закрытия; страховка от заблудившихся сегментов.
tracepoint	Статическая точка трассировки ядра; хук для eBPF/bpftrace.
TTL	Счётчик хопов IP; страховка от петель, топливо traceroute.
verifier (eBPF)	Статический проверяющий eBPF-программ: границы памяти, завершимость, типы.
VLAN	Виртуальная LAN: изоляция сегментов тегами 802.1Q внутри одной коммутационной фабрики.
XDP	Программируемая обработка пакетов на входе драйвера, до sk_buff.
ARP-кэш	Таблица соответствий IP→MAC с таймерами (REACHABLE/STALE/DELAY).
qdisc	Дисциплина очередей tc: netem, htb, fq_codel — управление исходящим трафиком.
домен коллизий	Сегмент, где передатчики мешают друг другу; разрезается коммутатором.
широковещ. домен	Область доставки broadcast-кадров; разрезается роутером/VLAN.
PDU	Protocol data unit: кадр (L2), пакет (L3), сегмент (TCP), дейтаграмма (UDP).
veth	Пара виртуальных Ethernet-интерфейсов, соединённых «проводом»; кирпичи netns-стендов.
netns	Network namespace: изолированный сетевой стек внутри одного ядра Linux.
ECN	Explicit Congestion Notification: маркировка перегрузки вместо отброса пакета.
NIC / кольцо	Сетевая карта и круговой буфер дескрипторов DMA, из которого NAPI вычитывает пакеты.
softirq / NAPI	Механизм отложенной обработки прерываний ядра; так пакет попадает в стек без шторма прерываний.
zero-copy	Передача данных без промежуточных копий (sendfile, splice); экономит CPU и латентность.

Термин	Определение
backpressure	Обратное давление: способ «тормознуть» отправителя при заполненном буфере (окно, очередь, 503).
CO-RE	Compile Once — Run Everywhere: переносимость eBPF-программ между ядрами через BTF.
bpftrace	Высокоуровневый трассировщик на eBPF: однострочники для наблюдаемости ядра.
power-of-two	Схема выбора из двух случайных бэкендов: дёшево приближает least-loaded балансировку.
MSS clamp	Перезапись MSS в SYN, чтобы сегменты влезали в уменьшенный MTU пути (VPN, PPPoE).

Глоссарий

ПЕРЕД PUSH, ПЕРЕД ОТЧЁТОМ, ПЕРЕД ДОКЛАДОМ

Приложение З. Чек-листы сдачи

З.1. Чек-лист программного задания (hw-N)

- Локально: cmake -B build && cmake --build build -j && ctest --test-dir build --output-on-failure — всё зелёное в Debug и Release (gcc и clang, если есть).
- Интерфейс include/*.h не менялся; код — только в src/ и tests/ (последнее — только если разрешено).
- README обновлён: что сделано, что не успело, ответы на вопросы «для размышления».
- Коммиты осмысленные («fix overflow in parse» — хорошо; «asdf» — плохо), в main, CI зелёный до дедлайна.
- Код перечитан глазами: нет закомментированных «запасных» вариантов, printf-мусора, магических чисел без имён.
- Готов ответить на вопросы защиты: почему такая структура данных, что будет при входе X, где могут быть утечки.

З.2. Чек-лист лабораторной работы

- Схема стенда актуальна (совпадает с тем, что реально собрано), адресация подписана.
- Скриншоты сделаны после завершения работы, выводы команд — полные (не обрезаны окном).
- Каждый скриншот имеет подпись: что доказывает этот вывод.
- Отрицательные проверки выполнены (что должно НЕ работать — и не работает).
- В выводах — три конкретных предложения с инженерным смыслом, а не «мы научились работать с сетью».

З.3. Чек-лист доклада по мини-исследованию

- Первый слайд/абзац отвечает: что исследуем и зачем это инженеру.
- Методика воспроизводима: версии ПО, железо, конфиги, команды замера перечислены.
- Результаты — числа и графики с подписанными осями, минимум три прогона, медианы.
- Есть неожиданная находка или ограничение — доклад без «всё как в учебнике» скучен.
- Выводы отвечают на вопрос цели, а не «тема изучена».
- Репетиция уложила выступление в 7 минут (проверьте секундомером).

ПОШАГОВЫЕ ИНСТРУКЦИИ ДЛЯ СТУДЕНТОВ

Приложение И. Лабораторные работы: полные постановки

В главах 6–7 лабораторные описаны в объёме семинара; здесь — рабочие инструкции «сделай шаг за шагом» с конфигурациями устройств. Студент обязан понять каждый шаг: защита лабы включает вопрос «почему именно эта команда». Скриншоты и выводы оформляются в отчёт по чек-листу З.2.

И.1. Лаба PT-1: VLAN и router-on-a-stick (неделя 10)

Топология: роутер 2911, два коммутатора 2960 (SW1, SW2), шесть ПК: PC-О1..О2 (VLAN 10 «Офис»), PC-С1..С2 (VLAN 20 «Серверы»), PC-Г1..Г2 (VLAN 30 «Гости»). Адресация: 192.168.10.0/24, 192.168.20.0/24, 192.168.30.0/24. Роутер: 192.168.N.1 в каждой сети.

1. Собрать физику: ПК → SW (FastEthernet), SW1 ↔ SW2 и SW1 ↔ R1 — GigabitEthernet (это будущие транки).
2. На обоих коммутаторах создать VLAN: vlan 10 / name OFFICE, vlan 20 / name SERVERS, vlan 30 / name GUESTS; проверить show vlan brief.
3. Назначить access-порты: interface range fa0/1 - 2 → switchport mode access; switchport access vlan 10; аналогично порты серверов (20) и гостей (30) на своём SW.
4. Транки: на SW1 gi0/1 (к SW2) и gi0/2 (к R1) — switchport mode trunk; switchport trunk allowed vlan 10,20,30. На SW2 — gi0/1 аналогично.
5. Роутер: подынтерфейсы gi0/0.10/20/30 с encapsulation dot1Q 10/20/30 и адресами .1; interface gi0/0 no shutdown. Конфиг подынтерфейсов — в семинаре 10.
6. Настроить адреса ПК вручную (Desktop → IP Configuration) по таблице; шлюз — .1 своей сети.
7. Проверки: ping внутри VLAN; ping между VLAN (через роутер); show interfaces trunk; show ip route на R1.
8. Режим Simulation: отправить ping О1→С1, раскрыть кадр на транке — найти 802.1Q тег; скриншот.
9. Отчёт по З.2: схема, таблицы VLAN/портов, выводы команд, скриншоты, выводы.

Типовые проблемы: забыт no shutdown на gi0/0 роутера (линк «up», подынтерфейсы молчат); access-порт вместо транка между SW (пингует только один VLAN); ошибка в allowed vlan (ping работает через один SW и падает через второй). Все три — обязательные кейсы разбора на защите.

И.2. Лаба PT-2: DHCP, NAT и ACL (неделя 11)

Топология: продолжение И.1 + внешний сегмент «Интернет»: ISP-роутер 2911 и веб-сервер Server-Web (203.0.113.10/24, шлюз 203.0.113.1). Связь R1↔ISP: 198.51.100.0/30, у R1 — 198.51.100.2 (внешний интерфейс gi0/1), у ISP — .1 и маршрут по умолчанию в никуда (ISP «интернет»: ip route 0.0.0.0 0.0.0.0 через null0 не нужен — PT допускает отсутствие маршрута при симуляции; на R1: ip route 0.0.0.0 0.0.0.0 198.51.100.1).

1. DHCP на R1: ip dhcp pool OFFICE → network 192.168.10.0 255.255.255.0; default-router 192.168.10.1; dns-server 203.0.113.10. Аналогично GUESTS. Серверы оставить статикой.
2. На ПК переключить адресацию на DHCP (IP Configuration → DHCP); проверить show ip dhcp binding на R1; скриншот DORA в Simulation (фильтр по DHCP/UDP).
3. NAT: доступ-лист «кто маскарадится»: access-list 10 permit 192.168.10.0 0.0.0.255; access-list 10 permit 192.168.30.0 0.0.0.255 (серверы — без NAT, мы же хотим видеть изоляцию). Внешний интерфейс: interface gi0/1 → ip nat outside; внутренние gi0/0.10/20/30 → ip nat inside. Транслируем: ip nat inside source list 10 interface gi0/1 overload.
4. Маршрут по умолчанию на R1 (см. выше); проверить ping O1 → 203.0.113.10; show ip nat translations во время ping — скриншот.
5. ACL: (а) гости только в интернет: access-list 130 permit ip 192.168.30.0 0.0.0.255 any; access-list 130 deny ip any 192.168.10.0 0.0.0.255; ... применить ip access-group 130 in на gi0/0.30; (б) офис → серверы только HTTPS: access-list 120 permit tcp 192.168.10.0 0.0.0.255 host 192.168.20.10 eq 443; access-list 120 deny ip any any; на gi0/0.10 in.
6. Отрицательные проверки: ping Г1→С1 (падает), http Г1→С1 (падает), https О1→С1 (работает, если на сервере включён HTTPS-сервис).
7. show access-lists с ненулевыми счётчиками совпадений — скриншот в отчёт.
8. Выводы: что произойдёт при выходе из строя gi0/1; как вынести DHCP с роутера (мостик к relay).

Типовые проблемы: NAT не работает без ip route 0.0.0.0 (трафик не «доходит» до outside); ACL по in/out перепутан (правило надо вешать на входящий интерфейс источника); DNS в DHCP-пуле забыт — сайты по имени не открываются, по IP открываются (отличный учебный момент).

И.3. Лаба QoS: tc/netem в netns (неделя 12)

Стенд: demos/nsmesh_full.sh (приложение Д.3). Канал роутера v-l-r ограничивается и делится на классы; замеры — iperf3 (устанавливается пакетом) и ping. Полный скрипт классов — в семинаре 12 (qdemo.sh). Пошагово:

1. Собрать стенд; убедиться в связности и отсутствии лишних qdisc (tc qdisc show).
2. База: iperf3 -s на left, iperf3 -c с right, -t 10; записать итог (должен быть ограничен только CPU/loopback-эквивалентом).
3. Конкуренция: два iperf3-клиента по -P 8 параллельно; записать сумму и распределение.
4. Включить qdemo.sh (htb 10mbit, класс EF prio 0 и BE prio 7); повторить замер: один клиент с маркировкой (iperf3 -S 0xB8), второй без.
5. Латентность: ping -i 0.2 во время закачки до и после QoS; записать min/avg/max.
6. Доп. эксперимент: netem delay 20ms на класс BE (tc filter/класс-leaf: tc qdisc add dev ... parent 1:30 handle 30: netem delay 20ms); проверить, что EF-поток латентность не получил.
7. Снять tc -s class show dev v-l-r; в отчёт — таблицу «конфигурация → throughput каждого потока → RTT», два графика (до/после).

Критерии: замеры с тремя прогонами и медианами; графики; вывод о том, кто заплатил за приоритет EF. Ошибка-ловушка: помечать DSCP на wrong-интерфейсе (метка должна быть выставлена до входа в классифицирующий qdisc — в стенде это вход left).

И.4. Лаба HAProxy: балансировка и отказоустойчивость (неделя 14)

Стенд: тот же nsmesh; бэкенды — два namespace (app1 10.200.1.2, app2 10.200.2.2), в каждом простой HTTP-сервер: python3 -m http.server 8000 в каталоге со своим index.html («APP1» / «APP2»), либо собственный hw-13-сервер. HAProxy ставится на router (apt install haproxy); конфиг — из семинара 14. Вместо netns допускается docker-compose (два сервиса-бэкенда, haproxy, nginx по желанию) по согласованию с преподавателем.

1. Настроить и запустить haproxy.cfg (haproxy -c -f проверка, systemctl restart haproxy).
2. Балансировка: 8 раз curl http://10.0.1.1:8080 — ответы чередуются; переключить на leastconn, повторить; на source — убедиться в «прилипании» одного клиента.
3. Sticky: curl -c jar.txt первый запрос, затем curl -b jar.txt — ответ от того же бэкенда; показать cookie SRV в -v выводе.
4. Отказы: kill -9 python3 на app1; наблюдать /stats (вывод сервера DOWN), curl продолжает работать (app2); вернуть app1 — RISE в статистике.
5. Замеры: wrk -t4 -c64 -d30s http://10.0.1.1:8080: (а) один бэкенд, (б) два, (в) два с sticky; записать RPS и p99 (wrk -L).
6. Отчёт: конфиги, скриншоты /stats до/во время/после отказа, таблица замеров, выводы (где упёрлись: haproxy? python? netns?).

Типовые проблемы: timeout connect 3s слишком мал для python-сервера под нагрузкой (научиться отличать «бэкенд медленный» от «конфиг жёсткий»); забыт check у серверов — отказ не обнаруживается; /stats не открыт — включить listen stats из семинара 14.

ОПОРНЫЕ БИЛЕТЫ ИТОГОВОГО ОПРОСА (СЕМИНАР 16)

Приложение К. Примеры итоговых билетов

Итоговый опрос проводится письменно (20 минут, до 6 баллов активности). Билет состоит из четырёх вопросов: по одному из блоков I–II, III, IV–V и VI + одна практическая мини-задача. Ниже — три опорных билета; остальные формируются из контрольных вопросов приложения Е и вопросов обсуждения глав 3–8. Критерии: полный ответ 1.5 б., существенный пропуск −0.5 б., ошибка, меняющая смысл, −1 б.

Билет 1

- Блок I–II: чем домен коллизий отличается от широковещательного и что их режет.
- Блок III: нарисовать handshake TCP с абсолютными seq/ack; зачем третий шаг.
- Блок IV–V: объяснить работу NAT через conntrack; почему без DNAT входящий пакет отбрасывается.
- Блок VI: перечислить действия XDP и объяснить, почему программа быстрее nftables.
- Практика: по выводу «ss -tan | grep :443 | wc -l = 2000, из них 1990 CLOSE_WAIT» назвать вероятную причину и что проверить в коде.

Билет 2

- Блок I–II: что покажет ip neigh show и как устроены состояния ARP-кэша.
- Блок III: сравнить UDP и TCP для голосового трафика; что потеряем/выиграем.
- Блок IV–V: почему policer хуже shaper для TCP; что делает DSCP EF с голосом.
- Блок VI: путь пакета от NIC до recv(): NAPI, sk_buff, буфер сокета; где очереди.
- Практика: «сайт по HTTPS не открывается за VPN, ping идёт» — гипотеза, план проверки (MTU/PMTUD), какие команды дать.

Билет 3

- Блок I–II: посчитать LPM для адреса 10.1.2.3 по таблице из пяти маршрутов (таблица выдаётся с билетом).
- Блок III: почему потери рушат throughput TCP; объяснить через AIMD и RTO.
- Блок IV–V: sticky-сессии: два механизма HAProxy; почему правильнее вынести состояние.
- Блок VI: верификатор eBPF: что он проверяет; почему это важно для стабильности ядра.
- Практика: по дампу (выдаётся pcap, 3 пакета) определить: был ли handshake, кто закрыл соединение, был ли RST.

ТАНЕНБАУМ (6-Е ИЗД.) + ДОПОЛНЕНИЯ, ЧАСОВ НА НЕДЕЛЮ

Приложение Л. Маршрут чтения по неделям

Норма самостоятельной работы — 2–3 часа в неделю, из них чтение примерно половина. Ссылки на главы Таненбаума даны по 6-му изданию; для 5-го издания соответствие: гл. 4 (6-е) ≈ гл. 4 (5-е), сетевой уровень — гл. 5 (6-е) ≈ гл. 5 (5-е), транспортный — гл. 6 (6-е) ≈ гл. 6 (5-е).

Нед.	Таненбаум, 6-е изд.	Дополнения	Часов
1	Гл. 1: использование сетей; эталонные модели (1.4)	RFC 1122 — обзорно	2
2	–	man ip, man tcpdump, man ss; wiki tcpdump-фильтров	2.5
3	Гл. 4: Ethernet, коммутаторы (4.3), VLAN (4.8)	IEEE 802.1Q — обзор	2.5
4	Гл. 4: MAC-подур. (CSMA/CD), остовные деревья; гл. 3: ARP	RFC 826 (целиком)	2.5
5	Гл. 5: IPv4, адресация (5.6), CIDR	RFC 1918; RFC 791 — разбор полей	3
6	Гл. 5: алгоритмы маршрутизации (5.2)	man 8 ip-route; OSPF/BGP обзорно	2.5
7	Гл. 6: UDP (6.2), сокетное API	RFC 768; RFC 1350 (TFTP)	3
8	Гл. 6: TCP (6.5): установление, завершение, FSM	RFC 9293 — разделы handshake/FIN	3
9	Гл. 6: управление окном, контроль перегрузки	RFC 5681; обзор BBR (статья Google)	3
10	Гл. 4: VLAN-повторение; гл. 5: внутренняя организация роутеров	Документация PT	2
11	–	RFC 2131 (DHCP); man nft; conntrack(8)	2.5
12	Гл. 4: качества обслуживания (если есть в издании) или гл. 5 QoS-разделы	man tc; fq_codel article	2.5
13	Гл. 7: DNS (7.1, 7.5), WWW/HTTP	RFC 9110 обзор; RFC 8446 обзор	2.5
14	–	HAProxy docs: balance, cookie, check; nginx proxy_pass	2.5
15	–	Documentation/bpf (введение, XDP); Borkmann talks	2.5
16	Повторение гл. 1–7 по своим конспектам	Стивенс: главы о сокетах — по желанию	3

Карта чтения

Вопросы из чтения обязательно выносятся в микроквиз следующего семинара — это поддерживает ритм самостоятельной работы. Если студент отстаёт по чтению, минимальный «выживательный» набор: разделы про модели (1), Ethernet/VLAN, IP/CIDR, TCP целиком, DNS и HTTP-обзор.

SRC/ETHERNET.C · ПОЛНЫЙ ЛИСТИНГ (ПОСЛЕ ДЕДЛАЙНА)

Приложение M. Эталонное решение hw-3

Эталон публикуется в репозитории курса после закрытия дедлайна hw-3 и служит двум целям: разбор на семинаре 4 и стартовая точка для hw-5 (структура парсера совпадает — меняется только заголовок). Студентам предлагается сравнить своё решение с эталоном и выписать три отличия, которые считают улучшением или ухудшением, — это зачитывается как микродоклад.

src/ethernet.c — эталонное решение (полностью)

```
/* src/ethernet.c — эталон hw-3: парсер/сериализатор Ethernet-кадра. */
#include "ethernet.h"
#include <string.h>
#define ET_VLAN_8021Q 0x8100
#define ET_VLAN_9100  0x88A8          /* провайдерский S-tag (QinQ) */
static uint16_t rd16(const uint8_t *p)             /* big-endian */
{
    return (uint16_t)((uint16_t)p[0] << 8 | p[1]);
}
static void wr16(uint8_t *p, uint16_t v)
{
    p[0] = (uint8_t)(v >> 8);
    p[1] = (uint8_t)(v & 0xFF);
}
/* Читает 0..2 VLAN-тегов начиная с оффсета 12. После вызова *off
 * указывает на поле данных, *etype — эффективный тип нагрузки.
 * Семантика QinQ: *outer_vlan = внешний (первый) тег, *vlan — внутренний. */
static int read_tags(const uint8_t *buf, size_t len, size_t *off,
                     uint16_t *etype, int *vlan, int *pcp, int *outer)
{
    uint16_t t = rd16(buf + *off);
    *vlan = -1; *pcp = 0; *outer = -1;
    if (t != ET_VLAN_8021Q && t != ET_VLAN_9100) {  /* тегов нет */
        *etype = t;
        return 0;
    }
    if (*off + 4 > len)                             /* обрезанный тег */
        return -1;
    uint16_t tci = rd16(buf + *off + 2);
    *vlan = tci & 0x0FFF;
    *pcp  = (tci >> 13) & 0x07;
    *off += 4;
    t = rd16(buf + *off);
    if (t == ET_VLAN_8021Q || t == ET_VLAN_9100) {  /* QinQ */
        if (*off + 4 > len)
            return -1;
        *outer = *vlan;                             /* внешний = первый */
        tci = rd16(buf + *off + 2);
        *vlan = tci & 0x0FFF;                       /* внутренний = второй */
        *off += 4;
        t = rd16(buf + *off);
    }
    *etype = t;
    return 0;
```

```
}
int eth_parse(const uint8_t *buf, size_t len, eth_frame_t *out)
{
    if (!buf || !out || len < 14)
        return -1;
    eth_frame_t f;
    memcpy(f.dst, buf, 6);
    memcpy(f.src, buf + 6, 6);
    size_t off = 12;
    if (read_tags(buf, len, &off, &f.ethertype,
                  &f.vlan, &f.pcp, &f.outer_vlan))
        return -1;
    f.payload     = buf + off;          /* указатель ВНУТРЬ буфера */
    f.payload_len = len - off;
    if (f.payload_len < ET_PAYLOAD_MIN || f.payload_len > ET_PAYLOAD_MAX)
        return -1;
    *out = f;
    return 0;
}
int eth_serialize(const eth_frame_t *f, uint8_t *buf, size_t len)
{
    if (!f || !buf)
        return -1;
    /* pcp/outer без vlan — противоречивая запись */
    if (f->vlan < 0 && (f->pcp != 0 || f->outer_vlan >= 0))
        return -1;
    if (f->vlan >= 0 && (f->vlan > 0x0FFF || f->pcp > 7))
        return -1;
    if (f->payload_len < ET_PAYLOAD_MIN || f->payload_len > ET_PAYLOAD_MAX)
        return -1;
    size_t ntags = (f->vlan >= 0 ? 1u : 0) + (f->outer_vlan >= 0 ? 1u : 0);
    size_t total = 14 + 4 * ntags + f->payload_len;
    if (len < total || total > 65535)
        return -1;
    memcpy(buf, f->dst, 6);
    memcpy(buf + 6, f->src, 6);
    size_t off = 12;
    if (f->outer_vlan >= 0) {                       /* внешний тег первым */
        wr16(buf + off, ET_VLAN_8021Q);
        wr16(buf + off + 2, (uint16_t)((f->pcp << 13) | f->outer_vlan));
        off += 4;
    }
    if (f->vlan >= 0) {
        wr16(buf + off, ET_VLAN_8021Q);
        wr16(buf + off + 2, (uint16_t)((f->pcp << 13) | f->vlan));
        off += 4;
    }
    wr16(buf + off, f->ethertype);
    off += 2;
    memcpy(buf + off, f->payload, f->payload_len);
    return (int)total;
}
```

Замечания к решению, которые проговариваются на разборе: чтение полей — только через rd16/wr16 (big-endian и отсутствие привязки к выравниванию); QinQ обнаруживается «схлопыванием» тегов в

цикле по двум максимумам; сериализация пишет внешний тег первым; упрощение — PCP копируется в оба тега QinQ, для честного S-тега понадобилось бы отдельное поле. Оценка сложности: O(n) по длине кадра, ноль динамической памяти.

RUNNING-CONFIG УСТРОЙСТВ · СВЕРЬТЕ СВОЙ СТАЕНД

Приложение Н. Полные конфигурации лаб PT-1/PT-2

Конфигурации соответствуют топологиям приложения И. Рекомендуемый порядок работы: собрать по инструкции, сверить проблемные места с этим приложением, но НЕ копировать целиком до защиты — на защите спрашивают смысл каждой строки. В листинге R1 части недели 10 (PT-1) и недели 11 (PT-2) разделены комментариями.

SW1: VLAN, access-порты, транки

```
hostname SW1
!
vlan 10
 name OFFICE
vlan 20
 name SERVERS
vlan 30
 name GUESTS
!
interface range FastEthernet0/1 - 2
 switchport mode access
 switchport access vlan 10
interface range FastEthernet0/3 - 4
 switchport mode access
 switchport access vlan 20
interface range FastEthernet0/5 - 6
 switchport mode access
 switchport access vlan 30
!
interface GigabitEthernet0/1
 switchport mode trunk
 switchport trunk allowed vlan 10,20,30
interface GigabitEthernet0/2
 switchport mode trunk
 switchport trunk allowed vlan 10,20,30
```

R1: роутер-на-палочке (неделя 10) + DHCP, NAT, ACL (неделя 11)

```
hostname R1
!
! ===== Неделя 10 (PT-1): маршрутизация между VLAN =====
interface GigabitEthernet0/0
 no shutdown
interface GigabitEthernet0/0.10
 encapsulation dot1Q 10
 ip address 192.168.10.1 255.255.255.0
interface GigabitEthernet0/0.20
 encapsulation dot1Q 20
 ip address 192.168.20.1 255.255.255.0
```

```
interface GigabitEthernet0/0.30
 encapsulation dot1Q 30
 ip address 192.168.30.1 255.255.255.0
!
! ===== Неделя 11 (PT-2): внешний линк, DHCP, NAT, ACL =====
interface GigabitEthernet0/1
 ip address 198.51.100.2 255.255.255.252
 ip nat outside
ip route 0.0.0.0 0.0.0.0 198.51.100.1
!
ip dhcp excluded-address 192.168.10.1 192.168.10.9
ip dhcp excluded-address 192.168.30.1 192.168.30.9
ip dhcp pool OFFICE
 network 192.168.10.0 255.255.255.0
 default-router 192.168.10.1
 dns-server 203.0.113.10
ip dhcp pool GUESTS
 network 192.168.30.0 255.255.255.0
 default-router 192.168.30.1
 dns-server 203.0.113.10
!
access-list 10 permit 192.168.10.0 0.0.0.255
access-list 10 permit 192.168.30.0 0.0.0.255
ip nat inside source list 10 interface GigabitEthernet0/1 overload
!
! Гости: в офис/серверы нельзя, наружу можно. Порядок правил решает!
access-list 130 deny   ip 192.168.30.0 0.0.0.255 192.168.10.0 0.0.0.255
access-list 130 deny   ip 192.168.30.0 0.0.0.255 192.168.20.0 0.0.0.255
access-list 130 permit ip 192.168.30.0 0.0.0.255 any
!
! Офис: к серверам только HTTPS, остальное (в т.ч. интернет) — можно
access-list 120 permit tcp 192.168.10.0 0.0.0.255 host 192.168.20.10 eq 443
access-list 120 deny   ip  192.168.10.0 0.0.0.255 192.168.20.0 0.0.0.255
access-list 120 permit ip any any
!
interface GigabitEthernet0/0.10
 ip nat inside
 ip access-group 120 in
interface GigabitEthernet0/0.20
 ip nat inside
interface GigabitEthernet0/0.30
 ip nat inside
 ip access-group 130 in
```

ISP: внешний сегмент

```
hostname ISP
interface GigabitEthernet0/0
 ip address 198.51.100.1 255.255.255.252
 no shutdown
interface GigabitEthernet0/1
 ip address 203.0.113.1 255.255.255.0
 no shutdown
! Server-Web: 203.0.113.10/24, шлюз 203.0.113.1
```

Проверка корректности ACL-порядка: поменяйте местами permit any и deny-строки списка 130 — гости снова получат доступ к серверам, хотя «deny есть». Это лучший демонстрационный эксперимент недели

11: ACL исполняется сверху вниз до первого совпадения.

ВСЁ, ЧТО НУЖНО ДЛЯ hw-7, hw-9, hw-13

Приложение О. Мини-справочник сокетного API

Вызов	Назначение	Замечания курса
socket(domain, type, proto)	Создать дескриптор	SOCK_STREAM/TCP, SOCK_DGRAM/UDP
bind(fd, addr, len)	Привязать локальный адрес	Серверам; SO_REUSEADDR — перед bind
listen(fd, backlog)	TCP: режим приёма	backlog — очередь полусоединений/accept
accept(fd, ...)	Забрать соединение	Новый fd на каждую сессию
connect(fd, addr, len)	Установить соединение	На UDP лишь фиксирует адресата
send/recv (write/read)	Обмен по соединению	TCP — поток: границы записей не сохраняются
sendto/recvfrom	Обмен дейтаграммами	UDP: адрес партнёра в каждом вызове
shutdown(fd, SHUT_WR)	Полузакрытие	FIN без закрытия fd; сигнализирует конец данных
close(fd)	Закрыть сокет	FIN; SO_LINGER=0 — немедленный RST
setsockopt SO_RCVTIMEO	Таймаут приёма	Альтернатива select/poll для простых клиентов
setsockopt TCP_NODELAY	Отключить Nagle	Первый совет для интерактивных протоколов
setsockopt SO_REUSEADDR	Рестарт без ожидания	Обходит TIME_WAIT на локальном порту
setsockopt SO_REUSEPORT	Много accept-еров	Ядро балансирует соединения по процессам
select/poll	Мультиплексирование fd	O(n) по дескрипторам; хватит для учебных задач
epoll_create/ctl/wait	Масштабный мультиплексор	O(активных); edge vs level-triggered
getsockopt SO_ERROR	Итог async connect	Проверять после готовности на запись

Системные вызовы и опции сокетов

Типовые грабли, на которые ссылается ревью: чтение заголовка «одним recv» (TCP — поток, запрос приходит кусками: накапливайте буфер и ищите границу запроса, как в hw-13); забытый check кода возврата send (частичная отправка — норма, доотправляйте остаток); блокирующий recv без таймаута в протоколах с ретрансмиссией (hw-7/hw-9) — используйте SO_RCVTIMEO или select с таймаутом.

ЕДИНАЯ СТРУКТУРА ДОКУМЕНТА

Приложение П. Шаблон отчёта по лабораторной и исследованию

Отчёты лаб и мини-исследований оформляются по одной структуре — это упрощает проверку и приучает к формату технических отчётов. Объём: лаба — 2–4 страницы, исследование — 4–6 (теоретические — до 8). Формат — PDF или Markdown, графики — PNG с подписями осей.

1. **Титул.** Название работы, номер варианта/темы, ФИО и группа, дата, ссылка на репозиторий/стенд (если есть).
2. **1. Постановка.** Что исследуем/настраиваем и по какому критерию признаём успех (измеримая метрика: «p99 при 2% потерь», «гости не пингуют серверы»).
3. **2. Стенд и методика.** Схема (скриншот/диаграмма), таблица адресации, версии ПО и железа, полный список команд замера — так, чтобы читатель мог воспроизвести. Листинги длинных конфигов — в приложение к отчёту.
4. **3. Ход работы.** Ключевые шаги и наблюдения; каждая нетривиальная команда с кратким «зачем». Скриншоты — только те, что доказывают вывод.
5. **4. Результаты.** Таблицы и графики. Замеры — три прогона, в таблицу — медианы; на графиках подписаны оси и единицы. Ничего «примерного» без пометки.
6. **5. Обсуждение.** Что означают результаты, где границы применимости, какие факторы не учтены; сравнение с теорией (что совпало, что нет — и почему).
7. **6. Выводы.** 3–5 пунктов, отвечающих на постановку; каждый вывод — инженерное утверждение, а не «мы многому научились».
8. **7. Источники.** RFC, документация, статьи — с указанием, что именно из них взято. Копипаст без ссылки — плагиат.

Частые замечания проверяющего: выводы не отвечают на постановку (вернитесь к разделу 1); скриншоты без подписей; «методика» без команд (воспроизвести невозможно); графики без единиц. Отчёт, прошедший чек-лист З.2, обычно свободен от всех четырёх замечаний.

КОРОТКИЕ ВЫСТУПЛЕНИЯ 3–5 МИНУТ · ДО 2 БАЛЛОВ АКТИВНОСТИ

Приложение Р. Микродоклады: темы по неделям

На каждый семинар — два-три микродоклада студентов. Тема берётся из таблицы ниже (или согласуется); доклад — это не пересказ Википедии, а «инженерная справка»: зачем это нужно, как работает, где ловушка, одна демонстрация или диаграмма. Регламент: 3–5 минут, затем два вопроса аудитории. Темы намеренно лежат «рядом» с темой семинара — они достраивают картину, не отрываясь от контекста.

Нед.	Тема доклада	Опора
1	История: почему TCP/IP победил OSI	Таненбаум гл. 1; статья Cerf & Kahn 1974
2	Что показывает и скрывает iperf3	man iperf3; демо на своём стенде
3	MAC-адреса: OUI, локальные адреса, MAC-randomization в Wi-Fi	IEEE OUI-база
4	SYN cookies: как сервер «помнит» без памяти	RFC 4987, раздел SYN flood
5	Почему адреса кончились: исчерпание IPv4 и рынки адресов	RFC 1918; статистика RIR
6	ECMP: почему у вас несколько маршрутов и как выбирается	документация ядра, FIB
7	QUIC: что это и почему всё внезапно UDP	RFC 9000, обзорная часть
8	SYN-proxy и SYN-flood: разбор одной атаки	Cloudflare Learning, tcpdump
9	BBR: как Google изменил congestion control	статья BBR (ACM Queue 2016)
10	VRRP/HSRP: зачем у шлюза «виртуальный» MAC	документация VRRPv2/v3
11	Почему FTP умер: NAT и ALG как антипаттерн	RFC 959 (исторически); conntrack
12	fq_codel и battle against bufferbloat	статья Codel, демо на стенде
13	DoH/DoT: DNS-приватность и её цена	RFC 8484; dig +tls
14	Канареечные релизы и percentage-based routing	HAProxy weighted балансировка
15	Katran: как Facebook балансирует через XDP	github katran, обзорная статья
16	Профессии: SRE, сетевой инженер, производительность — где чей курс	обзор вакансий

Банк тем микродокладов

Критерии микродоклада: конкретность (числа, имена, даты — 1 балл) и ответ на вопросы (1 балл). «Рассказал смысл» без конкретики — 1 балл. Один студент может взять максимум две темы за полугодие.

ЧЕСТНЫЕ КАРТИНКИ ЗА 10 МИНУТ

Приложение C. Графики для отчётов: правила и шаблон

Большинство отчётов курса содержит один-два графика (throughput, RTT, latency). Требования простые и неизменные: подписанные оси с единицами, легенда, единый масштаб для сравниваемых серий, векторный формат или PNG не ниже 150 dpi. Ниже — готовый шаблон gnuplot: правится одна строка данных — и график готов. Аналогичный результат даёт matplotlib/Python, если вам комфортнее скриптовать на нём.

Шаблон gnuplot: throughput двух конфигураций по времени

```
# plot.gp: gnuplot -e "load 'plot.gp'"
set terminal pngcairo size 900,420 enhanced font 'Arial,12'
set output 'throughput.png'
set xlabel 'Время, с'
set ylabel 'Пропускная способность, Мбит/с'
```

```
set grid ytics lt 0 lw 1 lc rgb '#c0d0e2'
set key top right box lw 0.5
set xrange [0:60]
plot 'cubic.dat' using 1:2 with lines lw 2 lc rgb '#1a4a7a' title 'CUBIC', \
     'bbr.dat'   using 1:2 with lines lw 2 lc rgb '#2d7ab3' title 'BBR'
# Формат .dat: <сек> <значение> построчно; один файл на серию.
```

- Один график — одна мысль; если серий больше трёх, разбейте на два графика.
- Никогда не сравнивайте серии в разных масштабах на одном рисунке (двойная ось — только с явной пометкой в подписи).
- Медианы и интервалы: если прогонов три, покажите все три точки или полосу min–max — честнее одной «средней».
- Скриншот терминала не заменяет график; текстовый вывод — в приложение отчёта.
- Заголовок графика дублирует подпись в отчёте («Рис. 2 — …») — оставьте заголовок в тексте отчёта, а не внутри картинки.

ЗАЯВКА НА ИССЛЕДОВАНИЕ · ВЕДОМОСТЬ · ЖУРНАЛ СДАЧ

Приложение Т. Формы документов курса

Т.1. Форма заявки на мини-исследование (сдаётся до недели 12)

research-application.md — шаблон заявки

```
# Заявка на мини-исследование
Студент: <ФИО, группа>
Тема: <из каталога или своя, согласованная>
Категория: прикладное / смешанное / теоретическое
## План (3–5 предложений)
Что именно будет сделано, на каком стенде, какие замеры или обзоры.
## Ожидаемый результат
Метрика/артефакт, по которому судят об успехе: для прикладных —
конкретная таблица замеров; для теоретических — разделы обзора.
## Инструменты и риски
Стенд (netns/PT/VM), ПО и версии; что может не сработать и план Б.
Дата, подпись. Решение преподавателя: согласовано / доработать.
```

Т.2. Ведомость курса (шаблон для ассистента)

Колонка	Формат
student	ФИО, группа
hw-1 … hw-15	набрано/максимум (например, «5/6»)
lab-pt1, lab-pt2, lab-qos, lab-haproxy	баллы + дата защиты
activity	микроквизы + микродоклады (расшифровка в комментарии)
research-report / research-talk	две компоненты по 15 и 10

Колонка	Формат
bonus	сумма бонусов со ссылками на PR
total / grade	сумма; порог зачёта 90, автомат 120; формулы — раздел 2.4
notes	поздние дедлайны, продления, инциденты

Колонки ведомости (CSV/таблица)

Т.3. Журнал сдач: что фиксируется автоматически

- Каждый push в репозиторий задания фиксируется CI: хэш, время, результат тестов — это и есть «штамп времени» сдачи (см. FAQ 2.7).
- Ассистент переносит в ведомость только итоговое состояние на момент дедлайна; история коммитов — доказательная база при спорах.
- Устные защиты отмечаются в ведомости датой и инициалами принимающего.

СТРУКТУРА И ПРАВИЛА РАБОТЫ

Приложение У. Учебный репозиторий курса

Репозиторий организации (faculty/net-course) — единая точка входа: отсюда студент попадает во все задания, здесь лежат демонстрации, шаблоны документов и эта методичка. Ассистент заведует выпуском заданий через GitHub Classroom; правила работы с репозиторием — обычный командный гигиенический минимум, который студенты потом переносят в свои проекты.

Структура репозитория курса

```
net-course/
├── README.md               # стартовая страница: как начать курс
├── syllabus/               # эта методичка (Markdown + PDF), карта курса
├── tasks/                  # шаблоны заданий hw-1…hw-15 (Use this template)
│   ├── hw-ethernet/        # эталон: см. приложение А
│   └── hw-*/               # остальные — та же структура
├── demos/                  # листинги приложения Д + Makefile
├── hidden/                 # ПРИВАТНО: скрытые тесты (faculty/hidden-*)
├── wiki/                   # разборы ревью, FAQ, глоссарий (редактируемый)
├── forms/                  # заявка на исследование, шаблоны отчётов (прил. П, Т)
└── .github/workflows/      # общий CI-профиль организации
```

- Правило веток: работа в feature-ветке, PR в main с линком на CI; прямые пуш-и в main разрешены, но не приветствуются — дедлайн фиксируется по последнему коммиту main.
- Issues: вопросы по заданиям задаются в issues соответствующего hw-* — ответ ассистента виден всем; «вопрос в личку» не получает приоритета.
- Wiki: обезличенные комментарии ревью (раздел 2.8) копируются ассистентом; студенты могут предлагать правки методички через PR — лучшие входят в следующую редакцию.
- Releases: после дедлайна задания публикуется релиз с эталонным решением и раскрытыми скрытыми тестами; на релиз ставится тег hw-N-final.

- Модель безопасности: токен скрытых тестов — secrets организации, выдаётся на год; приватные репозитории не форкаются наружу, участники — только преподавательский состав.

У.2. Сопровождение методички (для кафедры)

Методичка — живой документ: её ценность падает, если листинги отстают от актуальных версий инструментов, а банк тем исследования не обновляется. Ниже — минимальный годовой цикл сопровождения, вынесенный из опыта ведения курса; он рассчитан на одного преподавателя и занимает 2–3 дня в год.

- **После каждого семестра.** Собрать правки студентов (PR в syllabus/), дополнить главу 10 новыми «граблями», выпустить релиз методички с тегом версии; зафиксировать, какие темы вызвали наибольшие затруднения.
- **Ежегодная сверка.** Обновить карту курса (даты, баллы), листинги демо прогнать на актуальном Ubuntu LTS, пересмотреть дедлайны под академический календарь; проверить ссылки приложения Г.
- **Скрытые тесты.** Ротировать не менее трети скрытого набора ежегодно: задачи с фиксированным интерфейсом обрастают «шпаргалками» прошлых лет, и неротируемый набор перестаёт отличать понимание от копирования.
- **Темы исследований.** Две-три темы каталога (глава 9) обновлять под актуальность (новые протоколы, свежие инциденты), удалять темы, не набравшие интереса два года подряд.
- **Обратная связь.** Анонимный опрос на семинаре 16 (три вопроса: что углубить, что сократить, чего не хватило) — результаты напрямую превращаются в правки следующей редакции.